

國立中正大學

資訊工程研究所碩士論文

採用 16 奈米技術實現之用於即時表情
識別應用的低功耗階層式卷積神經網路
硬體加速器

**Low-Power Hierarchical CNN
Hardware Accelerator Using 16nm
Technology for Real-Time Facial
Expression Recognition Applications**

研究生：許成誼

指導教授：鍾菁哲 博士

中華民國 一 一 三 年 八 月



國立中正大學碩士學位論文考試審定書

資訊工程學系

研究生 許成誼 所提之論文

採用16奈米技術實現之用於即時表情識別應用的低功耗階層式卷積
神經網路硬體加速器

經本委員會審查，符合碩士學位論文標準。

學位考試委員會
召集人

李順裕 簽章

委員

黃崇勳

李順裕

蔡鐸

鍾景哲

指導教授

鍾景哲

簽章

中華民國 113 年 6 月 27 日

摘要

近年來，隨著 AI 熱潮的興起，越來越多的 AI 應用隨之誕生。在邊緣運算技術的加持下，AI 已被廣泛運用於生活中的多種應用場景。例如，通過識別人的表情可以了解其當前的心理狀態。對於情緒障礙患者，需要進行實時監控以確保其身心狀態穩定，但常人無法長時間觀看患者。因此，將表情辨識技術與機器人結合，可以實現長時間的監控，防止悲劇發生。

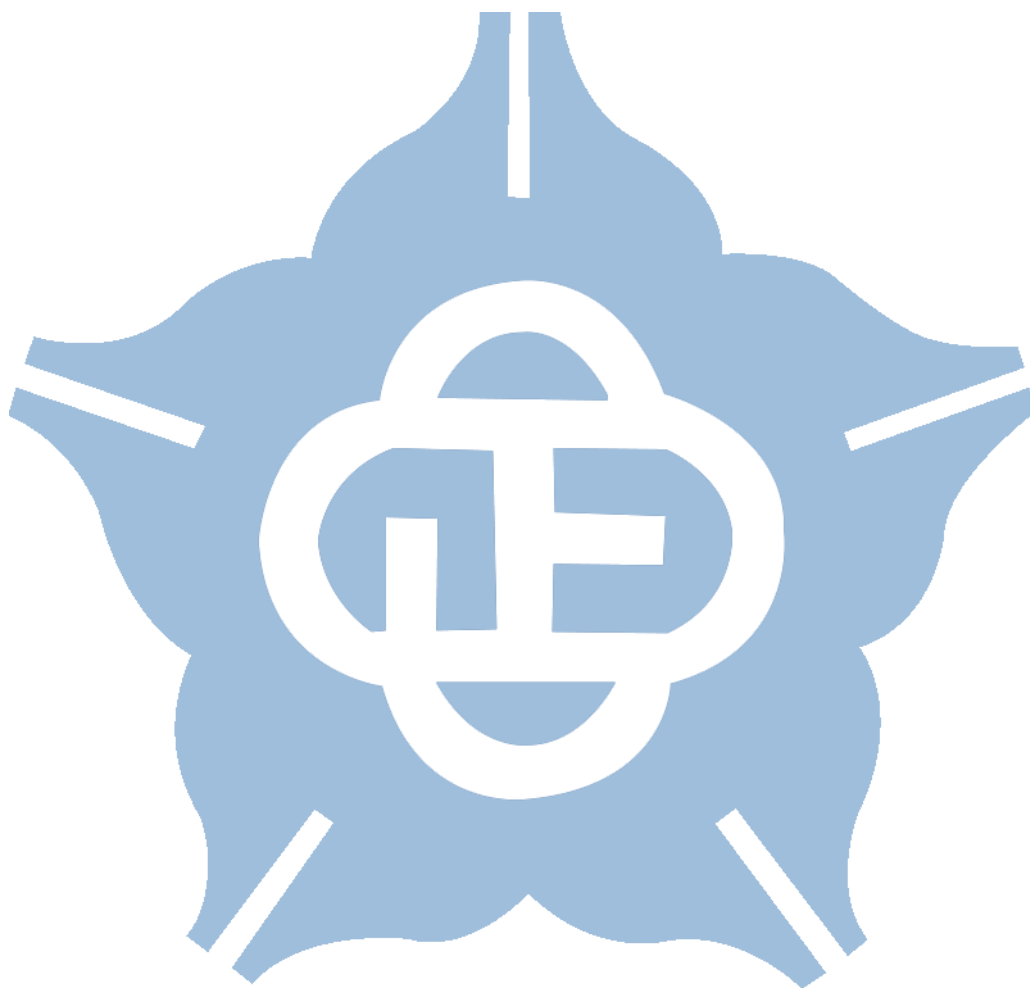
本論文使用分層深度可分離卷積神經網路（Hierarchical Depthwise Separable Convolutional Neural Network，HDSC）進行訓練，並以 16nm FinFET 製程實現。對於高準確度的類別，本架構的優點在於能在前幾層就辨識出類別，不同於傳統的卷積神經網路需要通過多層運算才能得到結果，從而浪費許多計算資源。研究數據顯示，使用該架構時，於第一層辨識出來相比於運算至最後一層，可以減少約 55.45% 的計算量，同時由於結合了深度可分離卷積（Depthwise Separable Convolutional），大幅減少了參數量。

此外，本論文分析了資料集，發現存在數據不平衡的問題，因此對不平衡資料集進行了處理。處理後的結果相較於未處理的效果顯著提升。考慮到硬體實現，本文採用了 DoReFa-Net 的方法對網路參數進行量化，並對激勵函數進行量化和定點化，同時對 batch normalization 後的結果進行定點化，以減少記憶體大小及計算量。除此之外，還設計了電源門控技術，在記憶體閒置狀態時關閉其電源，使整體架構在硬體上能夠進一步降低功耗。

本論文所提出之網路硬體加速器於 16nm CMOS 製程下實現，並透過電源門控技術將閒置記憶體的供電關閉，以進一步降低功耗。本論文所提出之電路經自動佈局繞線實現後，佈局後模擬驗時，工作頻率最快可達 555 MHz，功耗為 53.86 mW，同時電路對表情辨識達到了 71.70% 的準確度。

關鍵字：情感機器人、資料集不平衡、分層深度可分離卷積神經網路(HDSC)、卷

積神經網路(CNN)、低功耗



Abstract

In recent years, with the surge in AI technology, an increasing number of AI applications have emerged. Bolstered by edge computing, AI has been widely integrated into various real-world scenarios. For instance, by recognizing human facial expressions, it is possible to gauge an individual's current psychological state. For patients with emotional disorders, real-time monitoring is crucial to ensure their mental and physical well-being. However, it is impractical for humans to constantly observe these patients. By combining facial recognition technology with robotics, continuous monitoring can be achieved, thus preventing potential tragedies.

This paper employs a Hierarchical Depthwise Separable Convolutional Neural Network (HDSC) for training, implemented using a 16nm FinFET process. One of the key advantages of this architecture lies in its ability to identify categories in the early layers, which differs from traditional convolutional neural networks that require multiple layers of computation to achieve results, thereby wasting considerable computational resources. The research data indicates that using this architecture, approximately 55.45% of the computational load can be reduced at the first layer compared to performing computations up to the final layer. Additionally, by incorporating Depthwise Separable Convolutions, there is a significant reduction in the number of parameters.

Additionally, this paper addresses the issue of data imbalance in the dataset. After processing the imbalanced dataset, the results showed a significant improvement compared to the unprocessed data. Considering hardware implementation, this study adopted the DoReFa-Net method to quantize network parameters, including the quantization and fixed-point implementation of the activation function, as well as the fixed-point implementation of the batch normalization results, thereby reducing

memory size and computational overhead. Moreover, power gating technology was designed to turn off the memory power supply during idle states, further reducing the overall power consumption of the hardware architecture.

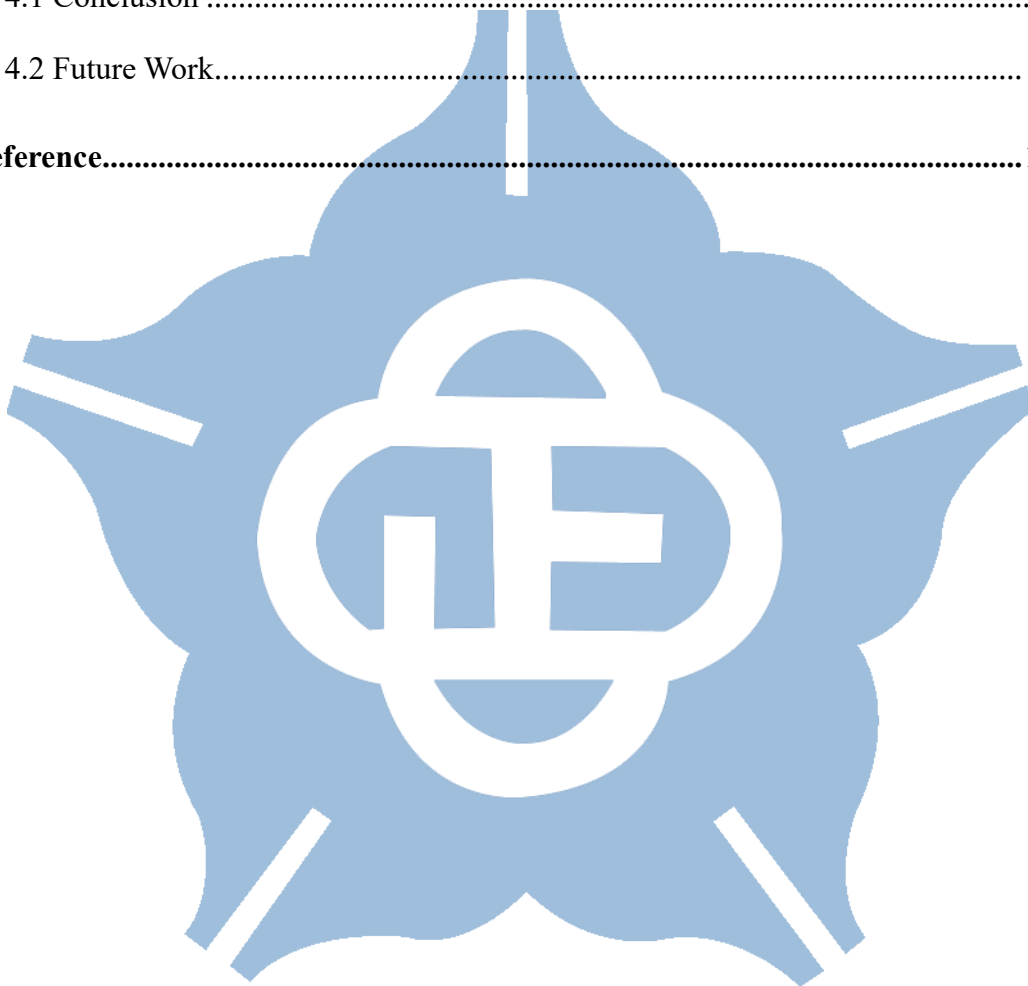
The proposed network hardware accelerator was implemented using a 16nm CMOS process. By employing power gating technology to turn off the power supply to idle memory, further power consumption reduction was achieved. After automatic layout and routing, post-layout simulation verified that the circuit could operate at a maximum frequency of 555 MHz, with a power consumption of 53.86 mW, while achieving an accuracy of 71.70% in facial expression recognition.

Keywords: Emotional robots, imbalanced datasets, Hierarchical Depthwise Separable Convolutional Neural Networks (HDSC), Convolutional Neural Networks (CNN), low-power

Content

摘要.....	III
Abstract.....	V
Content.....	VII
List of Figures.....	IX
List of Tables.....	XII
Chapter 1 Introduction.....	1
1.1 Introduction to Facial Expression Recognition and Their Applications	1
1.2 Introduction to Convolutional Neural Networks	11
1.3 Introduction to Hierarchical convolutional neural network	17
1.4 Introduction to Class Imbalance Training Methods.....	25
1.5 Compression Techniques on Neural Networks	30
1.6 Motivation.....	37
1.7 Thesis Organization	39
Chapter 2 Software Implementation.....	40
2.1 Architecture Overview	40
2.2 Implementation Hierarchical convolutional neural network on software and analysis.....	42
2.3 The impact of resolving class imbalance and analysis.....	61
2.4 Weight Quantization Method	66
2.5 Quantization Methods for Activation Function	68
Chapter 3 Hardware Implementation.....	71
3.1 Fixed-Point weights and Batch Normalization Layer	71

3.2 HDSC Hardware Accelerator Architecture	76
3.3 Memory Power Gating.....	81
3.4 Hardware Implementation Analysis and Comparison.....	88
3.5 Summary	97
Chapter 4 Conclusion and Future Works.....	99
4.1 Conclusion	99
4.2 Future Work.....	100
Reference.....	101



List of Figures

Figure 1.1 The methodology of Neighborhood-Aware Edge Directional Pattern [4]....	2
Figure 1.2 The various local structures generated through the local prominent directional pattern [5].....	2
Figure 1.3 The systemic components of evaluation of online teaching quality [7].....	3
Figure 1.4 The emotional recognition system for crime prediction [8].	4
Figure 1.5 The facial expression recognition system for drivers [9].	5
Figure 1.6 Images in dataset FER-2013 [12].	6
Figure 1.7 The CNN architecture [11].	7
Figure 1.8 The VGGNet architecture [10].	7
Figure 1.9 The different outcomes of flipping [16].	8
Figure 1.10 The mini-Xception architecture [14].	9
Figure 1.11 The SDNet architecture [15].....	9
Figure 1.12 The Dual CNNs architecture [37].....	9
Figure 1.13 Example of the convolution operation.	11
Figure 1.14 The pooling operation.....	13
Figure 1.15 The architecture of LeNet-5 CNN used for digit recognition [18].....	13
Figure 1.16 The architecture of AlexNet [19].....	14
Figure 1.17 Residual block [21].....	15
Figure 1.18 The standard convolution and depthwise separable convolution [22].	16
Figure 1.19 Traditional CNN classification [23].	17
Figure 1.20 Modular neural network for classification [23].....	18
Figure 1.21 The MNN-Tree for the CIFAR-10 dataset [23].....	18
Figure 1.22 The MNN-Tree consumes the least inference time on Raspberry Pi 3 and Raspberry Pi Zero, respectively. [23].....	20

Figure 1.23 MNN-Tree consumes the least energy on both Raspberry Pi 3 and Raspberry Pi Zero, respectively [23].	20
Figure 1.24 Confusion matrix on test data for random forest classifier [24].	21
Figure 1.25 The structure of the proposed hierarchical classifier [24].	22
Figure 1.26 Detail comparison of accuracy [24].	23
Figure 1.27 Accuracy comparison on the Iris Dataset [36].	24
Figure 1.28 Accuracy comparison results on the Anneal Dataset [36].	24
Figure 1.29 Comparisons of methods for achieving category balance [26].	26
Figure 1.30 The probability distribution of the data [26].	26
Figure 1.31 Example of imbalanced binary classification [25].	28
Figure 1.32 The three-stage compression process of pruning, quantization, and Huffman coding [27].	30
Figure 1.33 Weight sharing by K-means clustering [27].	31
Figure 1.34 Two new exit points are added to the model [29].	32
Figure 2.1 Flowchart for designing the hierarchical convolutional neural network software model.	41
Figure 2.2 The entire operation of first layer.	42
Figure 2.3 The entire operation of each layer, except for the first layer.	42
Figure 2.4 The confusion matrix of Type-1 structure.	46
Figure 2.5 Type-2 classification from start to finish.	47
Figure 2.6 Confusion matrix for Type-2 classification.	52
Figure 2.7 Type-3 classification from start to finish.	53
Figure 2.8 Confusion matrix for Type-3 classification.	58
Figure 2.9 A statistical chart of the quantity within the FER2013 dataset.	61
Figure 2.10 A statistical analysis of the number of categories in the doorNet training dataset.	62

Figure 2.11 The dataset of ImproveNet0 comprises a detailed statistical analysis of the number of categories.....	62
Figure 2.12 The flowchart for the DRW.	64
Figure 3.1 The proposed HDSC model hardware architecture.	76
Figure 3.2 Counting cycles at the RTL level across various hierarchical levels.....	80
Figure 3.3 The power gating control of HDSC.....	81
Figure 3.4 Flow chart of doorNet in HDSC.....	86
Figure 3.5 Flow chart of ImproveNet0 in HDSC.	86
Figure 3.6 Flow chart of ImproveNet1 in HDSC.	87
Figure 3.7 Compare the power consumption of the doorNet model in HDSC at 555 MHz with and without power management.....	89
Figure 3.8 Compare the power consumption of the doorNet and ImproveNet0 models in HDSC at 555 MHz with and without power management.	89
Figure 3.9 Compare the power consumption of the doorNet, ImproveNet0, and ImproveNet1 models in HDSC at 555 MHz with and without power management. ..	90
Figure 3.10 The layout of the HDSC model chip.	94

List of Tables

Table 1.1 The MNN-Tree about model size, operations, and test error[23]	19
Table 1.2 The classification error rates of ResNet-32 with different loss functions and class balancing were evaluated on long-tail CIFAR-10 and CIFAR-100 datasets. [26]	27
Table 1.3 The classification error rates of ResNet-32 with different loss functions and class balancing were evaluated on long-tail CIFAR-10 and CIFAR-100 datasets. [25]	29
Table 2.1 Accuracy of Type-1 with different kernel sizes.....	43
Table 2.2 Accuracy of Type-1 model with different filter sizes.....	44
Table 2.3 The Type-1 model architecture.....	44
Table 2.4 The doorNet model architecture in Type-2 classification.	47
Table 2.5 The ImproveNet0 model architecture in Type-2 classification.	48
Table 2.6 The ImproveNet1 model architecture in Type-2 classification.	48
Table 2.7 The parameter count of doorNet in Type-2 classification.	49
Table 2.8 The parameter count of ImproveNet0 in Type-2 classification.....	50
Table 2.9 The parameter count of ImproveNet1 in Type-2 classification.....	50
Table 2.10 The accuracy and parameters of the Type-2 classification.....	52
Table 2.11 The doorNet model architecture in Type-3 classification.	54
Table 2.12 The ImproveNet0 model architecture in Type-3 classification.	54
Table 2.13 The model architecture of ImproveNet1 and ImproveNet2 in Type-3 classification.	55
Table 2.14 The parameter count of doorNet in Type-3 classification.....	55
Table 2.15 The parameter count of ImproveNet0 in Type-3 classification.....	56
Table 2.16 The parameter count of ImproveNet1 and ImproveNet2 in Type-3	

classification.	57
Table 2.17 The accuracy and parameters of the Type-3 classification.....	58
Table 2.18 Comparison of Type-1, Type-2, and Type-3	59
Table 2.19 Comparison of Type-1 architecture with Type-2 classification in terms of FLOPs.	60
Table 2.20 Comparison of FLOPs for each model in Type-2 classification.	60
Table 2.21 Handling data imbalance using different methods resulted in varying accuracy for doorNet.....	64
Table 2.22 Employ different methods to address data imbalance and enhance the accuracy of ImproveNet0.....	65
Table 2.23 Compare the differences between the results after applying IR-NET and the original results.....	66
Table 2.24 After quantization with DoReFa-Net, the accuracy varies with different bit numbers.....	67
Table 2.25 The accuracy of using different scaling factors.	69
Table 2.26 The accuracy associated with varying activation bit widths.....	69
Table 2.27 Hyperparameters setting and loss function used in hierarchical models. ..	70
Table 3.1 Comparison of accuracy of weight across different bit levels.	71
Table 3.2 Comparison of accuracy of γ' across different bit levels.....	72
Table 3.3 Comparison of accuracy of β' across different bit levels.	73
Table 3.4 The accuracy comparison table of fixed-point values after batch normalization.	74
Table 3.5 The fixed-point decision for hardware implementation.	75
Table 3.6 Memory usage table for each layer of doorNet.....	78
Table 3.7 Memory usage table for each layer of ImproveNet0.	78
Table 3.8 Memory usage table for each layer of ImproveNet1.	79

Table 3.9 The power state table of doorNet.	82
Table 3.10 The power state table of ImproveNet0.	83
Table 3.11 The power state table of ImproveNet1.	84
Table 3.12 When there is no power control, the power consumption of each memory module under the condition of running to ImproveNet1.	91
Table 3.13 When the CEB switch control is set to "on," the power consumption of each memory unit, under the condition of running to ImproveNet1.	91
Table 3.14 When the CEB switch control is set to "on" and includes power-gating, the power consumption of each memory module, under the condition of running to ImproveNet1.	92
Table 3.15 Power consumption at each stage at 555 MHz with and without power management control.	93
Table 3.16 Time required for accessing ROM.	95
Table 3.17 The HDSC hardware implementation results	96
Table 3.18 Comparison with other methods.	97

Chapter 1 Introduction

1.1 Introduction to Facial Expression Recognition and Their Applications

The relationship between facial expressions and emotions is an important research field in social psychology. According to Darwin's research [1], emotions exhibited by humans and animals through facial expressions are inherently related to innate emotions. Ekman and Friesen proposed six basic emotions in 1971 [2], namely happiness, sadness, fear, disgust, surprise, and anger. These basic emotions have distinct content and corresponding facial expressions, sometimes referred to as typical emotional expressions.

In addition, as mentioned in [3], a significant amount of research in social psychology indicates that facial expressions facilitate conversation coordination, and they have a much greater impact on whether listeners like or dislike the speaker than the speaker's verbal expressions. This implies that facial expressions are a primary mode of human communication. Specifically, the linguistic part of information (i.e., verbal language) contributes only 7% to the overall impact of information, the vocal part (e.g., intonation) contributes 38%, while the speaker's facial expressions contribute 55% to the impact of verbal information. These research findings emphasize the close connection between emotions and facial expressions, providing us with a deeper understanding of emotional expression and interpersonal communication.

Building upon traditional methods for facial expression recognition, as mentioned in [4], the approach employs manual calculation. The method utilizes Neighborhood-Aware Edge Directional Pattern to recognize facial expressions, as illustrated in Figure 1.1. This involves computing the gradient direction of each pixel in the image, followed

by checking whether the gradient direction of neighboring edge pixels matches predefined templates. Finally, feature vectors are generated for facial expression recognition.

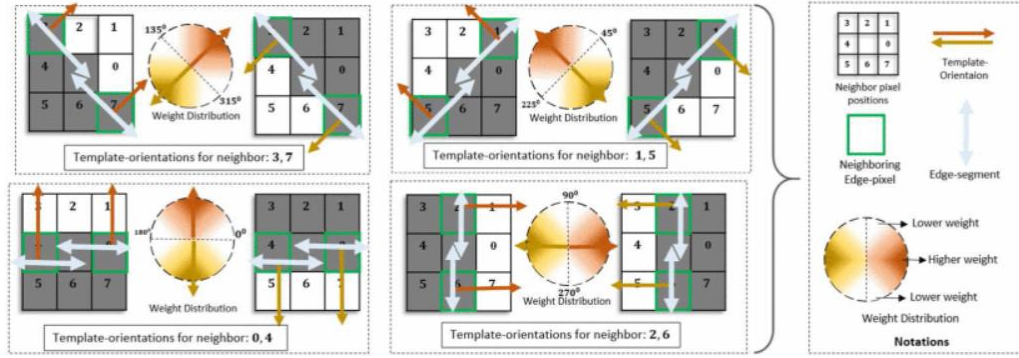


Figure 1.1 The methodology of Neighborhood-Aware Edge Directional Pattern [4].

Additionally, in [5], a local prominent directional pattern method is employed for facial expression recognition. This method generates robust features by statistically utilizing edge information within local neighborhoods and establishes a mechanism to prevent code generation in flat regions, thereby encoding meaningful facial features. As demonstrated in Figure 1.2, it shows resilience to noisy conditions. The accuracy achieved on the CK+ facial expression dataset is 94.09%, while on the MMI facial expression dataset, it is 69.05%.

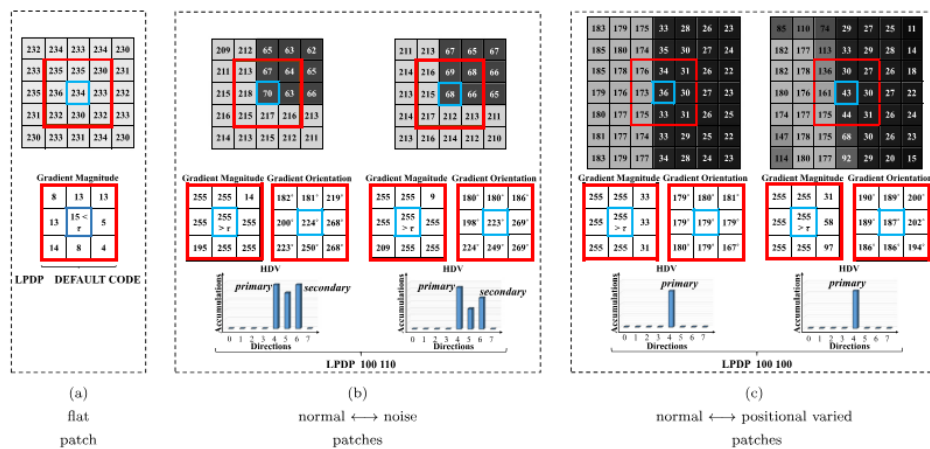


Figure 1.2 The various local structures generated through the local prominent directional pattern [5].

In traditional methods, the computational approach is particularly complex, making it difficult to apply effectively in real-time facial expression recognition. However, with recent advancements in computer vision technology, especially the application of deep learning and convolutional neural networks, facial expression recognition technology has overcome previous limitations. As mentioned in [6], earlier facial expression recognition methods were prone to misjudgments in complex environments. However, current technology can more accurately identify facial expressions, achieving a high level of accuracy in emotion recognition even in images captured under controlled conditions and consistent environments.

With the breakthroughs in facial expression recognition technology, it plays an increasingly crucial role in today's society, with applications spanning across various fields. As indicated by ref. [7], this technology can be applied in the educational domain, addressing the challenge faced by educators in online teaching of not being able to promptly understand students' learning statuses. A system has been designed employing automated facial detection and expression recognition techniques to assist educators in gauging students' classroom engagement, thereby enabling teachers to better understand the emotional states of individual students during lectures. The system, depicted in Figure 1.3, utilizes OpenCV to access the camera for real-time capture of students' classroom videos, followed by facial detection and expression analysis to determine students' emotions and teachers' teaching conditions.

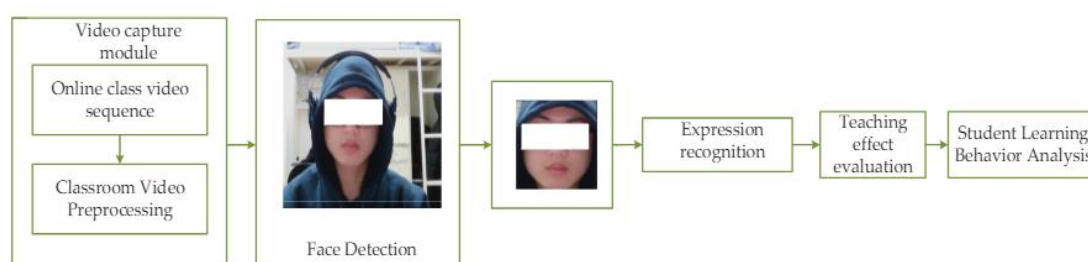


Figure 1.3 The systemic components of evaluation of online teaching quality [7].

In addition, in reference [8], crime prediction is applied using intelligent cameras and the Internet of Things (IoT) to collect image samples with time and location tags within the city. Subsequently, facial expression recognition technology is employed to quantify emotions, thereby predicting crimes. This method assists the police and law enforcement agencies in forecasting the potential locations, times, and types of criminal incidents, thus enhancing public security management and crime prevention effectiveness.

As shown in Figure 1.4, the emotion recognition system first defines the necessary emotion category labels for the research, determining the classification method for emotions. Then, the input images are convolved through filters in the convolutional layers, producing feature maps, which are then subjected to max pooling to reduce computational load. Subsequently, the system constructs a fully connected layer that converts the feature maps into the output for emotion recognition. Finally, based on the output of the Softmax classifier, the system accurately identifies and outputs the corresponding emotional information. However, upon examining the structure, it can be observed that this framework employs numerous convolutional layers, which inevitably increases the computational load, making real-time recognition difficult.

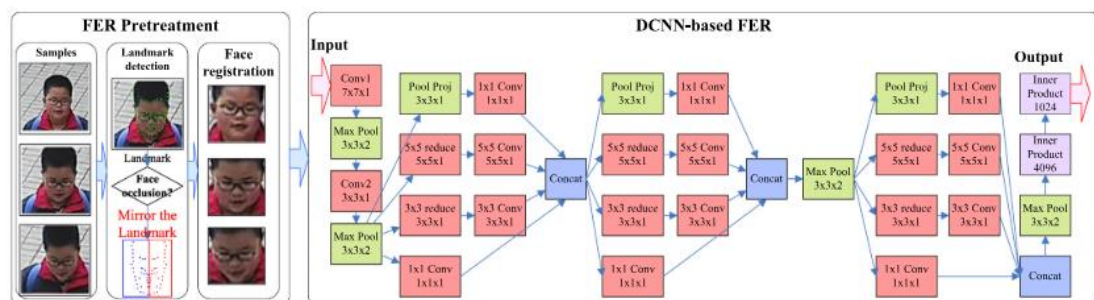


Figure 1.4 The emotional recognition system for crime prediction [8].

In recent years, with the rapid development of the Internet of Things (IoT) and edge computing, facial expression recognition technology has been widely applied to

edge devices, as described in [9], such as its application in cars. Driving is indispensable in the daily lives of many people, thus driving safety is crucial. Emotional fluctuations during driving may impact safety; therefore, identifying the driver's facial expressions has become an important basis for improving road safety. The system, as shown in Figure 1.5, preprocesses input images to remove irrelevant data and enhances detected relevant data. Subsequently, facial expressions are used for feature extraction, ultimately leading to the classification of the driver's facial expressions.

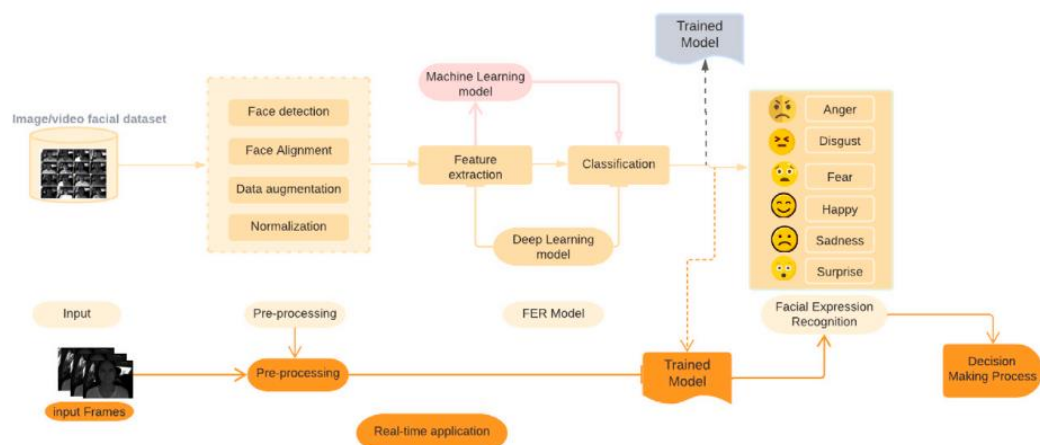


Figure 1.5 The facial expression recognition system for drivers [9].

On the other hand, [17] extensively discusses the application and impact of social robots in the field of mental health. The text indicates that multiple studies have shown that the intervention of social robots can positively influence emotions and quality of life. They do so by identifying individuals in need of emotional care through facial expressions and providing emotional support, guidance, and encouragement to help improve the psychological well-being of the subjects. In [10, 12], facial expression recognition technology is applied to social robots, utilizing the FER2013 dataset proposed at the 2013 International Conference on Machine Learning (ICML), as shown in Figure 1.6. Due to its challenging nature under natural conditions, it has become a benchmark for comparing the performance of emotion recognition models. It is also

one of the most commonly used datasets for evaluating the performance of emotion recognition models.

Considering the numerous applications mentioned above, it is evident that facial expression recognition demands real-time capabilities and, in the case of robots, also requires low power consumption. However, the architectures used in these applications primarily focus on accuracy, resulting in extremely large models that are challenging to implement for real-time recognition. Additionally, these models are difficult to deploy on edge devices such as robots or automobiles. Therefore, it is crucial to develop an architecture capable of real-time facial expression recognition while maintaining low power consumption. Moreover, maintaining recognition accuracy is essential for practical applications.



Figure 1.6 Images in dataset FER-2013 [12].

For the performance evaluation of the FER2013 dataset, in [11], the authors utilized the architecture depicted in Figure 1.7. They concatenated the results of three CNN subnetworks in the fully connected layer and ultimately achieved an accuracy of 62.44% through a softmax layer. In [10], the authors employed the VGGNet architecture shown in Figure 1.8 for their research. They explored the impact of optimizers and learning rate schedulers during training and ultimately achieved an accuracy of 73.28%.



Figure 1.7 The CNN architecture [11].

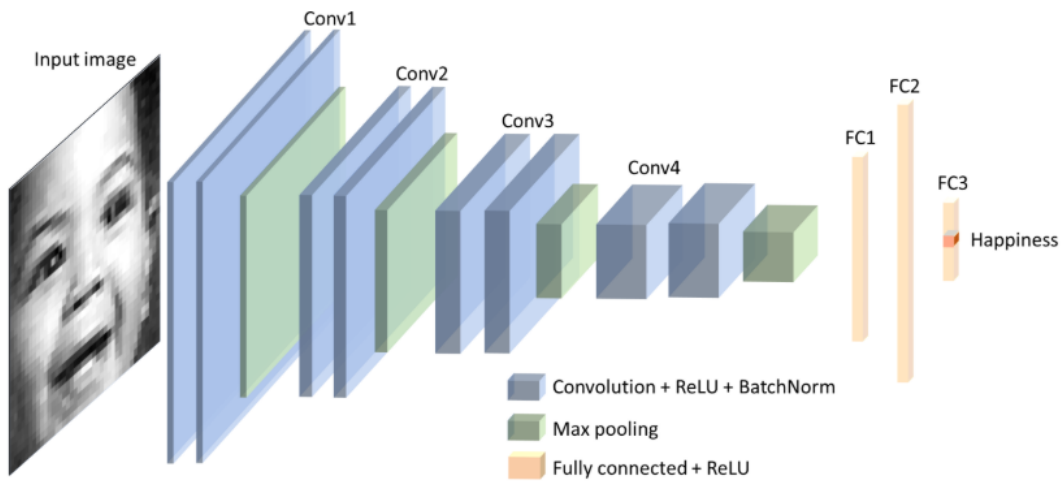


Figure 1.8 The VGGNet architecture [10].

In [16], it was observed that flipping the dataset yielded different results in network recognition, as depicted in Figure 1.9. Therefore, during training, augmented flipped images were utilized to enhance accuracy, reaching 70%. In [12], the investigation focused on the impact of layer count and the addition of batch normalization on the architecture. Additionally, data augmentation was performed using histogram equalization for illumination correction. Parameter quantities and accuracies were tested across VGG, Inception, and ResNet, resulting in accuracies of 72.7%, 71.6%, and 72.4%, respectively.



Figure 1.9 The different outcomes of flipping [16].

Additionally, some studies have replaced the fully connected layers with Global Average Pooling (GAP) in the VGG-16 architecture to reduce the likelihood of overfitting. One such study [13] demonstrated that this modified model achieved an accuracy of 69.40% on the test set. In another study [14], by incorporating GAP and Separable Depthwise Convolutions, along with adopting the mini-Xception architecture as illustrated in Figure 1.10, the model's parameter count was reduced to 60K, achieving an accuracy of 66%. Another study [15] utilized GAP and Separable Depthwise Convolutions, and introduced the Convolutional Block Attention Module (CBAM) to enhance accuracy. Through these improvements, they successfully developed a lightweight network while maintaining a relatively high accuracy of 67.08%, as depicted in the schematic diagram in Figure 1.11. Moreover, in [37], a Dual CNNs architecture was employed, achieving an accuracy of 72.27%, with the schematic diagram shown in Figure 1.12.

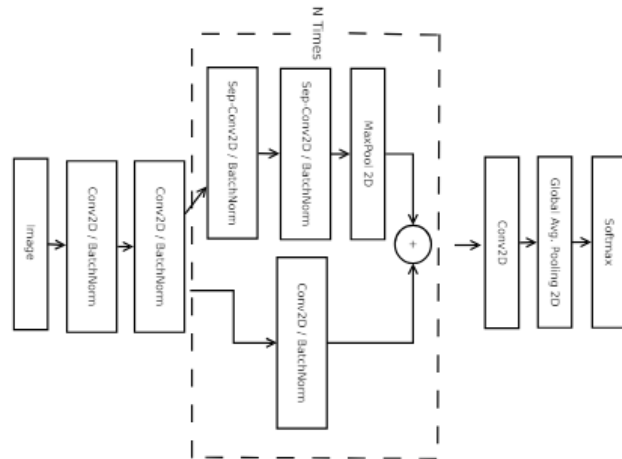


Figure 1.10 The mini-Xception architecture [14].

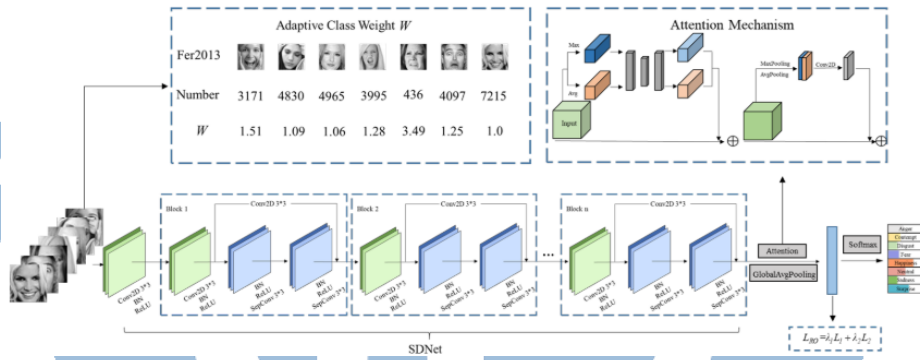


Figure 1.11 The SDNet architecture [15].

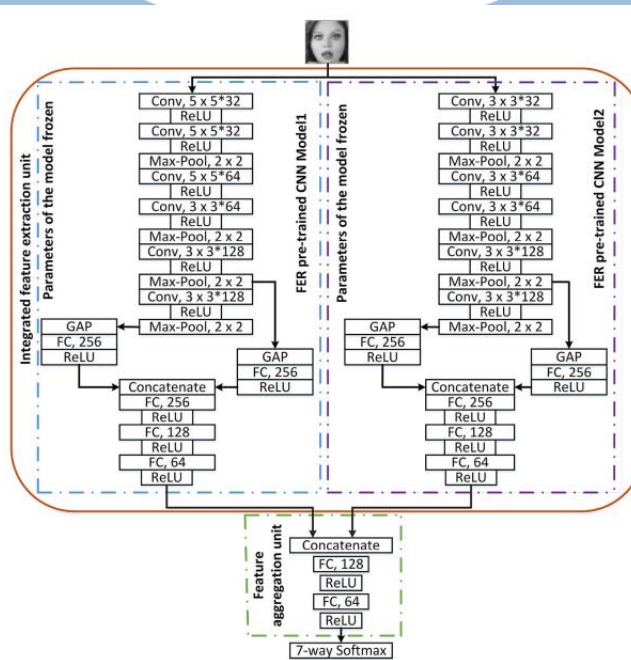


Figure 1.12 The Dual CNNs architecture [37].

In recent years, several studies have revised the FER2013 dataset by removing some misclassified images and re-annotating their categories, resulting in the so-called FER2013plus dataset. In [35], researchers utilized this dataset and applied CycleGAN to generate new images for minority classes. This approach, combined with transfer learning for training, achieved a test accuracy of 91.76% on the FER2013plus dataset. However, the overall structure became highly complex, leading to a significant increase in computational load, which caused the FPS to drop to just 2.27. In [38], the FER2013plus dataset was also employed in conjunction with transfer learning, but the neutral class was excluded to suit the application scenario. In the 6-class test on the FER2013plus dataset, this method achieved an accuracy of 82.1%.

Most studies applying architectures to the FER2013 dataset primarily focus on accuracy, often increasing the model size without restraint, yet ultimately achieving an accuracy of around 70%. Although some models employ relatively lightweight methods for reduction, overall, applying them to edge devices or achieving real-time recognition remains challenging. This is because most models require substantial storage and involve extensive computations, making them difficult to implement in practical applications.

1.2 Introduction to Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are a type of artificial neural network used for processing and analyzing data with grid-like structures. Initially designed for handling image data, they have also been successfully applied to sequence data and other types of structured data. Currently, CNNs find widespread applications in the field of computer vision, including tasks such as image classification, object detection, and semantic segmentation. The operation of convolution, as shown in Figure 1.13, involves applying a kernel to the input data. At each position of the kernel, element-wise multiplication is performed with the input data, and then all the products are summed to obtain a single output value, resulting in a feature map.

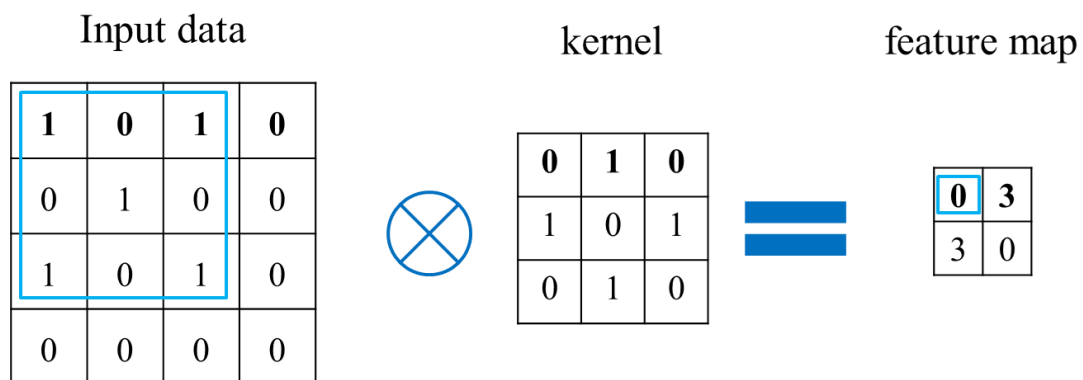


Figure 1.13 Example of the convolution operation.

After performing the convolution operation, to enhance the expressive power and non-linear fitting capability of the network, each element in the output matrix is typically subjected to a non-linear transformation through an activation function. Commonly used activation functions include ReLU and sigmoid. The ReLU function retains the input value in the positive range and outputs zero in the negative range, thus enabling fast computation. The sigmoid function is characterized by a smooth S-shaped

curve with a continuous derivative, making it highly suitable for optimization algorithms such as gradient descent. The transformation formulas for these two activation functions are given by Equations 1.1 and 1.2 respectively.

$$\text{ReLU}(x) = \max(0, x) \quad (1.1)$$

$$\text{Sigmoid}(x) = \frac{1}{1 + e^{-x}} \quad (1.2)$$

Batch normalization is a widely used deep learning technique aimed at accelerating neural network training speed and improving performance. Its core idea is to normalize the inputs of each layer in the neural network to ensure stable data distribution, thus facilitating gradient propagation and network convergence. The steps involve first calculating the mean and standard deviation for each mini-batch of data. Next, the data within the mini-batch is normalized using these statistics. Subsequently, the normalized data is linearly transformed using learnable scaling and bias parameters. Finally, the transformed data is used as the input for the next layer, completing one iteration of the Batch normalization process. The formulas for Batch normalization are given in Equations 1.3 to 1.6.

$$\mu_B \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad (1.3)$$

$$\alpha_B^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2 \quad (1.4)$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_B}{\sqrt{\alpha_B^2 + \epsilon}} \quad (1.5)$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad (1.6)$$

Pooling reduces the spatial dimensions of feature maps by aggregating information within each pooling window. The most common pooling operations are Max Pooling and Average Pooling, as shown in Figure 1.14. They are primarily employed to decrease

the spatial size of feature maps, thereby reducing the network's parameter count and computational load.

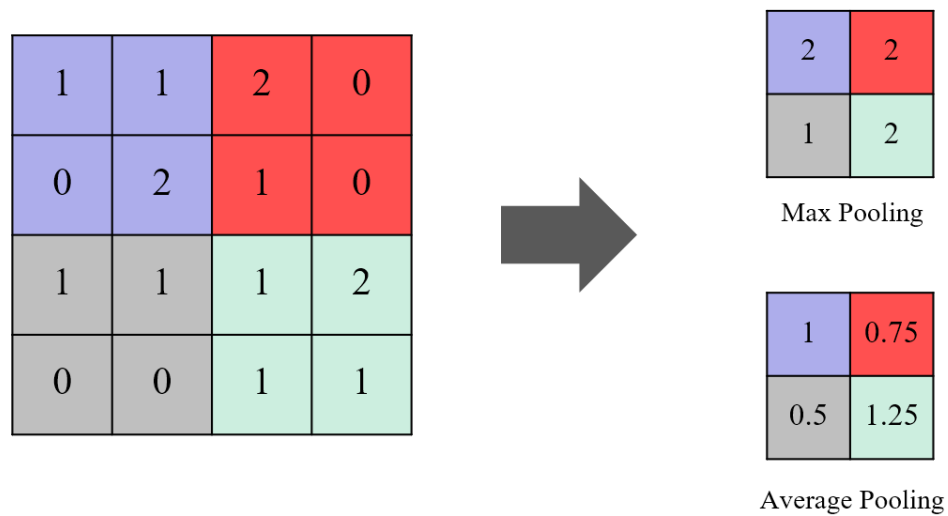


Figure 1.14 The pooling operation.

In 1998, Yann LeCun [18] introduced the earliest Convolutional Neural Network (CNN) architecture for handwritten digit recognition, known as LeNet-5. It consists of two convolutional layers, two pooling (subsampling) layers, and three fully connected layers, as depicted in Figure 1.15. The convolutional layers are employed to extract features from images, while the pooling layers reduce the dimensionality of feature maps while preserving essential information. The fully connected layers then map the extracted features to the final output categories. However, due to its relatively simple architecture and shallow depth, LeNet-5 is not well-suited for recognizing large-scale or overly complex datasets.

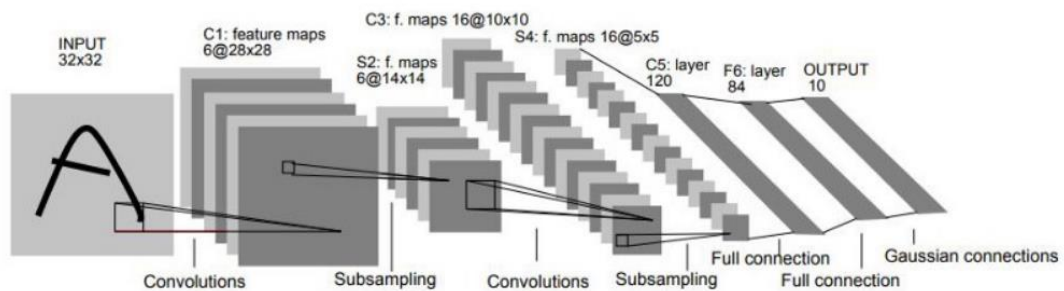


Figure 1.15 The architecture of LeNet-5 CNN used for digit recognition [18].

AlexNet, as shown in Figure 1.16, is eight layers deeper than the LeNet-5 model and was developed by Alex Krizhevsky in 2012 [19]. It performs well in identifying complex and large datasets. However, its computational cost is high, requiring the use of multiple GPUs in parallel during training.

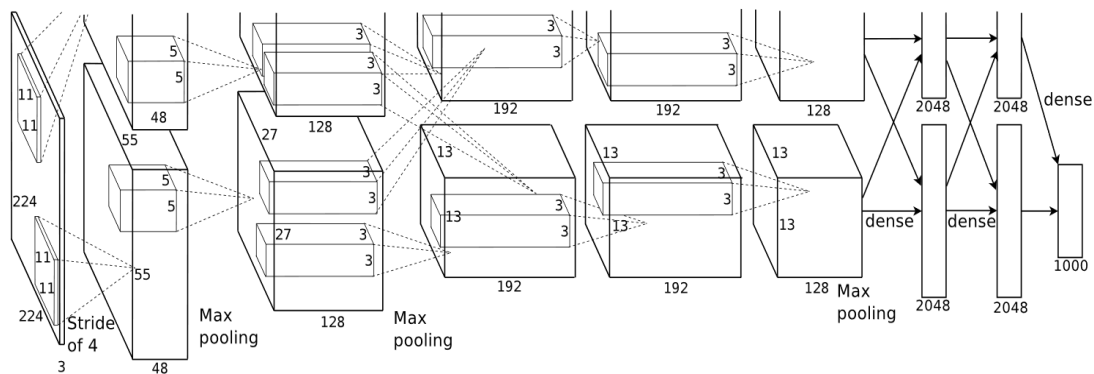


Figure 1.16 The architecture of AlexNet [19].

Continuing with the introduction, VGGNet [20], which secured second place in the ImageNet competition, is renowned for its consistent use of 3×3 convolutional kernels in convolutional operations. This characteristic enhances the network's depth, thereby improving performance and showcasing exceptional transfer learning capabilities. Widely applied in the field of computer vision, this deep learning model was proposed by Karen Simonyan and her team in 2014. VGGNet typically refers to VGG16, consisting of thirteen convolutional layers and three fully connected layers. Despite its relatively high number of layers, this also results in slower training speeds and a substantial parameter count in the fully connected layers.

The champion of the 2015 ImageNet competition was ResNet [21]. It addressed the issue of gradient vanishing in deep networks by introducing residual structures, as shown in Figure 1.17. Through this, it can be observed that current convolutional neural networks have achieved significant success in many fields. However, they face major challenges for deployment on edge devices, including high computational costs, large

power consumption, and insufficient computational resources, making them unsuitable for edge deployment.

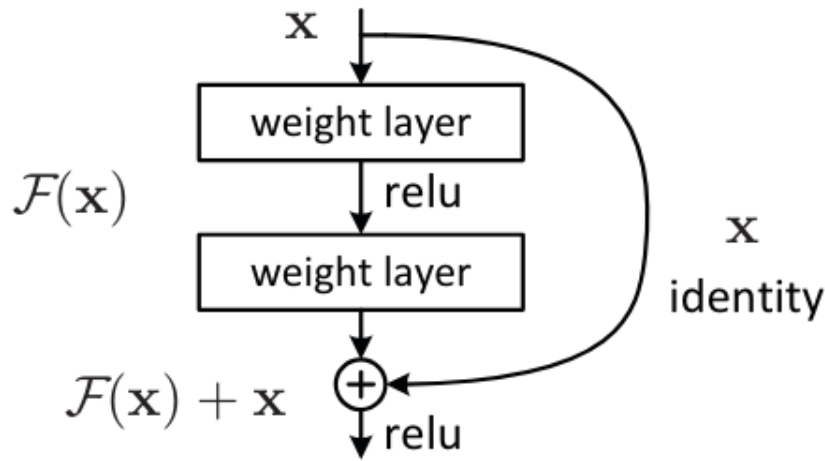


Figure 1.17 Residual block [21].

MobileNet[22] is specifically designed for resource-constrained environments such as mobile devices and embedded systems. Proposed by the Google team in 2017, its aim is to address challenges faced in deploying deep learning models on mobile platforms, including limited computational power, memory, and battery life constraints. It adopts Depthwise Separable Convolution (DSC), where standard convolution is decomposed into depthwise convolution and pointwise convolution steps, as shown in Figure 1.18.

Depthwise convolution performs independent convolutions on each channel, reducing computational load, while pointwise convolution uses 1×1 convolution kernels to merge features from different channels. These features of MobileNet are utilized to reduce the number of model parameters and computational overhead, enabling efficient operation on resource-constrained devices. Additionally, MobileNet employs techniques such as global average pooling and lightweight activation functions like ReLU6 to further simplify the network structure, significantly reducing parameter count and computational load while preserving accuracy.

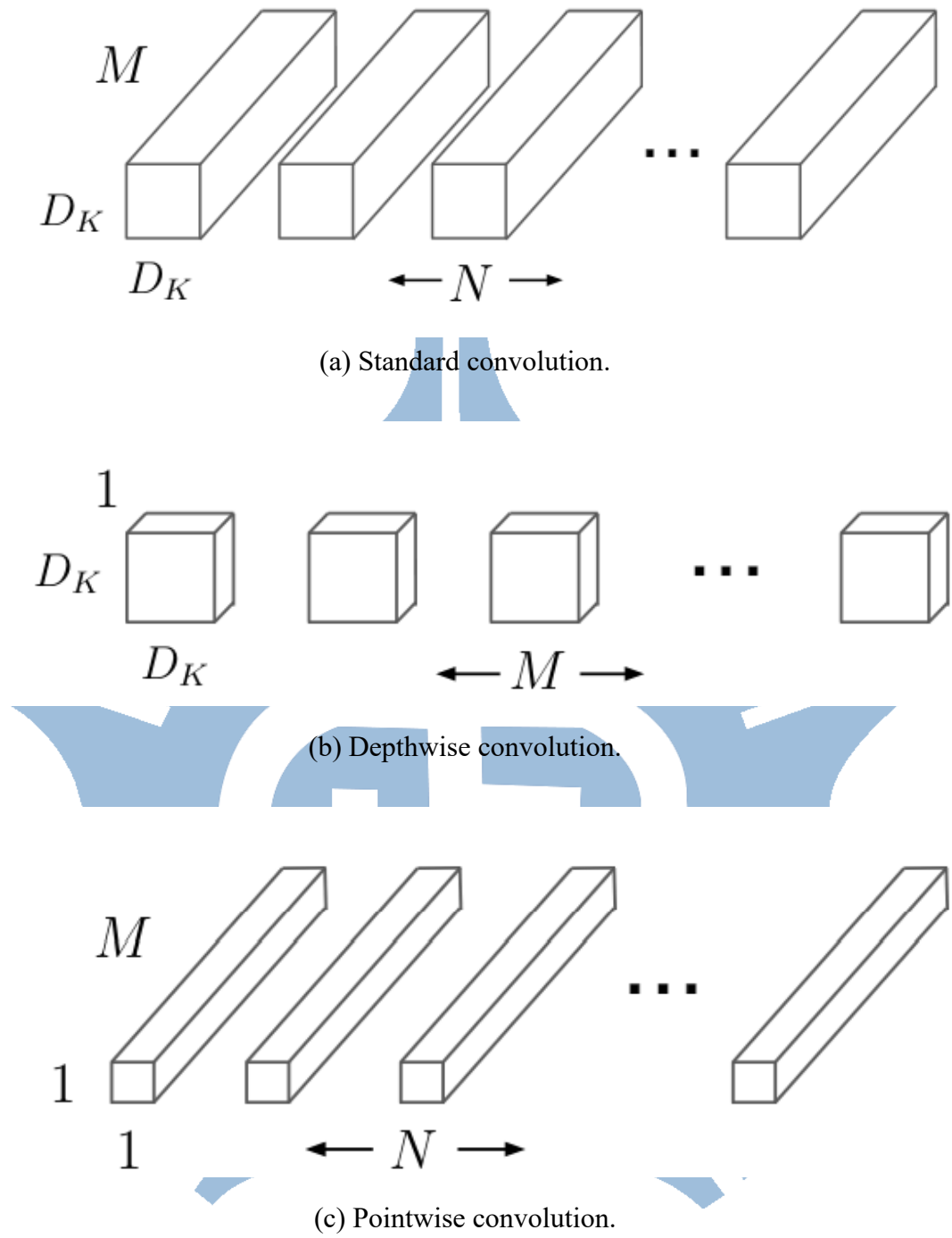


Figure 1.18 The standard convolution and depthwise separable convolution [22].

1.3 Introduction to Hierarchical convolutional neural network

When applying deep learning neural networks to hardware, challenges arise in dealing with the computational demands of the network. In a neural network, each neuron is connected to all neurons in the preceding layer and possesses individual weights. During computation, neurons in the network perform weighted summations of input data with their respective weights. This process involves millions of operations, consuming significant energy. To address this, Abhinav Goel proposed a Modular Neural Network (MNN) architecture [23]. This architecture first classifies images and then further divides categories into multiple modules until the final classification is achieved. This approach reduces memory requirements, enhances efficiency, and minimizes redundant operations in large neural networks, as illustrated in Figures 1.19 and 1.20.

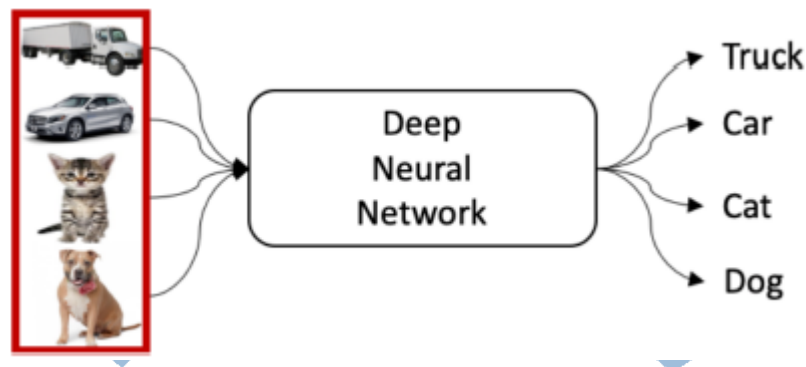


Figure 1.19 Traditional CNN classification [23].

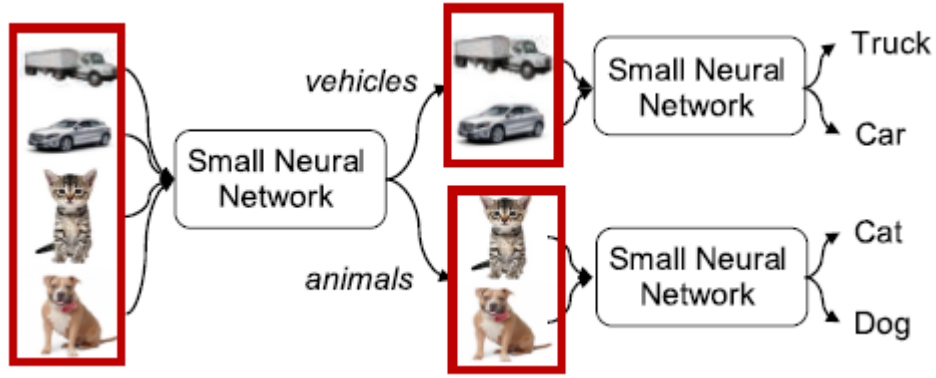


Figure 1.20 Modular neural network for classification [23].

Most CNNs, such as ResNet or VGGNet, differ from MNN in that they adopt a single DNN architecture, handling all feature identification at once instead of using multiple modules for category refinement like MNN. However, this single architecture can lead to unnecessary energy consumption. While only a few neurons are active when processing each image, achieving classification and computational accuracy for all categories still requires many layers and neurons, resulting in considerable waste.

The CIFAR10 dataset comprises a total of 60,000 32×32 color images, categorized into 10 classes, with each class containing 6,000 images. MNN-tree employs an intuitive concept of visual similarity, grouping images of similar classes together. Categories such as dogs and cats, as depicted in Figure 1.21, are initially grouped together due to their similarities, while dissimilar categories like horses or ships are treated separately. The system employs multiple neural network models, traversing from the root model to the final child-model, to identify each category until reaching the terminal leaf node of the classification.

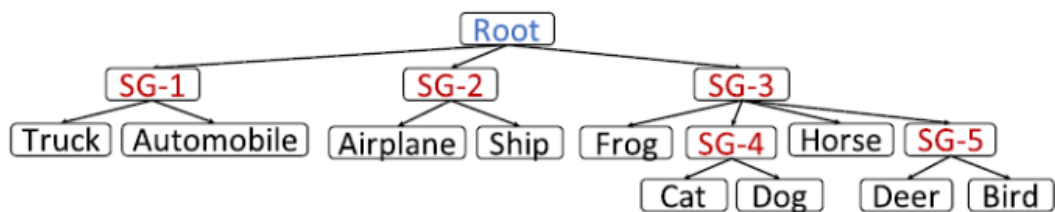


Figure 1.21 The MNN-Tree for the CIFAR-10 dataset [23].

After implementing the hierarchical convolutional neural network tree, the experimental results in [23] show significant reductions in memory usage, inference time, and energy consumption. The memory and operations are detailed in Table 1.1, indicating a reduction in memory requirements by 50% to 99%, and a 15% decrease in operations. Regarding inference time and energy consumption, as depicted in Figures 1.22 and 1.23, there's a decrease in inference time by 55% to 95%, and a reduction in energy consumption by 52% to 94%. Additionally, this approach has been successfully applied to embedded systems such as Raspberry Pi 3 and Raspberry Pi Zero.

Table 1.1 The MNN-Tree about model size, operations, and test error[23]

Dataset	Technique	Model Size (KB)	No. of Operations	Test Error
CIFAR-10	VGG-16 [13]	78,410	313 M	0.067
	VGG-Pruned [27]	28,200	206 M	0.066
	DenseNet-190 [74]	102,000	9,388 M	0.070
	CondenseNet-160 [22]	43,000	1,084 M	0.034
	MobileNet V2 [31]	8,800	100 M	0.060
	MNN-Tree⁺	806	28 M	0.079
CIFAR-100	VGG-16 [13]	78,590	207 M	0.295
	VGG-Pruned [27]	28,910	210 M	0.252
	DenseNet-190 [74]	103,000	9,400 M	0.171
	CondenseNet-160 [22]	44,500	1,080 M	0.184
	Wide ResNet-28,10 [87]	141,100	25,800 M	0.192
	MNN-Tree⁺	832	34 M	0.209
SVHN	DenseNet-190 [74]	102,000	9,388 M	0.017
	Wide ResNet-16,4 [87]	11,000	1,935 M	0.016
	MNN-Tree⁺	522	30 M	0.018
EMNIST	EDEN [88]	—	—	0.117
	MNN-Tree⁺	363	4 M	0.078
ImageNet 2012 (subset)	VGG-16 ⁺ [13]	528,000	15,300 M	0.076
	ResNet-34 ⁺ [7]	84,000	3,640 M	0.081
	DenseNet-121 ⁺ [74]	32,300	3,000 M	0.080
	SqueezeNet ⁺ [30]	5,120	837 M	0.146
	MobileNet v2 ⁺ [31]	8,820	585 M	0.104
	MNN-Tree⁺	1,872	605 M	0.124
ImageNet 2012	VGG-16 [13]	528,120	15,300 M	0.295
	ResNet-34 [7]	84,100	3,640 M	0.276
	DenseNet-121 [74]	32,400	3,000 M	0.230
	SqueezeNet [30]	5,330	837 M	0.425
	MobileNet v2 [31]	8,910	585 M	0.280
	MNN-Tree⁺	2,515	713 M	0.313

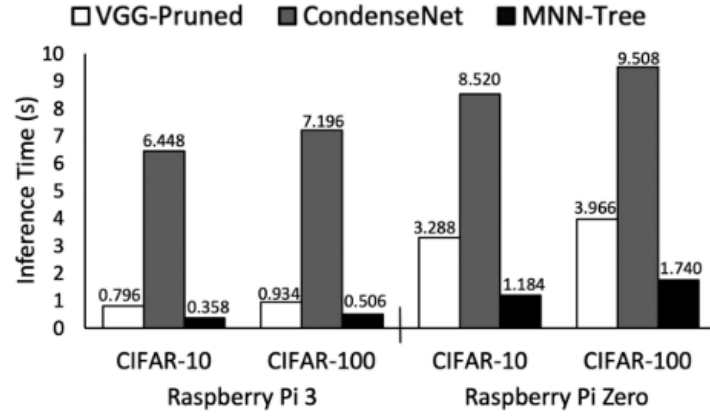


Figure 1.22 The MNN-Tree consumes the least inference time on Raspberry Pi 3 and Raspberry Pi Zero, respectively. [23].

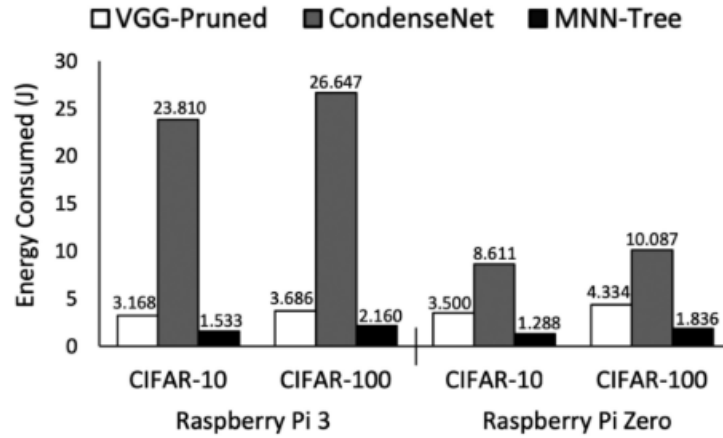


Figure 1.23 MNN-Tree consumes the least energy on both Raspberry Pi 3 and Raspberry Pi Zero, respectively [23].

In Ref. [24], a hierarchical architecture is adopted to enhance the accuracy of the network. The classification method involves categorizing based on the results of the confusion matrix. As shown in Figure 1.24, categories that are easily confused are merged into the same category before proceeding with more detailed classification. This ultimately forms a hierarchy similar to that in Figure 1.25. Compared to the original classification method, as illustrated in Figure 1.26, the accuracy is improved by approximately 2%.

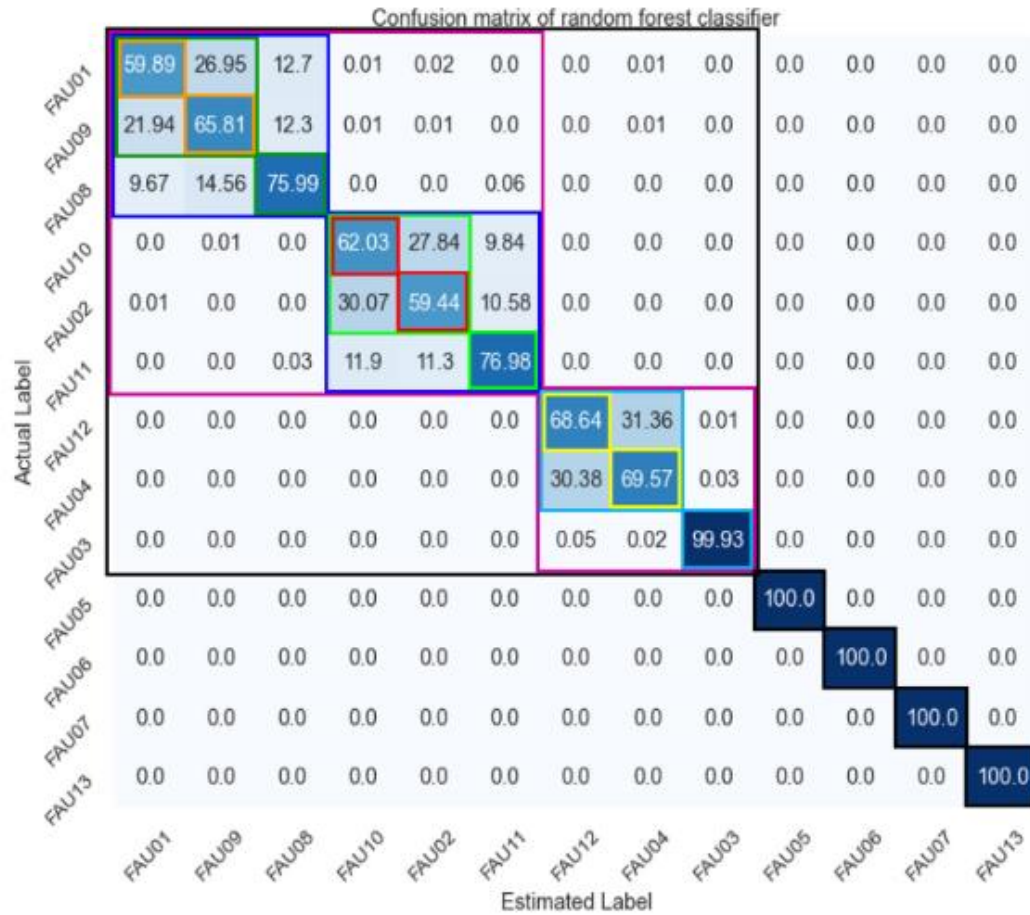


Figure 1.24 Confusion matrix on test data for random forest classifier [24].

In the hierarchical structure shown in Figure 1.25, a comparison with the confusion matrix in Figure 1.24 reveals that Level 1 prioritizes the classification of categories with higher recognition accuracy. Other easily confused categories are grouped into L1, which is then further classified into Level 2 to Level 5 based on the confusion matrix.

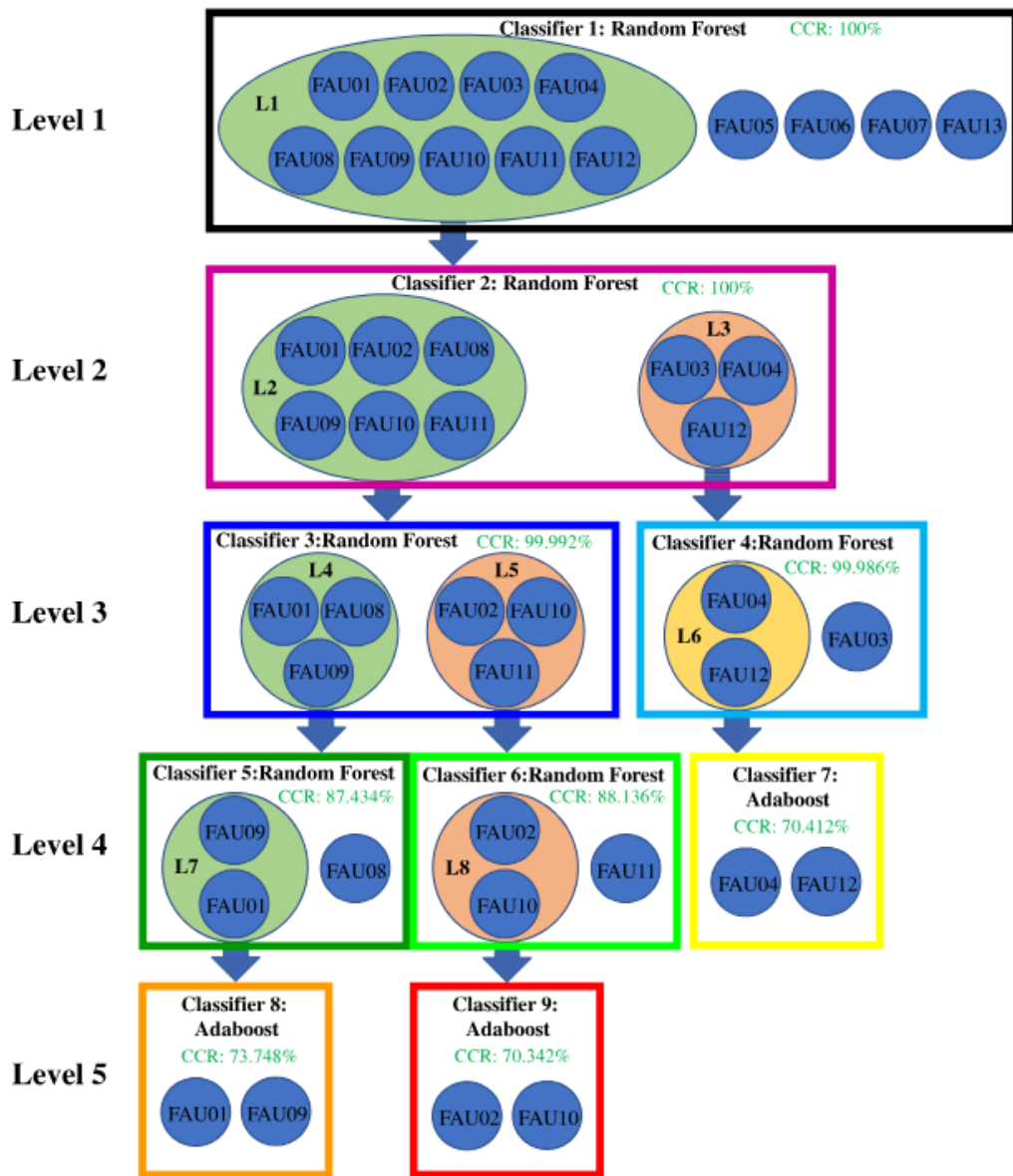


Figure 1.25 The structure of the proposed hierarchical classifier [24].

In the final result, as shown in Figure 1.26, it can be observed that the adoption of hierarchical classification has led to a significant improvement in the accuracy of most categories. Overall, there has been an enhancement in accuracy.

	Random Forest Classifier				Hierarchical Classifier			
FAU01	59.89%	26.95%	12.70%	Within Group CCR 67.23%	69.35%	24.43%	6.69%	Within Group CCR 70.67%
FAU09	21.94%	65.81%	12.30%		24.82%	68.13%	6.50%	
FAU08	9.67%	14.56%	75.99%		12.44%	12.07%	75.53%	
FAU10	62.03%	27.84%	9.84%	66.15%	66.32%	28.29%	5.20%	69.94%
FAU02	30.07%	59.44%	10.58%		26.40%	67.83%	5.83%	
FAU11	11.90%	11.30%	76.98%		12.15%	12.41%	75.66%	
FAU12	68.64%	31.36%		69.10%	70.04%	29.86%		70.10%
FAU04	30.38%	69.57%			29.91%	70.17%		
FAU03	99.93%			99.93%	99.97%			99.97%
	Overall CCR			79.59%	Overall CCR			81.62%

Figure 1.26 Detail comparison of accuracy [24].

By analyzing the confusion matrix as described in [36], incorporating the results of the confusion matrix can enhance overall resolution. When combined with the original framework, accuracy can be improved by approximately 2%, as illustrated in Figures 1.27 and 1.28. This is achieved by examining the relationships between categories in the classification results and measuring the similarity within the classifier's confusion matrix. From the two decision trees with the highest similarity, the tree with the poorer classification performance is selected, and a threshold is set. If the performance of this tree falls below the threshold, it is excluded to enhance overall classification accuracy.

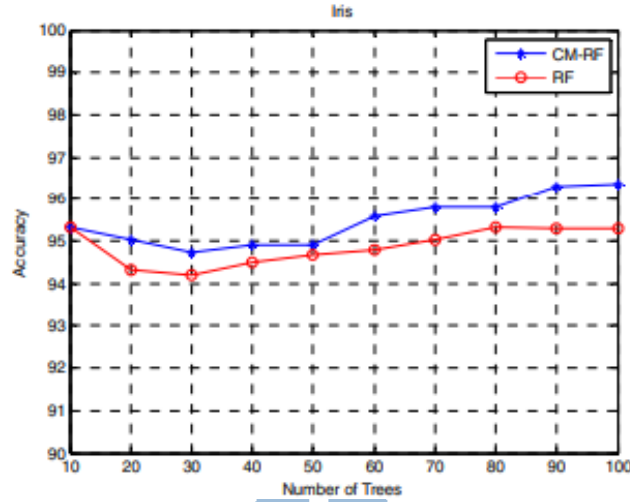


Figure 1.27 Accuracy comparison on the Iris Dataset [36].

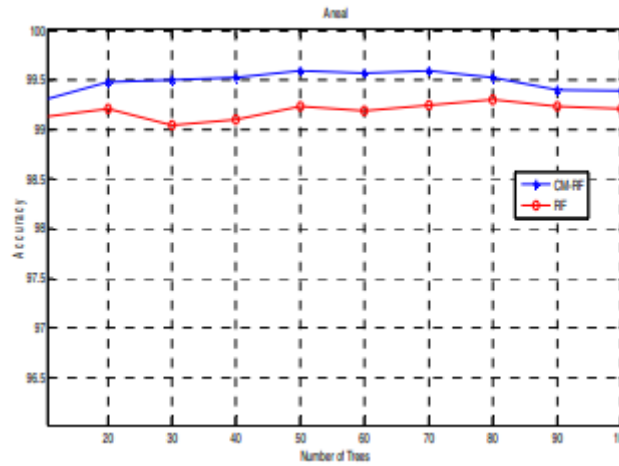


Figure 1.28 Accuracy comparison results on the Anneal Dataset [36].

In summary, the related research shows that using confusion matrix analysis for manual classification can improve accuracy. Additionally, employing a hierarchical classification framework offers advantages over a single DNN architecture that processes all features at once. The hierarchical framework, in consideration of hardware constraints, can reduce unnecessary computational energy consumption for certain categories. Therefore, adopting a hierarchical structure in conjunction with confusion matrix analysis presents numerous benefits.

1.4 Introduction to Class Imbalance Training Methods

In numerous datasets like CIFAR and ImageNet, category labels are typically distributed evenly. However, in real-world datasets, a common occurrence is skewed distribution, often presenting a so-called long-tail distribution. This indicates that within the entire dataset, some categories possess the majority of samples, while others have relatively fewer samples. Such datasets, when used to train models, often result in poor performance for minority categories.

Figure 1.29 illustrates that when training models on such imbalanced datasets, they tend to lean towards dominant categories. Some methods attempt to address this issue by adjusting the weights within the loss function. However, these methods typically perform poorly on highly imbalanced datasets. In Ref. [26] proposed a theoretical framework aiming to quantify the effective sample size by considering data overlap and utilizing the inverse effective sample size to reweight the loss function. They introduced a category balancing term through this approach to improve model performance. In experiments, training with this category balancing loss led to enhanced model performance.

In [26], a method for re-weighting each category is proposed, which can quickly improve the accuracy of imbalanced datasets. Figure 1.30 introduces a method to calculate the degree of data overlap and uses this method to quantify the number of effective samples. By calculating the effective sample size, a method for computing the effective sample size can be obtained. Finally, the weights are recalibrated using the effective sample size to achieve category balance.

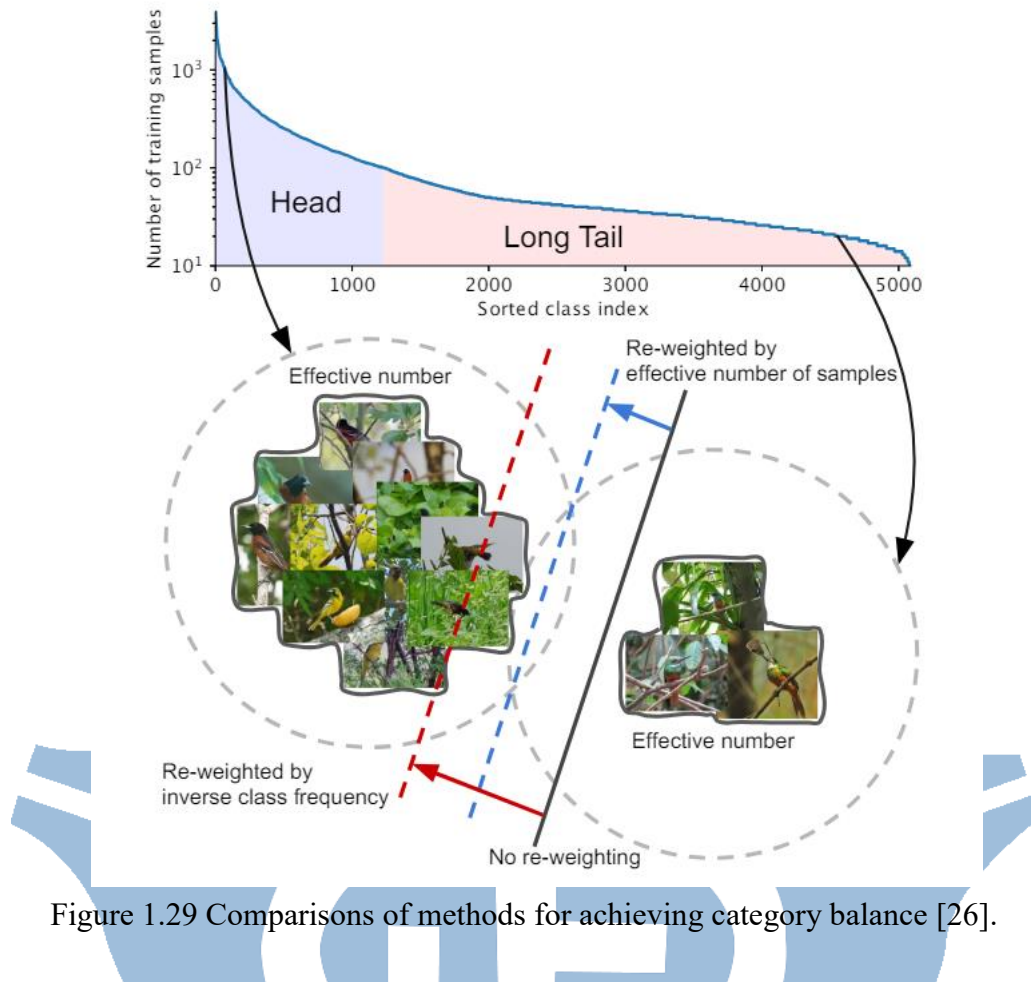


Figure 1.29 Comparisons of methods for achieving category balance [26].

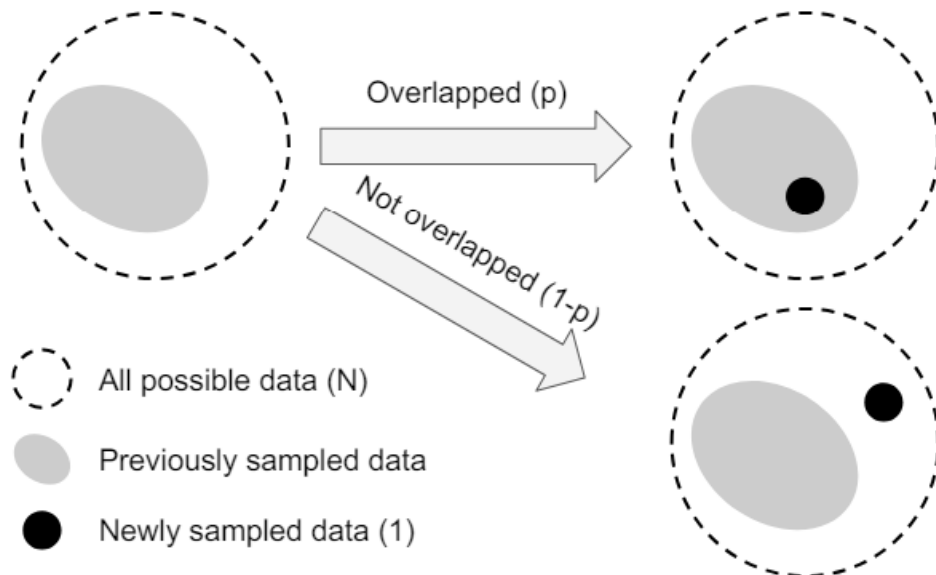


Figure 1.30 The probability distribution of the data [26].

In Table 1.2, a higher Imbalance value indicates a more imbalanced dataset. The table compares the performance of various loss functions on imbalanced datasets, including Softmax, Sigmoid, Focal Loss with different γ values, and Class-Balanced Loss with optimal hyperparameters. The γ value in the Focal Loss function represents the emphasis on difficult versus easy samples, and this holds true for the γ value in the Class-Balanced Loss as well. The β value in the Class-Balanced Loss indicates the ease of distinguishing each class: the larger the β value, the easier it is to distinguish between classes, and vice versa.

From the table, it can be observed that the Class-Balanced method achieves lower loss values compared to other loss functions, indicating its superiority. Additionally, the data shows that for the CIFAR-10 dataset, the optimal hyperparameter β value for best classification is 0.9999, while for the CIFAR-100 dataset, the optimal β values vary and are generally smaller under different imbalance factors.

Table 1.2 The classification error rates of ResNet-32 with different loss functions and class balancing were evaluated on long-tail CIFAR-10 and CIFAR-100 datasets. [26]

Dataset Name	Long-Tailed CIFAR-10						Long-Tailed CIFAR-100					
Imbalance	200	100	50	20	10	1	200	100	50	20	10	1
Softmax	34.32	29.64	25.19	17.77	13.61	6.61	65.16	61.68	56.15	48.86	44.29	29.07
Sigmoid	34.51	29.55	23.84	16.40	12.97	6.36	64.39	61.22	55.85	48.57	44.73	28.39
Focal ($\gamma = 0.5$)	36.00	29.77	23.28	17.11	13.19	6.75	65.00	61.31	55.88	48.90	44.30	28.55
Focal ($\gamma = 1.0$)	34.71	29.62	23.29	17.24	13.34	6.60	64.38	61.59	55.68	48.05	44.22	28.85
Focal ($\gamma = 2.0$)	35.12	30.41	23.48	16.77	13.68	6.61	65.25	61.61	56.30	48.98	45.00	28.52
Class-Balanced	31.11	25.43	20.73	15.64	12.51	6.36*	63.77	60.40	54.68	47.41	42.01	28.39*
Loss Type	SM	Focal	Focal	SM	SGM	SGM	Focal	Focal	SGM	Focal	Focal	SGM
β	0.9999	0.9999	0.9999	0.9999	0.9999	-	0.9	0.9	0.99	0.99	0.999	-
γ	-	1.0	2.0	-	-	-	1.0	1.0	-	0.5	0.5	-

In [25], a method is proposed to minimize margin generalization boundaries by computing Label-Distribution-Aware Margin Loss, aiming to address class imbalance issues. The core idea is to enlarge margins between minority classes. Figure 1.31 elaborates on this concept, where the dashed lines represent ideal classification boundaries, blue points denote majority classes, and green points denote minority

classes. The dashed lines connect the closest data points from each class to the boundary, ensuring equal distances from these points to the dashed line (representing the ideal classification boundary). The approach aims to position the dashed line (ideal boundary) as close as possible to the majority classes, thereby alleviating classification difficulties for minority classes. Additionally, a method named delayed reweighting (DRW) is introduced, where the model initially learns feature representations (initial stage) before applying re-weighting techniques to reduce class losses.

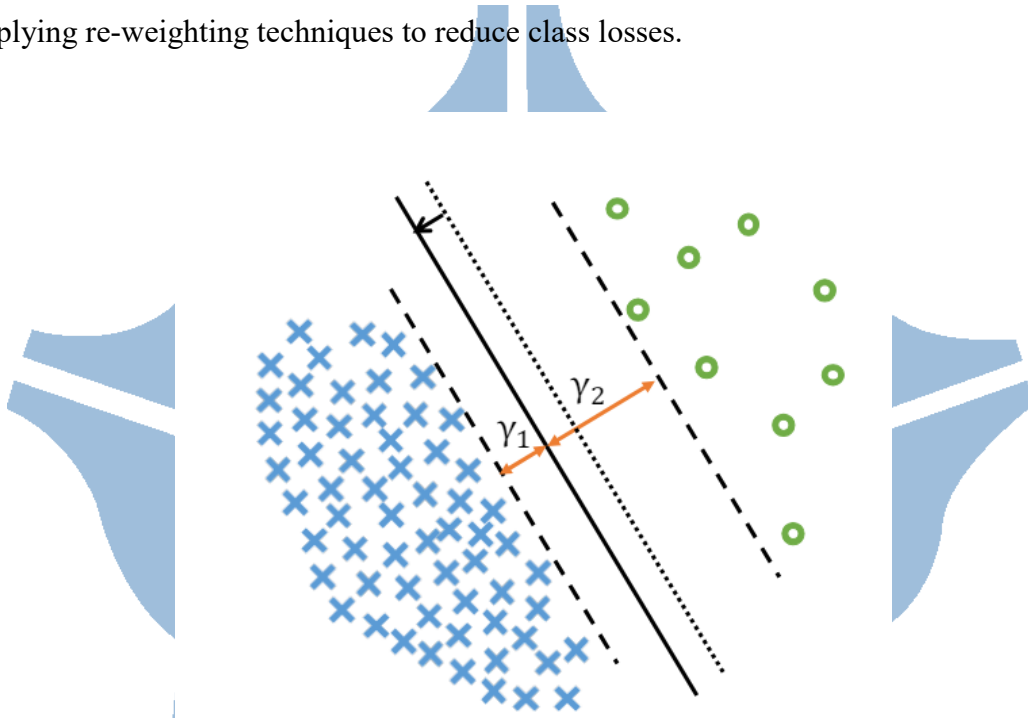


Figure 1.31 Example of imbalanced binary classification [25].

Table 1.3 shows the top-1 validation error rates of various methods on the imbalanced versions of CIFAR-10 and CIFAR-100. Notably, after adopting the method outlined in [25], the overall performance demonstrates better results compared to other methods.

Table 1.3 The classification error rates of ResNet-32 with different loss functions and class balancing were evaluated on long-tail CIFAR-10 and CIFAR-100 datasets. [25]

Dataset	Imbalanced CIFAR-10				Imbalanced CIFAR-100			
Imbalance Type	long-tailed		step		long-tailed		step	
Imbalance Ratio	100	10	100	10	100	10	100	10
ERM	29.64	13.61	36.70	17.50	61.68	44.30	61.45	45.37
Focal [Lin et al., 2017]	29.62	13.34	36.09	16.36	61.59	44.22	61.43	46.54
LDAM	26.65	13.04	33.42	15.00	60.40	43.09	60.42	43.73
CB RS	29.45	13.21	38.14	15.41	66.56	44.94	66.23	46.92
CB RW [Cui et al., 2019]	27.63	13.46	38.06	16.20	66.01	42.88	78.69	47.52
CB Focal [Cui et al., 2019]	25.43	12.90	39.73	16.54	63.98	42.01	80.24	49.98
HG-DRS	27.16	14.03	29.93	14.85	-	-	-	-
LDAM-HG-DRS	24.42	12.72	24.53	12.82	-	-	-	-
M-DRW	24.94	13.57	27.67	13.17	59.49	43.78	58.91	44.72
LDAM-DRW	22.97	11.84	23.08	12.19	57.96	41.29	54.64	40.54



1.5 Compression Techniques on Neural Networks

In CNNs, extensive floating-point operations are required before the output of each layer, leading to significant consumption of computational time and power. Due to their typically large parameter counts, hardware implementations necessitate substantial memory usage.

To minimize resource consumption on hardware, model compression becomes imperative. S. Han [30] proposed the method of Deep Compression, which involves three steps: pruning, quantization, and Huffman coding, as illustrated in Figure 1.32. During pruning, weights in the original network below a certain threshold are removed. The resulting sparse network is then retrained until convergence. Next, K-means clustering is used to group the weights, with each group assigned a common centroid, and finally, fixed-point processing is carried out.

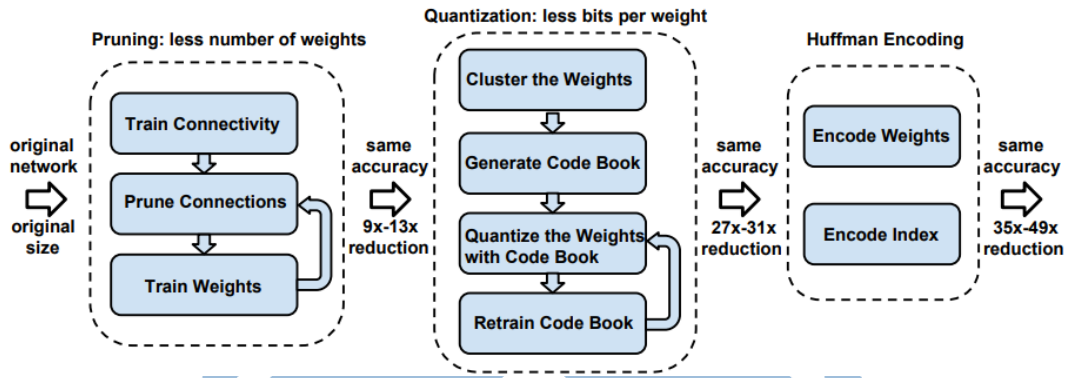


Figure 1.32 The three-stage compression process of pruning, quantization, and Huffman coding [27].

During weight updates, gradients within the same cluster are aggregated, multiplied by the learning rate (lr), and then adjusted by subtracting the shared centroid, as depicted in Figure 1.33. Finally, weights and indices are further compressed using Huffman coding. Experimental results indicate a trade-off relationship between

compression rate and accuracy, with the compression rate dependent on hardware specifications. Deep compression effectively reduces memory usage in large network architectures such as AlexNet and VGGNet but does not significantly enhance speed.

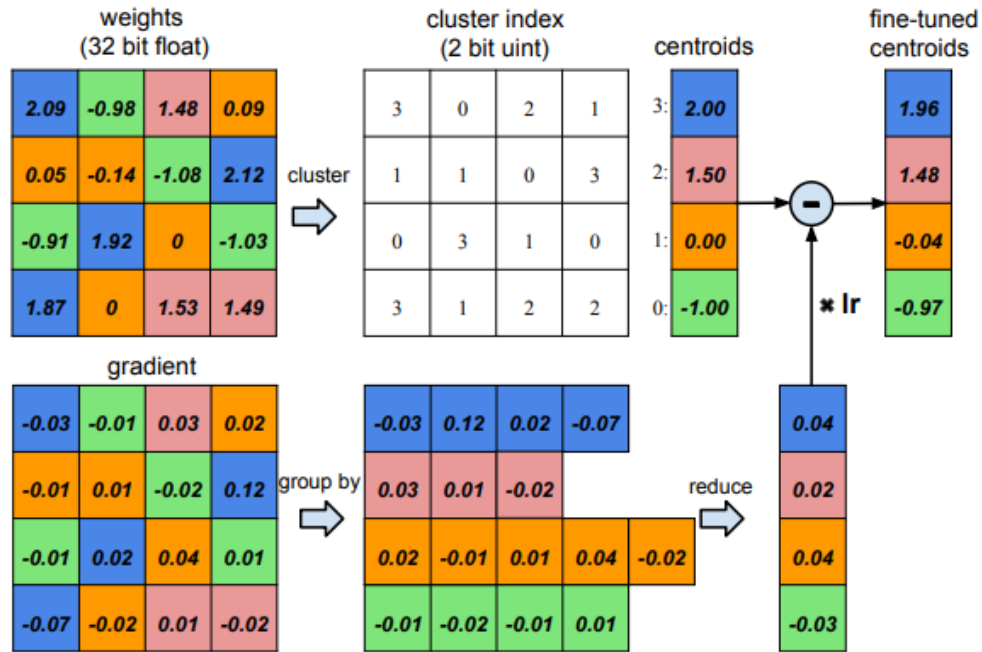


Figure 1.33 Weight sharing by K-means clustering [27].

Knowledge distillation [28] is a technique used to compress complex neural network models. Its fundamental concept lies in training a large-scale model (teacher model) to grasp the features and patterns of the data, and then transferring this knowledge to a smaller, simpler model (student model) in some way. Throughout this process, the student model endeavors to mimic the behavior of the teacher model, thus acquiring representations similar to the original data, rather than learning directly from the training data. Through this approach, it is possible to reduce the model size and computational resource consumption while maintaining relatively high performance levels.



Figure 1.34 Two new exit points are added to the model [29].

Early model termination is an optimization method for the computational efficiency of deep learning models [29]. Its core idea is to establish early exit points in the model, so that when the model has achieved sufficient accuracy in recognizing input data, computation can be terminated in advance to save computational resources and time, as represented in Figure 1.34. Specifically, by setting an accuracy threshold, when the model's confidence in predicting a certain input data exceeds this threshold, computation will be terminated prematurely and the prediction result will be returned. Conversely, if the accuracy falls below the threshold, the model will continue computation until the final prediction result. As a result, when applied to models capable of accurately predicting data, unnecessary computational overhead can be avoided, thereby improving computational efficiency.

In order to maximize reduction in memory usage, the Binary Neural Network (BNN) [29] was proposed in 2016. Through Equation 1.7 during training, this formula is utilized to constrain the originally 32-bit floating-point weights and activations to either 1 or -1. The results demonstrate that BNN is highly effective in reducing the memory footprint of parameters and speeding up inference time. Despite the favorable compression ratio achieved by BNN, its accuracy is easily compromised by noise when identifying complex and large datasets. Consequently, this method results in significant loss of information.

$$x_b = \text{sign}(x) = \begin{cases} 1, & \text{if } x > 0 \\ -1, & \text{otherwise} \end{cases} \quad (1.7)$$

In recent years, there have been researches proposing improvements to Binary Neural Networks (BNNs), such as IR-NET [30]. They incorporate Libra Parameter Binarization (Libra-PB) during forward propagation to increase information entropy, which leads to a more balanced weight distribution. The Libra-PB function, represented by Equation 1.8, quantifies the probability distribution of binarized weights and activations. Equation 1.9 is then used to calculate the information entropy, aiming for higher values indicative of greater disorder and thus a more balanced weight distribution. To minimize information loss, the Libra-PB function is defined as in Equation 1.10.

To preserve the information derived from the loss function during backpropagation, an Error Decay Estimator (EDE) is introduced in the backward pass. This reduces the discrepancy between the forward binarization function and the backward approximation function, ensuring all parameters receive reasonable updates. Through these methods, the effectiveness of BNN is enhanced.

$$f(b) = \begin{cases} p, & \text{if } b = +1 \\ 1 - p, & \text{if } b = -1 \end{cases} \quad (1.8)$$

$$H(Q_x(x)) = H(B_x) = -p \ln(p) - (1 - p) \ln(1 - p) \quad (1.9)$$

$$\min J(Q_x(x)) - \lambda H(Q_x(x)) \quad (1.10)$$

PACT (Parameterized Clipping Activation for Quantized Neural Networks) is a method for quantized neural networks' activation function [31]. In traditional quantized neural networks, commonly used activation functions tend to degrade in performance when subjected to low-bit quantization (such as binarization or ternarization), as they are ill-suited for representation on a limited number of bits. PACT addresses this issue by introducing a trainable parameter, which can dynamically adjust during training, making the activation function more suitable for low-bit quantization representation.

The core concept of PACT involves introducing a clipping mechanism within the activation function, controlled by a learnable parameter. Specifically, PACT's activation function is represented by Equation 1.11. During training, optimization methods like gradient descent are employed to update α , enabling the model to adaptively learn the optimal clipping range.

$$y = PACT(x) = 0.5(|x| - |x - \alpha| + \alpha) = \begin{cases} 0, & x \in (-\infty, 0) \\ x, & x \in [0, \alpha) \\ \alpha, & x \in [\alpha, +\infty] \end{cases} \quad (1.11)$$

In order to enhance the flexibility of quantization, the DoReFaNet quantization method was proposed in [32]. Its quantization implementation is illustrated in Equation 1.12. DoReFaNet allows weights, activations, and gradients to be quantized to arbitrary bit widths, thus increasing the flexibility of quantization. It is a non-uniform quantization method, as shown in Equations 1.12 and 1.13, where the input value x is first transformed using the tanh function, and then quantized into k bits. The Equation for low-bit width weight quantization is depicted in Equation 1.13, where the max function is taken from all weights in the same layer. The tanh function is used here to

confine the numerical range of weights to $[-1, 1]$, ensuring that the quantized weights are k -bit fixed-point numbers between -1 and 1.

$$\text{quantized}_k(x) = \frac{1}{2^{k-1}} \text{round}((2^k - 1)x) \quad (1.12)$$

$$f_w^k(r) = 2 \text{quantize}_k \left(\frac{\tanh(r)}{2 \max(|\tanh(r)|)} + \frac{1}{2} \right) - 1 \quad (1.13)$$

Failure to utilize bounded activation functions during activation quantization can impede the convergence of the neural network model during training. Thus, it is imperative to ensure that input activations y are adjusted within a specific range. In this regard, bounded activation functions are employed to facilitate convergence post-quantization. Furthermore, the choice of bounded activation functions indirectly impacts the accuracy of the network, as evidenced by experimental findings. Three effective bounded activation functions, denoted as $h(y)$ in Equations 1.14, 1.15, and 1.16, have been identified.

$$h(y) = \frac{\tanh(y)+1}{2} \quad (1.14)$$

$$h(y) = \text{clip}(y, 0, 1) \quad (1.15)$$

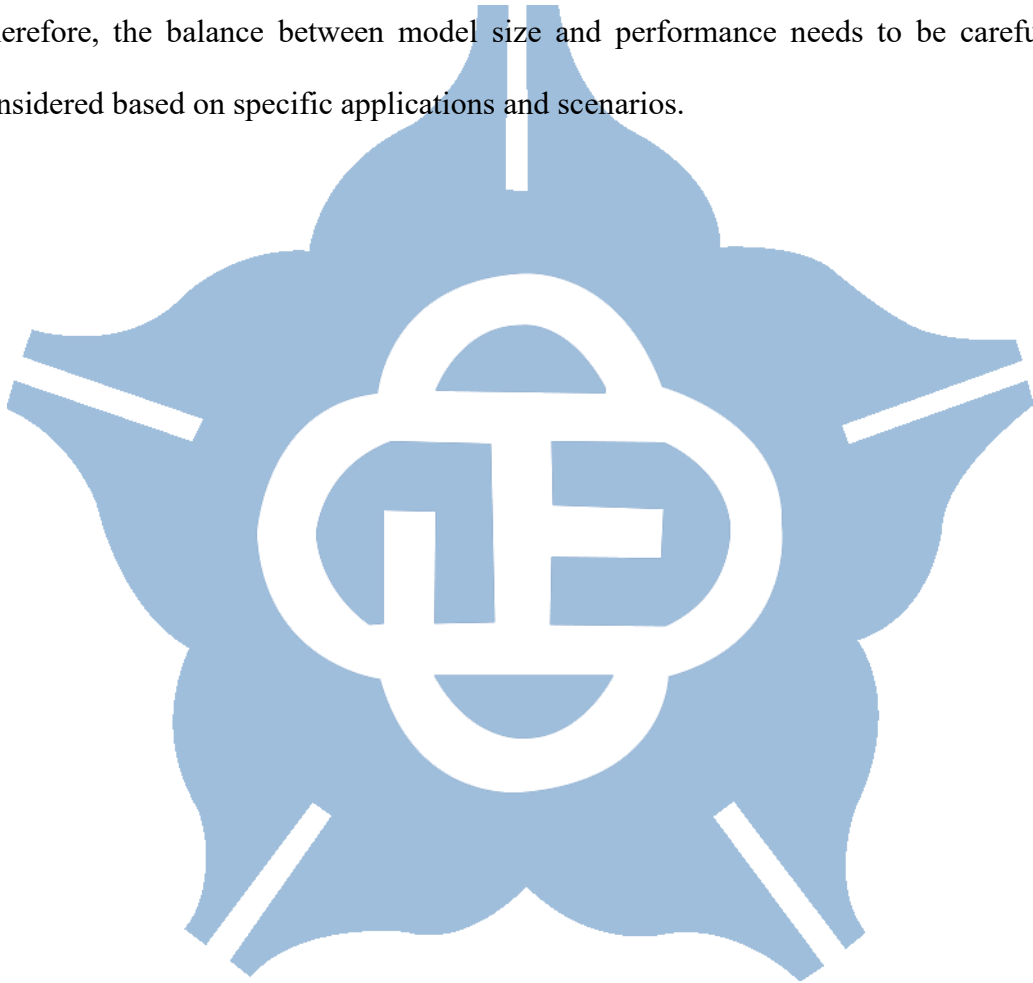
$$h(y) = \min(1, |y|) \quad (1.16)$$

Equation 1.17 demonstrates the quantization of activations with low bit-width. As the bounded activation functions mentioned earlier have already scaled the activations, only the output results of $h(y)$ need to be directly quantized. The results indicate that if the bit-width of activations is smaller than that of weights, the performance of the network, such as classification accuracy, will significantly decrease.

$$f_a^k(r) = \text{quantize}_k(r) \quad (1.17)$$

According to the aforementioned quantization methods, it can be observed that

quantization greatly assists in compressing models. Neural networks often suffer from the issue of over-parameterization, meaning there are many unnecessary parameters. This makes running them on edge devices or some embedded systems difficult. However, through quantization, memory usage can be significantly reduced, and in some cases, the network's performance can even be accelerated. Nevertheless, excessive model compression may lead to a substantial decline in the model's accuracy. Therefore, the balance between model size and performance needs to be carefully considered based on specific applications and scenarios.



1.6 Motivation

Numerous psychological studies have indicated a positive correlation between facial expressions and psychological states. However, traditional manual computation methods face certain difficulties in recognition, as their complex computational processes render facial expression recognition in real-world applications less effective and thus not widely adopted. With the advent of deep learning technology, its remarkable performance in real-world facial expression recognition has been realized, leading to its widespread application in modern society. Nonetheless, the immense computational demands of deep neural networks pose significant challenges to edge devices such as cars and robots. Therefore, there is a need for a method that balances performance and accuracy.

This thesis constructs a Hierarchical Depthwise Separable Convolutional Neural Network (HDSC) for a facial expression recognition dataset. The model introduces a hierarchical classification approach and incorporates depthwise separable convolutions, significantly reducing computational load and parameter count. During the training phase, the dataset's imbalance was addressed to ensure that minority classes maintained a certain level of accuracy in the overall results. Moreover, to lower hardware resource costs, weight and activation quantization techniques were employed to reduce memory space requirements.

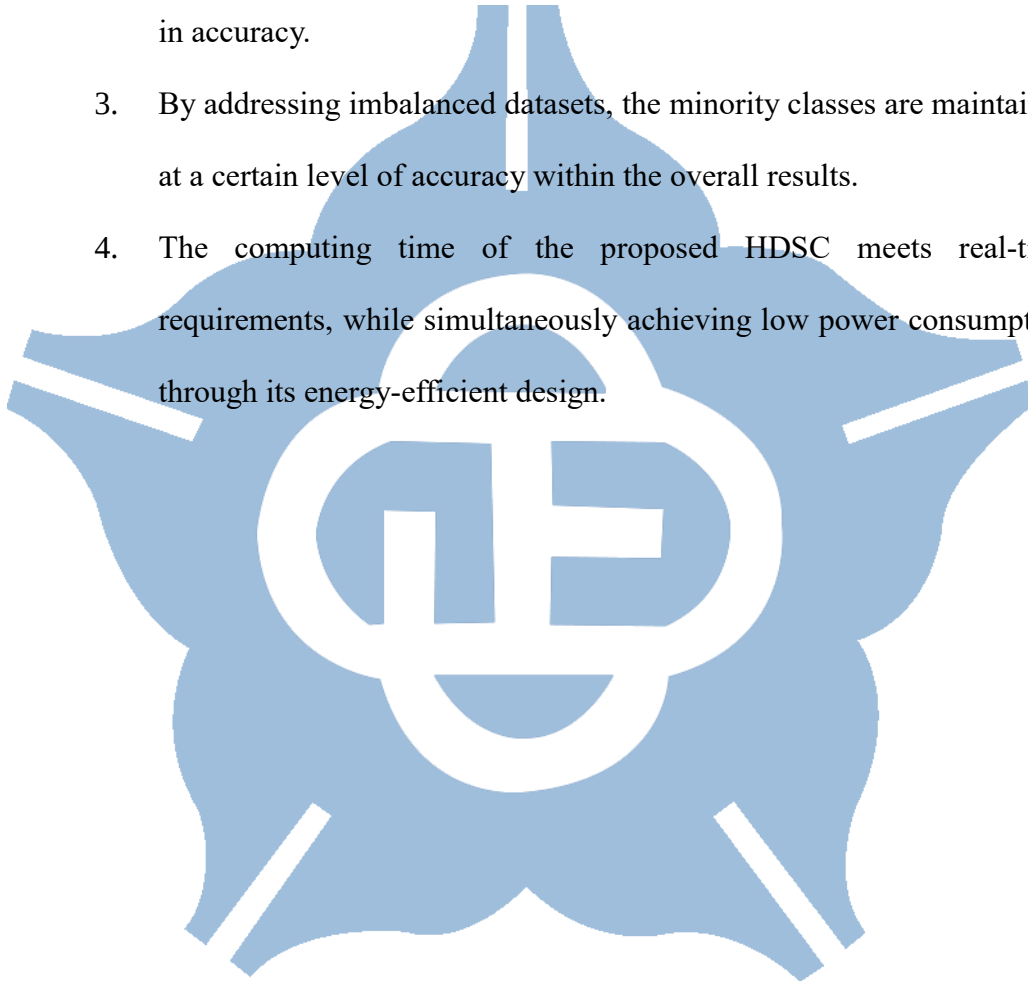
Finally, the hardware circuit was implemented using 16-nanometer technology to perform computations, providing a hardware accelerator for emotional robot applications. The hierarchical structure reduced energy consumption, further enhancing energy efficiency.

The main contributions in this thesis are summarized as follows:

1. By designing a hierarchical convolutional neural network and

conducting experiments, it was discovered that compared to traditional methods, it further reduces the total number of parameters required by the neural network while maintaining similar accuracy.

2. By integrating depthwise separable convolution (DSC) into the architecture, it reduces memory usage and FLOPs (floating-point operations) compared to using traditional CNNs, with only minimal loss in accuracy.
3. By addressing imbalanced datasets, the minority classes are maintained at a certain level of accuracy within the overall results.
4. The computing time of the proposed HDSC meets real-time requirements, while simultaneously achieving low power consumption through its energy-efficient design.



1.7 Thesis Organization

Chapter 1 discusses the application of facial expression recognition and compares the differences between traditional methods and deep learning networks. Additionally, the HCNN architecture is introduced. Furthermore, discussions are held regarding the imbalance in datasets and exploration of model compression methods.

Chapter 2 introduces the proposed hierarchical depthwise separable convolutional neural network and compares its handling of imbalanced data on the FER2013 dataset. It also compares its proposed architecture with related research on facial expression recognition.

Chapter 3 presents the hardware implementation of the proposed hierarchical depthwise separable convolutional neural network and discusses the utilization of hardware resources.

Finally, Chapter 4 concludes and outlines future work.

Chapter 2 Software Implementation

2.1 Architecture Overview

This thesis investigates the use of convolutional neural networks for emotion recognition, utilizing the FER2013 dataset. The FER2013 dataset encompasses seven distinct emotions: happy, surprise, disgust, sadness, fear, angry, and neutral, as illustrated in Figure 1.6. The design flow to construct a hierarchical convolutional neural network is depicted in Figure 2.1.

To mitigate the confusion in emotion recognition classification, three models were adopted: DoorNet, ImproveNet0, and ImproveNet1. Due to the unavoidable data imbalance among categories, the training process incorporated data balancing techniques to enhance the classification accuracy for each emotion category. Additionally, as mentioned in Section 1.1, image flipping techniques were employed during the training process to improve the model's recognition capability across all categories.

Moving forward, by referring to the current state-of-the-art, set up the model after training with a final accuracy goal of 69%, in order to complete the construction of the hierarchical convolutional neural network. Following the completion of model construction, the subsequent step involved model quantization. Given the requirement to deploy the model on edge hardware such as robots, this thesis aims to reduce computational demands while preserving high precision. Following extensive validation, DoReFa-Net was selected as the quantization method, which permits model parameters to be quantized to arbitrary bit-widths, offering greater flexibility than traditional quantization methods.

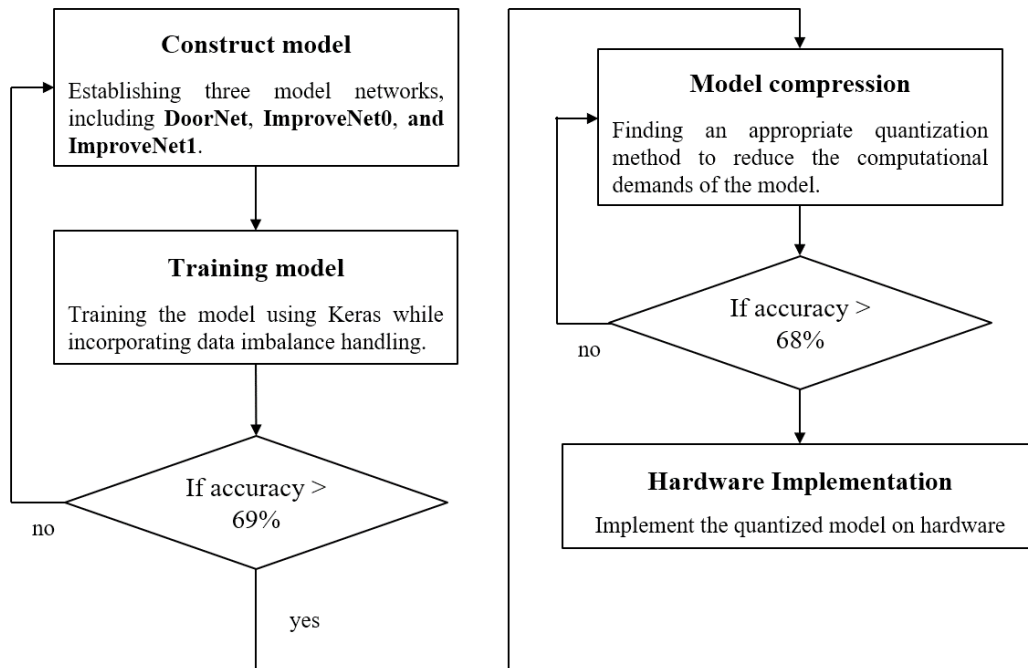


Figure 2.1 Flowchart for designing the hierarchical convolutional neural network software model.

2.2 Implementation Hierarchical convolutional neural network on software and analysis

As discussed in Section 1.3, previous researchers have initially classified easily recognizable categories through the analysis of the confusion matrix, followed by a more detailed differentiation of categories that are easily confused. This approach aims to enhance the overall classification efficiency of the model. This thesis adopts this method by analyzing the convolutional model's confusion matrix, initially identifying highly distinguishable categories, followed by grouping and further subdividing the easily confused categories.

This thesis first examines the initially proposed Type-1 model. Following the first convolutional operation, ReLU activation and batch normalization were incorporated to enhance the model's ability to understand data regularities. To prevent rapid feature reduction, max-pooling was not introduced after the first layer, as illustrated in Figure 2.2. Zero-padding was added at the beginning of each subsequent layer to prevent the images from becoming too small for convolutions with larger kernel sizes. Next, depthwise separable convolution (DSC) was employed, which includes both depthwise convolution and pointwise convolution, to reduce the overall computational load. Following the pointwise convolution, ReLU activation and batch normalization were again added to improve the model's learning capabilities, as depicted in Figure 2.3.

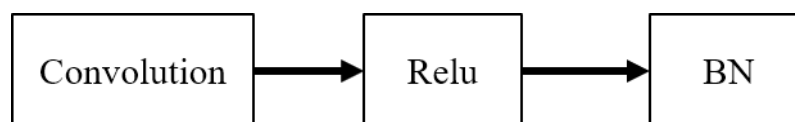


Figure 2.2 The entire operation of first layer.

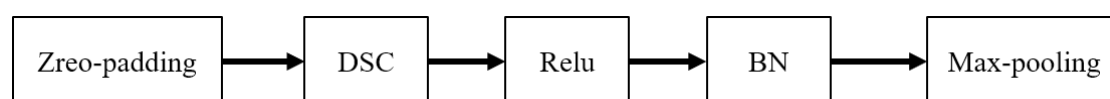


Figure 2.3 The entire operation of each layer, except for the first layer

Next, the impact of different kernel sizes on the accuracy of the Type-1 model is analyzed, with specific results presented in Table 2.1. It was observed that from case1 to case4, accuracy steadily increased with the enlargement of the kernel size. However, beyond case4, further increases in kernel size did not result in a significant improvement in accuracy and instead led to a decline. Consequently, subsequent discussions will use case4, which achieved the highest accuracy, as the basis.

Table 2.1 Accuracy of Type-1 with different kernel sizes.

Layer	Layer1 (Conv)	Layer2 (DSC)	Layer3 (DSC)	Layer4 (DSC)	Layer5 (DSC)	Parameters	Accuracy
Filters	32	64	64	64	128		
Kernel size (Original)	3×3	3×3	3×3	3×3	3×3	23,623	60.77%
Kernel size (case1)	3×3	5×5	3×3	3×3	3×3	24,135	61.03%
Kernel size (case2)	3×3	5×5	5×5	3×3	3×3	25,159	60.35%
Kernel size (case3)	3×3	5×5	5×5	5×5	3×3	26,183	61.91%
Kernel size (case4)	3×3	5×5	5×5	5×5	5×5	27,207	62.55%
Kernel size (case5)	5×5	5×5	5×5	5×5	5×5	27,719	61.09%
Kernel size (case6)	7×7	7×7	7×7	7×7	7×7	33,863	61.41%

In Table 2.2, the previously determined kernel sizes for each layer were retained, and the impact of the number of filters on accuracy was discussed. For the purpose of subsequent hardware implementation, Case 1 explored the possibility of reducing the number of filters in the initial layers. Although this approach effectively reduced memory usage, it resulted in a significant decrease in accuracy compared to the original settings. Therefore, subsequent tests maintained the number of filters in the initial layers

at 32. Next, in Case 2 and Case 3, the effect of increasing the number of filters in the later layers was tested. The results showed that Case 2 yielded improvements over the original settings, but also significantly increased the number of parameters. In contrast, Case 3 performed worse than Case 2, despite having even more parameters. Consequently, the Type-1 architecture presented in Table 2.3, which is based on Case 2, was ultimately adopted.

Table 2.2 Accuracy of Type-1 model with different filter sizes.

Layer	Layer1 (Conv)	Layer2 (DSC)	Layer3 (DSC)	Layer4 (DSC)	Layer5 (DSC)	Parameters	Accuracy
Kernel size	3×3	5×5	5×5	5×5	5×5		
Filters size (Original)	32	64	64	64	128	27,207	62.55%
Filters size (case1)	16	64	64	64	128	19,040	59.44%
Filters size (case2)	32	64	64	128	256	59,399	64.43%
Filters size (case3)	32	64	128	128	256	73,671	64.18%

Table 2.3 The Type-1 model architecture.

Operation	Data size of input (width × height × in_channel)	Data size of output (width × height × in_channel)	Kernel size
Convolution 1	48×48×1	46×46×32	3×3×32
DW Conv1	46×46×32	42×42×32	5×5×32
PW Conv1	42×42×32	42×42×64	1×1×64
Max-pooling1	42×42×64	21×21×64	
DW Conv2	21×21×64	21×21×64	5×5×64
PW Conv2	21×21×64	21×21×64	1×1×64

Max-pooling2	21×21×64	10×10×64	
DW Conv3	10×10×64	10×10×64	5×5×64
PW Conv3	10×10×64	10×10×128	1×1×128
Max-pooling3	10×10×128	5×5×128	
DW Conv4	5×5×128	5×5×128	5×5×128
PW Conv4	5×5×128	5×5×256	1×1×256
Max-pooling4	5×5×256	2×2×256	
Global average pooling	2×2×256	1×1×256	
Fully connected	1×1×256	1×1×7	

After adopting the Type-1 architecture, the confusion matrix obtained from the statistics is shown in Figure 2.4. Analyzing the confusion matrix reveals that "happy" and "surprise" are rarely confused with other categories, whereas "disgust" and "angry" are particularly prone to being confused with the other five categories. Therefore, for these two categories, a more refined recognition method needs to be employed to improve the classification accuracy. Different colored boxes have been roughly added in the figure to identify the classifications, with the same color used for the same category.

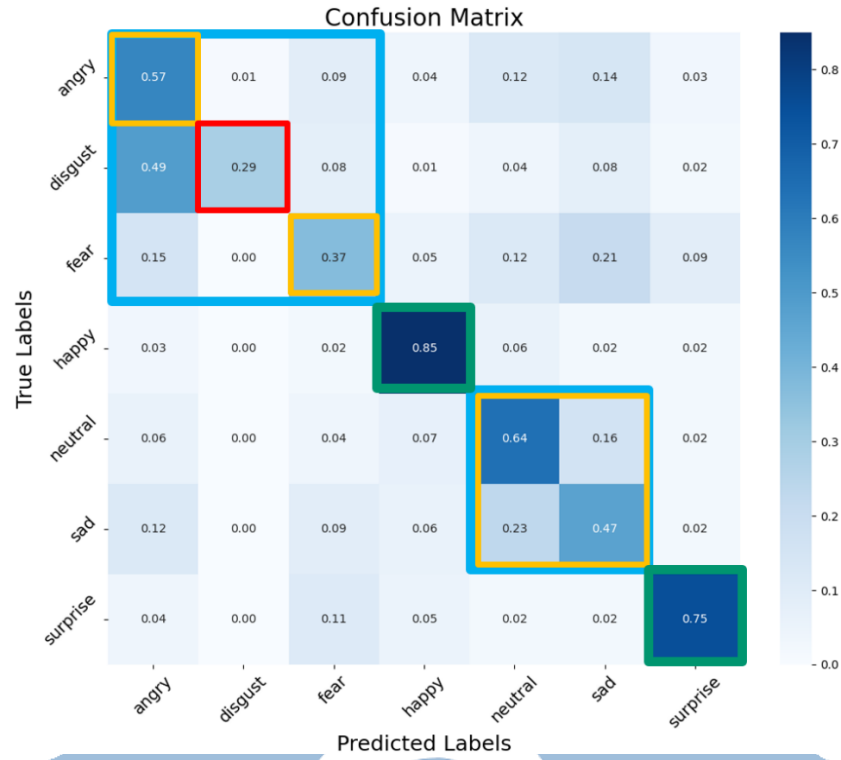


Figure 2.4 The confusion matrix of Type-1 structure.

Considering the aforementioned discussion, propose a Type-2 architecture as illustrated in Figure 2.5, uses doorNet in the first layer to distinguish positive expressions like happy and surprise, categorizing the remaining classes as negative expressions. Due to the significant confusion between "disgust" and "angry," ImproveNet0 in the second layer separately distinguishes disgust, followed by ImproveNet1 to recognize the remaining categories individually. The Type-2 structures of the three models in this architecture are detailed in Tables 2.4, 2.5, and 2.6.

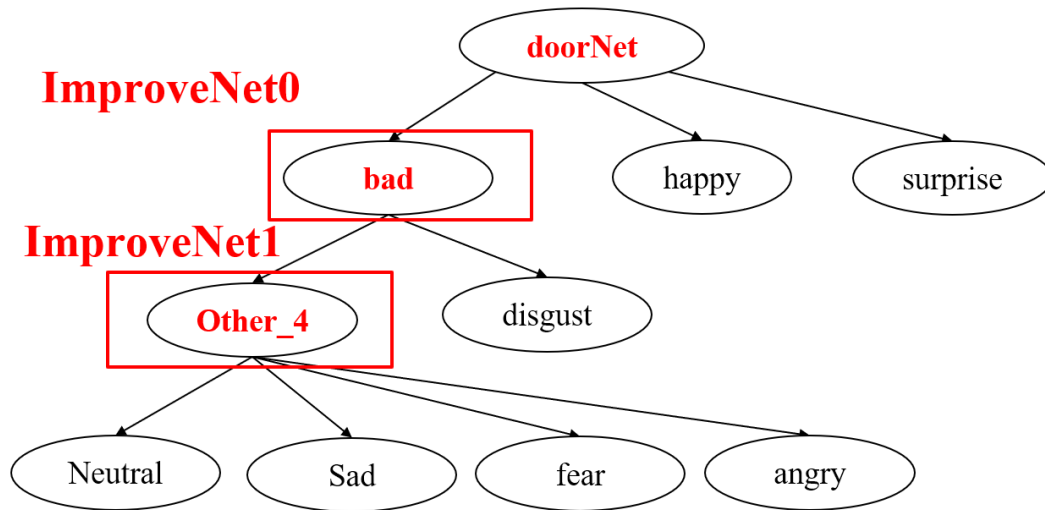


Figure 2.5 Type-2 classification from start to finish.

Table 2.4 The doorNet model architecture in Type-2 classification.

Operation	Data size of input (width × height × in_channel)	Data size of output (width × height × in_channel)	Kernel size
Convolution 1	48×48×1	46×46×32	3×3×3
DW Conv1	46×46×32	42×42×32	5×5×32
PW Conv1	42×42×32	42×42×64	1×1×64
Max-pooling1	42×42×64	21×21×64	
DW Conv2	21×21×64	17×17×64	5×5×64
PW Conv2	17×17×64	17×17×64	1×1×64
Max-pooling2	17×17×64	8×8×64	
DW Conv3	8×8×64	4×4×64	5×5×64
PW Conv3	4×4×64	4×4×128	1×1×128
Max-pooling3	4×4×128	2×2×128	
Global average pooing	2×2×128	1×1×128	
Fully connected	1×1×128	1×1×3	

Table 2.5 The ImproveNet0 model architecture in Type-2 classification.

Operation	Data size of input (width × height × in_channel)	Data size of output (width × height × in_channel)	Kernel size
Convolution 1	48×48×1	46×46×32	3×3×32
DW Conv1	46×46×32	42×42×32	5×5×32
PW Conv1	42×42×32	42×42×64	1×1×64
Max-pooling1	42×42×64	21×21×64	
DW Conv2	21×21×64	17×17×64	5×5×64
PW Conv2	17×17×64	17×17×64	1×1×64
Max-pooling2	17×17×64	8×8×64	
Global average pooling	8×8×64	1×1×64	
Fully connected	1×1×64	1×1×2	

Table 2.6 The ImproveNet1 model architecture in Type-2 classification.

Operation	Data size of input (width × height × in_channel)	Data size of output (width × height × in_channel)	Kernel size
Convolution 1	48×48×1	46×46×32	3×3×32
DW Conv1	46×46×32	42×42×32	5×5×32
PW Conv1	42×42×32	42×42×64	1×1×64
Max-pooling1	42×42×64	21×21×64	
DW Conv2	21×21×64	17×17×64	5×5×64
PW Conv2	17×17×64	17×17×64	1×1×64
Max-pooling2	17×17×64	8×8×64	
DW Conv3	8×8×64	4×4×64	5×5×64
PW Conv3	4×4×64	4×4×64	1×1×64

Max-pooling3	4×4×64	2×2×64	
Global average pooling	2×2×64	1×1×64	
Fully connected	1×1×64	1×1×4	

Furthermore, according to Equations 2.1, 2.2, 2.3, 2.4, and 2.5, the parameters of each convolution operation are related to the number of channels in each layer. The batch normalization operation includes four parameters. The parameter counts for the three models using the Type-2 classification method are detailed in Tables 2.7, 2.8, and 2.9, respectively.

$$Conv\ parameters = in_channel \times kernel\ size \times out_channel \quad (2.1)$$

$$DW\ conv\ parameters = in_channel \times kernel\ size \times 1 \quad (2.2)$$

$$PW\ conv\ parameters = in_channel \times kernel\ size \times out_channel \quad (2.3)$$

$$BN\ parameters = 4 \times out_channel \quad (2.4)$$

$$FC\ parameters = feature\ map\ size \times out_channel \quad (2.5)$$

Table 2.7 The parameter count of doorNet in Type-2 classification.

Operation	Number of parameters	Sum of parameters
Convolution 1	1×3×3×32	288
Batch Normalization 1	4×32	128
DW Convolution 2	32×5×5×1	800
PW Convolution 2	32×1×1×64	2,048
Batch Normalization 2	4×64	256
DW Convolution 3	64×5×5×1	1,600
PW Convolution3	64×1×1×64	4,096
Batch Normalization 3	4×64	256

DW Convolution 4	$64 \times 5 \times 5 \times 1$	1,600
PW Convolution 4	$64 \times 1 \times 1 \times 128$	8,192
Batch Normalization 4	4×128	512
Fully connected	128×3	384
Total parameters		20,160

Table 2.8 The parameter count of ImproveNet0 in Type-2 classification.

Operation	Number of parameters	Sum of parameters
Convolution 1	$1 \times 3 \times 3 \times 32$	288
Batch Normalization 1	4×32	128
DW Convolution 2	$32 \times 5 \times 5 \times 1$	800
PW Convolution 2	$32 \times 1 \times 1 \times 64$	2,048
Batch Normalization 2	4×64	256
DW Convolution 3	$64 \times 5 \times 5 \times 1$	1,600
PW Convolution3	$64 \times 1 \times 1 \times 64$	4,096
Batch Normalization 3	4×64	256
Fully connected	64×2	128
Total parameters		9,600

Table 2.9 The parameter count of ImproveNet1 in Type-2 classification.

Operation	Number of parameters	Sum of parameters
Convolution 1	$1 \times 3 \times 3 \times 32$	288
Batch Normalization 1	4×32	128
DW Convolution 2	$32 \times 5 \times 5 \times 1$	800
PW Convolution 2	$32 \times 1 \times 1 \times 64$	2,048
Batch Normalization 2	4×64	256

DW Convolution 3	$64 \times 5 \times 5 \times 1$	1,600
PW Convolution3	$64 \times 1 \times 1 \times 64$	4,096
Batch Normalization 3	4×64	256
DW Convolution 4	$64 \times 5 \times 5 \times 1$	1,600
PW Convolution 4	$64 \times 1 \times 1 \times 64$	4,096
Batch Normalization 4	4×64	256
Fully connected	64×4	256
Total parameters		15,680

The confusion matrix following the implementation of the Type-2 classification method, as illustrated in Figure 2.6, reveals that categories which were previously prone to misclassification in the original framework have become more distinguishable. Notably, the categories disgust and angry, which were highly confusable in the Type-1 framework, show significant improvement when classified using the Type-2 method. The individual and overall accuracy, along with the parameter quantities, are detailed in Table 2.10.

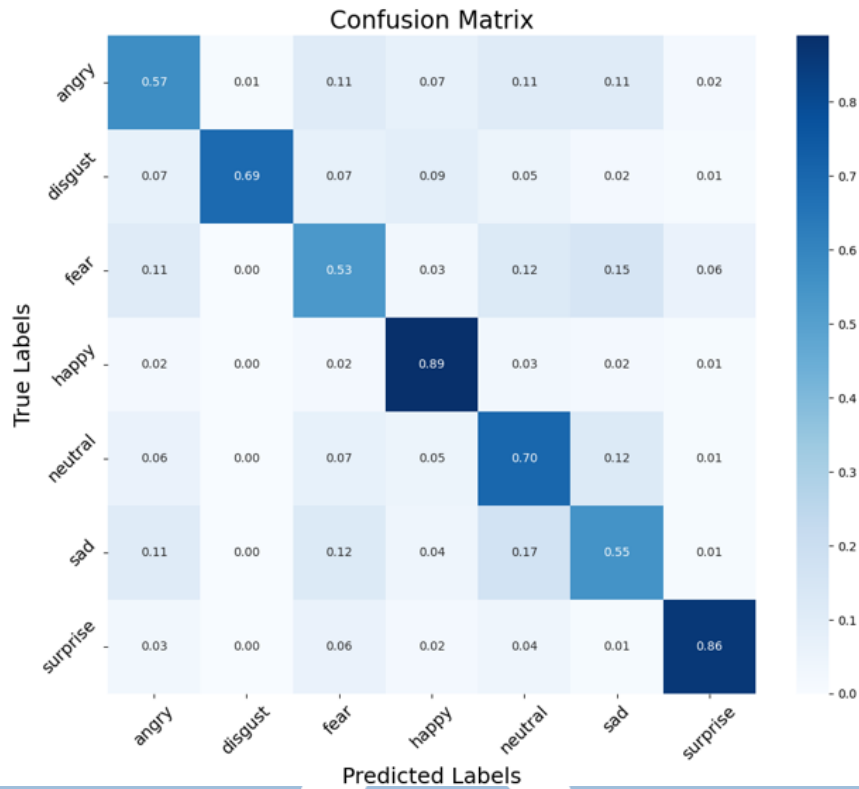


Figure 2.6 Confusion matrix for Type-2 classification.

Table 2.10 The accuracy and parameters of the Type-2 classification.

Model name	Accuracy	Sum of parameters
doorNet	91.36%	20,160
ImproveNet0	99.19%	9,600
ImproveNet1	63.00%	15,680
Overall accuracy: 69.61 %		
Total parameters: 45,440		

In view of the Type-2 classification method, which reduces the likelihood of confusion by categorizing easily confusable expressions into different groups, the Type-3 classification method has been proposed. Referring to Figure 2.4, it can be observed that neutral and sad are easily confusable, as are fear and sad. Therefore, the Type-3 classification framework, as illustrated in Figure 2.7, has been proposed. In the first

layer, the classification method is the same as in Type-1; however, in the second layer, the expressions are divided into three categories: one category for disgust, and the remaining four categories are divided into different groups based on their confusability, as shown in ImproveNet1 and ImproveNet2 in the figure. Finally, the ultimate four categories are distinguished through these two models.

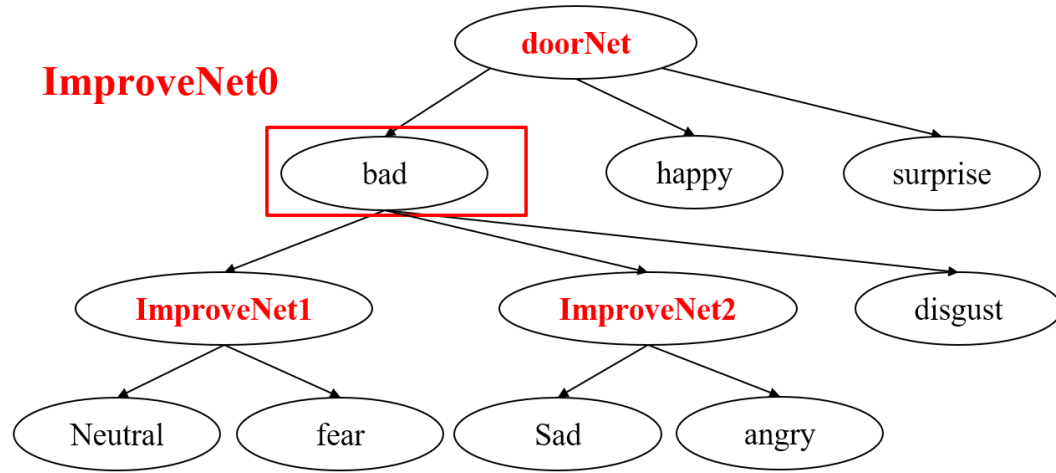


Figure 2.7 Type-3 classification from start to finish.

In the Type-3 classification method, the four models are doorNet, ImproveNet0, ImproveNet1, and ImproveNet2. The structures of doorNet and ImproveNet0 are shown in Tables 2.11 and 2.12, respectively, while ImproveNet1 and ImproveNet2 adopt the same architecture, as shown in Table 2.13. Additionally, the parameter counts for these four models are listed in Tables 2.14 through 2.16, with the parameter counts for doorNet and ImproveNet0 presented in Tables 2.14 and 2.15, and those for ImproveNet1 and ImproveNet2 in Table 2.16.

Table 2.11 The doorNet model architecture in Type-3 classification.

Operation	Data size of input (width × height × in_channel)	Data size of output (width × height × in_channel)	Kernel size
Convolution 1	48×48×48	46×46×32	3×3×32
DW Conv1	46×46×32	42×42×32	5×5×32
PW Conv1	42×42×32	42×42×64	1×1×64
Max-pooling1	42×42×64	21×21×64	
DW Conv2	21×21×64	17×17×64	5×5×64
PW Conv2	17×17×64	17×17×64	1×1×64
Max-pooling2	17×17×64	8×8×64	
DW Conv3	8×8×64	4×4×64	5×5×64
PW Conv3	4×4×64	4×4×128	1×1×128
Max-pooling3	4×4×128	2×2×128	
Global average pooling	2×2×128	1×1×128	
Fully connected	1×1×128	1×1×3	

Table 2.12 The ImproveNet0 model architecture in Type-3 classification.

Operation	Data size of input (width × height × in_channel)	Data size of output (width × height × in_channel)	Kernel size
Convolution 1	48×48×1	46×46×32	3×3×32
DW Conv1	46×46×32	42×42×32	5×5×32
PW Conv1	42×42×32	42×42×64	1×1×64
Max-pooling1	42×42×64	21×21×64	
DW Conv2	21×21×64	17×17×64	5×5×64
PW Conv2	17×17×64	17×17×64	1×1×64

Max-pooling2	17×17×64	8×8×64	
Global average pooling	8×8×64	1×1×64	
Fully connected	1×1×64	1×1×3	

Table 2.13 The model architecture of ImproveNet1 and ImproveNet2 in Type-3 classification.

Operation	Data size of input (width × height × in_channel)	Data size of output (width × height × in_channel)	Kernel size
Convolution 1	48×48×1	46×46×32	3×3×32
DW Conv1	46×46×32	42×42×32	5×5×32
PW Conv1	42×42×32	42×42×64	1×1×64
Max-pooling1	42×42×64	21×21×64	
DW Conv2	21×21×64	17×17×64	5×5×64
PW Conv2	17×17×64	17×17×64	1×1×64
Max-pooling2	17×17×64	8×8×64	
Global average pooling	8×8×64	1×1×64	
Fully connected	1×1×64	1×1×2	

Table 2.14 The parameter count of doorNet in Type-3 classification.

Operation	Number of parameters	Sum of parameters
Convolution 1	1×3×3×32	288
Batch Normalization 1	4×32	128
DW Convolution 2	32×5×5×1	800
PW Convolution 2	32×1×1×64	2,048
Batch Normalization 2	4×64	256

DW Convolution 3	$64 \times 5 \times 5 \times 1$	1,600
PW Convolution3	$64 \times 1 \times 1 \times 64$	4,096
Batch Normalization 3	4×64	256
DW Convolution 4	$64 \times 5 \times 5 \times 1$	1,600
PW Convolution 4	$64 \times 1 \times 1 \times 128$	8,192
Batch Normalization 4	4×128	512
Fully connected	128×3	384
Total parameters		20,160

Table 2.15 The parameter count of ImproveNet0 in Type-3 classification.

Operation	Number of parameters	Sum of parameters
Convolution 1	$1 \times 3 \times 3 \times 32$	288
Batch Normalization 1	4×32	128
DW Convolution 2	$32 \times 5 \times 5 \times 1$	800
PW Convolution 2	$32 \times 1 \times 1 \times 64$	2,048
Batch Normalization 2	4×64	256
DW Convolution 3	$64 \times 5 \times 5 \times 1$	1,600
PW Convolution3	$64 \times 1 \times 1 \times 64$	4,096
Batch Normalization 3	4×64	256
Fully connected	64×3	192
Total parameters		9,664

Table 2.16 The parameter count of ImproveNet1 and ImproveNet2 in Type-3 classification.

Operation	Number of parameters	Sum of parameters
Convolution 1	$1 \times 3 \times 3 \times 32$	288
Batch Normalization 1	4×32	128
DW Convolution 2	$32 \times 5 \times 5 \times 1$	800
PW Convolution 2	$32 \times 1 \times 1 \times 64$	2,048
Batch Normalization 2	4×64	256
DW Convolution 3	$64 \times 5 \times 5 \times 1$	1,600
PW Convolution3	$64 \times 1 \times 1 \times 64$	4,096
Batch Normalization 3	4×64	256
Fully connected	64×2	128
Total parameters		9,600

After employing the Type-3 classification method, as depicted in Figure 2.8, it was observed that additional modules were introduced to reduce confusion particularly between 'neutral' and 'sad', as well as 'fear' and 'sad', compared to the Type-2 classification approach. However, the results from Type-3 were similar to those from the Type-2 method. Finally, Table 2.17 illustrates the individual and overall accuracy and parameter quantities of the four models.

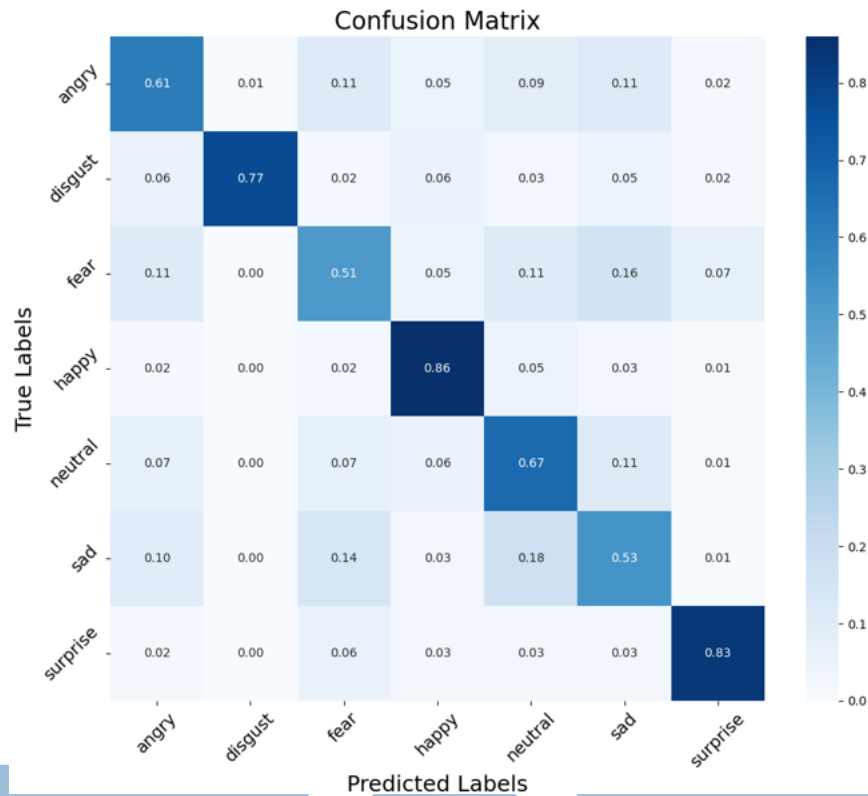


Figure 2.8 Confusion matrix for Type-3 classification.

Table 2.17 The accuracy and parameters of the Type-3 classification.

Model name	Accuracy	Sum of parameters
doorNet	91.36%	20,160
ImproveNet0	70.39%	9,664
ImproveNet1	79.44%	9,600
ImproveNet2	76.01%	9,600
Overall accuracy: 68.28 %		
Total parameters: 49,024		

Based on the previously discussed Type-2 and Type-3 classification methods, both show an improvement in accuracy compared to the Type-1 framework. However, the Type-3 method, which employs more parameters than the Type-2 method, exhibits a slight decrease in accuracy. The comparison of the Type-1, Type-2, and Type-3 methods

is illustrated in Table 2.18. Therefore, the Type-2 classification method has been chosen as the framework for this thesis.

Table 2.18 Comparison of Type-1, Type-2, and Type-3

Method	Accuracy	Parameters
Type-1 framework	64.43%	59,399
Type-2 classification	69.61%	45,440
Type-3 classification	68.28%	49,024

The overall computational cost (FLOPs) for both the Type-1 and the final selected method (Type-2) is summarized in Table 2.19. It can be observed that the overall computational cost decreases when using the Type-2 classification method compared to Type-1. Furthermore, Table 2.20 provides a breakdown of the computational cost for each model within the Type-2 framework, showing that if the recognition can occur early, the computational load of the Type-2 architecture can be reduced by approximately 65.9% compared to the one-time recognition structure of Type-1. Additionally, within the Type-2 framework, if recognition can be achieved in the first layer, the computational cost can be reduced by about 55.45% compared to recognizing at the final layer. These results indicate that using Type-1 for recognizing happy and surprise emotions would waste considerable unnecessary resources, whereas the Type-2 classification framework allows for more targeted recognition. Therefore, the decision was made to adopt the Type-2 method for subsequent quantization and hardware implementation.

Table 2.19 Comparison of Type-1 architecture with Type-2 classification in terms of FLOPs.

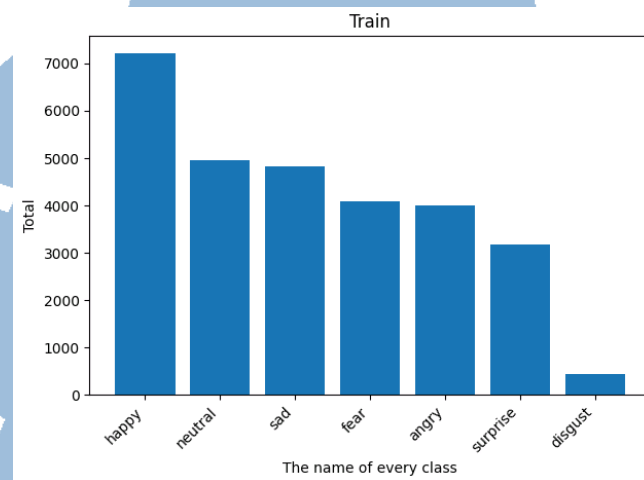
Method	FLOPs
Type-1 framework	0.115M
Type-2 classification	0.088M

Table 2.20 Comparison of FLOPs for each model in Type-2 classification.

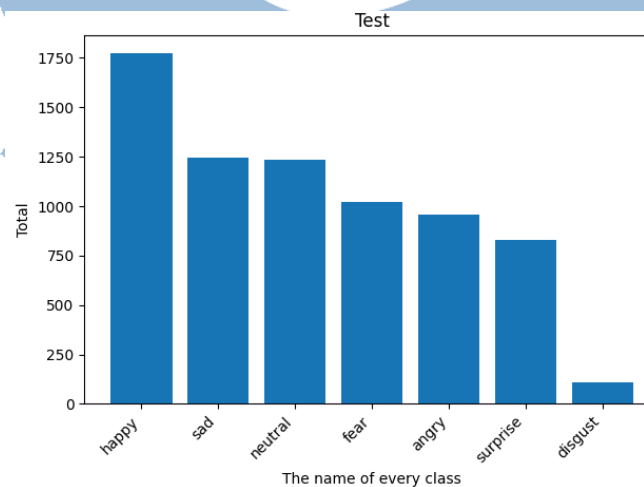
Model name	FLOPs
doorNet	0.0392M
ImproveNet0	0.0186M
ImproveNet1	0.0305M
Overall FLOPs:0.088M	

2.3 The impact of resolving class imbalance and analysis

The FER2013 dataset utilized in this study exhibits a significant data imbalance. As illustrated in the statistical chart in Figure 2.9, the "disgust" category has particularly few samples in both the training and testing phases. Conversely, the "happy" category is the most prevalent, greatly exceeding the second most common "neutral" category. This distribution reveals a significant imbalance among the categories within the dataset, necessitating rectification during the training process.



(a) Training data



(b) Test data

Figure 2.9 A statistical chart of the quantity within the FER2013 dataset.

In the classification method employed in this thesis, there exists a significant imbalance during the training phase. Figure 2.10 illustrates the quantities of each category in the dataset used to train the doorNet model, showing a considerable disparity, particularly for the data sent to the next model, ImproveNet0, compared to the other two categories. Figure 2.11 further demonstrates an even greater gap between categories in the dataset used to train ImproveNet0. To address the imbalance issue in the training dataset, the Label-Distribution-Aware Margin Loss (LDAM) method introduced in Section 1.4 was utilized, along with a combination of delayed reweighting (DRW) and a straightforward comparison with delayed reweighting (DRW) alone.

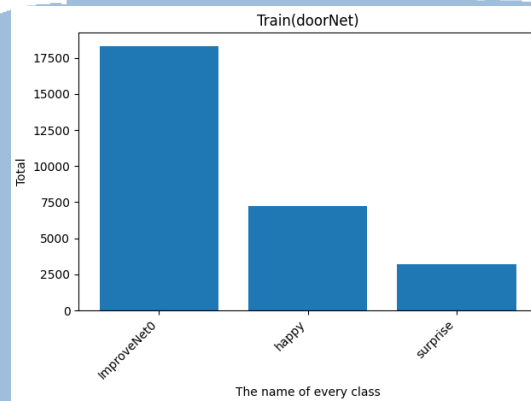


Figure 2.10 A statistical analysis of the number of categories in the doorNet training dataset.

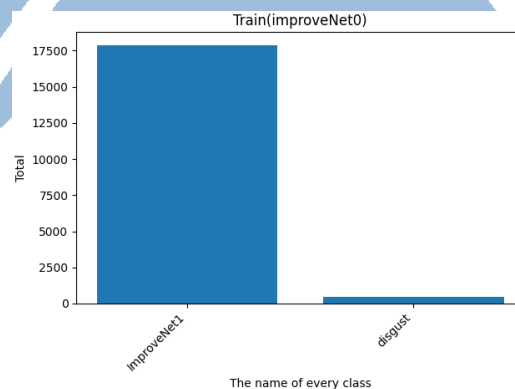


Figure 2.11 The dataset of ImproveNet0 comprises a detailed statistical analysis of the number of categories.

In the DRW flowchart shown in Figure 2.12, when the epoch is less than 10, the class weights remain unchanged, and no adjustments are made to the learning rate. However, after the epoch exceeds 10, each class is reweighted, and the learning rate is multiplied by 0.005. The entire training process continues until the epoch reaches 300. The method for reweighting each class follows the same approach as discussed in Section 1.4, with the β value set at 0.9999, which has been identified as the optimal value based on experimental results in that section. The corresponding weights are then calculated based on the number of samples in each class during training, and cross-entropy is used as the loss function for training.

Additionally, LDAM adjusts the margin for each class by calculating the square root of the number of samples for that class and taking its inverse proportion, thereby assigning a larger margin to classes with fewer samples. Specifically, for each sample, the corresponding margin adjustment is subtracted from the predicted results, generating new predictions. These new predictions are then compared with the actual values through softmax cross-entropy to calculate the loss, thus adjusting the model parameters during the training process. The LDAM-DRW method combines the aforementioned LDAM calculations with the DRW operations.

In Table 2.21, it can be observed that after employing the LDAM in combination with delayed reweighting (DRW) (LDAM-DRW), the recognition rates across various categories improved, thereby enhancing the overall accuracy of the doorNet model. Furthermore, simply applying DRW led to a further increase in the recognition rates of individual categories, which consequently improved the final accuracy of the doorNet model.

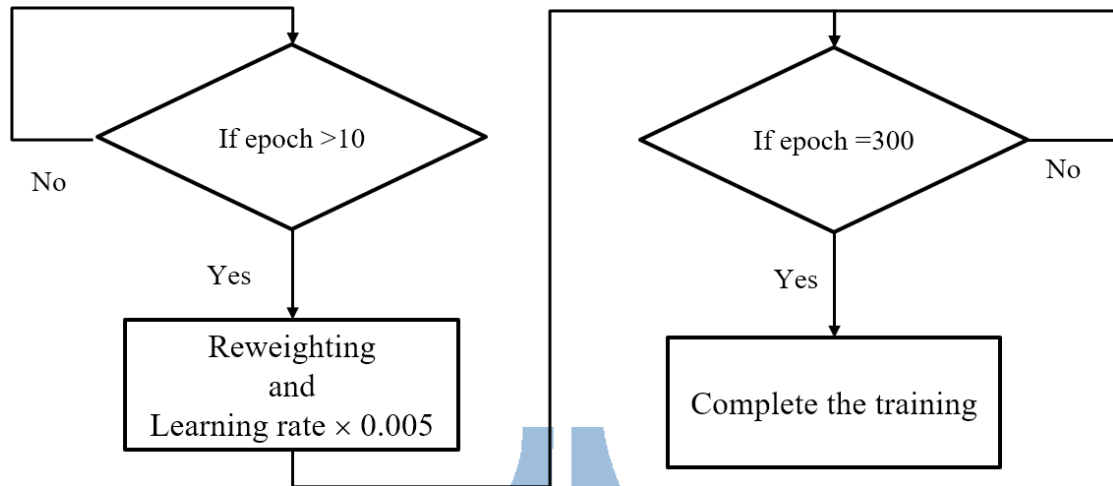


Figure 2.12 The flowchart for the DRW.

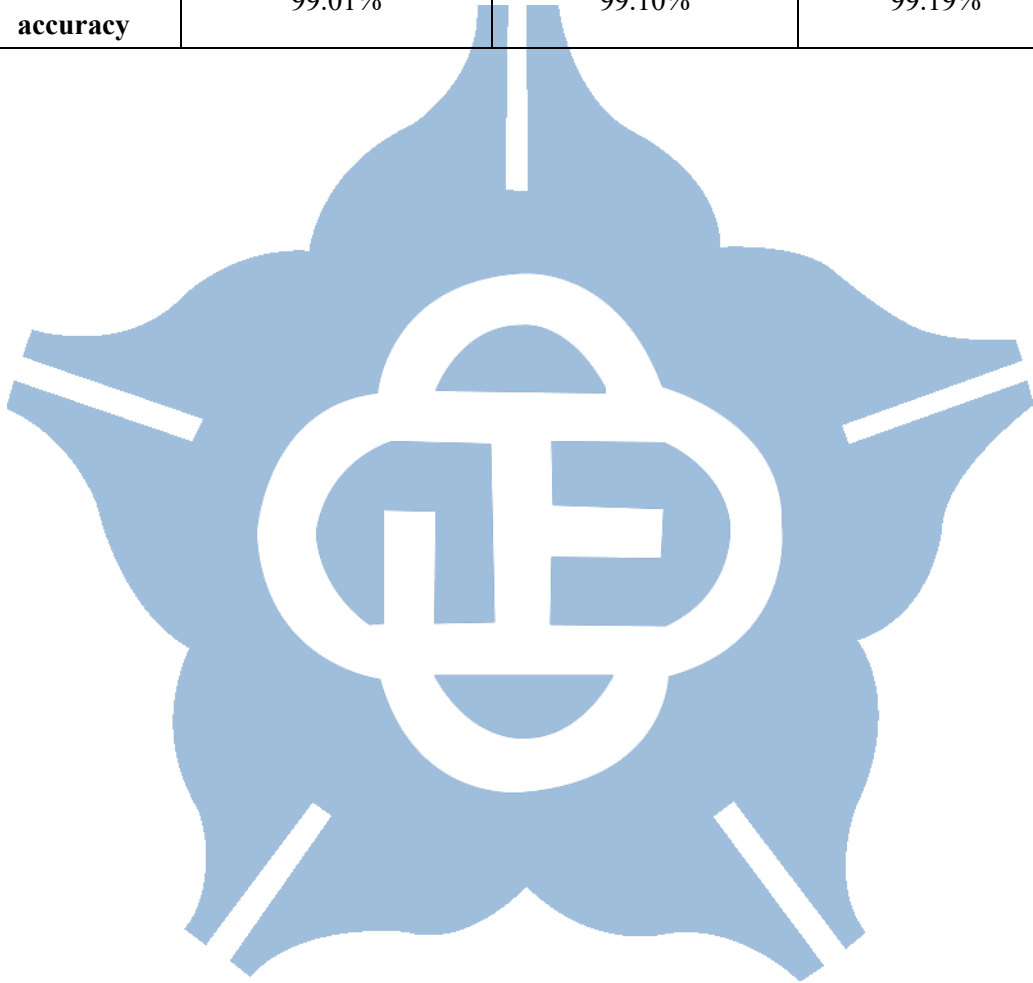
Table 2.21 Handling data imbalance using different methods resulted in varying accuracy for doorNet.

Class name	Original accuracy	The accuracy of LDAM-DRW	The accuracy of DRW
happy	86%	87%	88%
ImproveNet0	93%	93%	94%
surprise	87%	86%	86%
Overall accuracy	91.06%	91.12%	91.36%

In Table 2.22, it is observed that employing Label-Distribution-Aware Margin Loss combined with delayed reweighting (LDAM-DRW) leads to improvements in the accuracy of each category for ImproveNet0. Consequently, the overall recognition accuracy of the model also increases correspondingly. Moreover, solely utilizing delayed reweighting (DRW) further enhances the recognition capability of each category, thereby significantly boosting the final recognition accuracy of ImproveNet0.

Table 2.22 Employ different methods to address data imbalance and enhance the accuracy of ImproveNet0.

Class name	Original accuracy	The accuracy of LDAM-DRW	The accuracy of DRW
disgust	79%	80%	82%
ImproveNet1	99%	99%	100%
Overall accuracy	99.01%	99.10%	99.19%



2.4 Weight Quantization Method

Neural network inference requires trained weights for computation. Storing these weights in floating-point format consumes substantial memory. To deploy neural networks on edge devices, weights must be quantized from 32-bit floating-point precision to lower bit precision. Accordingly, this thesis adopts the IR-Net method as introduced in Section 1.5. Incorporating the Libra-PB (Libra Parameter Binarization) function during forward propagation, as shown in Equation 1.10, minimizes information loss through computational techniques. An Error Decay Estimator (EDE) is then added during backpropagation to reduce discrepancies between the forward binarization function and the backward approximation function, ensuring proper parameter updates. Experimental results applying this method to various models are shown in Table 2.23. The results indicate a significant decrease in model accuracy compared to pre-quantization, demonstrating that quantization somewhat reduces overall neural network accuracy.

Table 2.23 Compare the differences between the results after applying IR-NET and the original results.

Module name	Accuracy of Float32	Accuracy after using IR-NET
doorNet	91.36%	73.57%
ImproveNet0	99.19%	97.59%
ImproveNet1	63.00%	42.83%
The original overall accuracy: 69.61% Overall accuracy after using IR-NET: 40.97%		

Due to the poor accuracy obtained in the previous attempts to use IR-NET, this thesis adopts DoReFa-Net for weight quantization, as described in Section 1.5. DoReFa-Net utilizes the tanh function from Equation 1.13 to limit the weight values to

the range between -1 and 1, thereby quantizing the weights into fixed-point numbers within this range. For hardware implementation considerations, all three models of the hierarchical neural network are quantized to the same bit width. After weight quantization, the experimental results shown in Table 2.24 indicate that the accuracy using 10-bit and 9-bit quantization is comparable to that of the original Float32 format, but a further reduction in bit width significantly decreases accuracy. Therefore, it is ultimately decided to use DoReFa-Net to quantize the weights to 9 bits.

Table 2.24 After quantization with DoReFa-Net, the accuracy varies with different bit numbers.

Bits	Overall Accuracy
Float32	69.61%
11	69.48%
10	69.21%
9	69.57%
8	69.08%
7	68.58%

The final choice for this thesis was to employ DoReFa-Net for quantization. After applying the DoReFa-Net method, and retraining the proposed model architecture, the weights were reduced from 32-bit floating point numbers to 9-bit fixed point numbers. Quantizing the model parameters allowed for a reduction in hardware costs required by the proposed model architecture, enabling implementation on hardware with limited resources.

2.5 Quantization Methods for Activation Function

Maintaining model accuracy during weight quantization is relatively straightforward; hence, most previous research has focused on weight quantization. However, feature maps actually require more memory storage space than model weights. Therefore, this thesis uses DoReFa-Net, as described in Section 1.5, for activation quantization. Before applying DoReFa-Net for activation quantization, a bounded activation function must be used to scale the input activations to a specific range to ensure the model converges to the desired accuracy.

By performing a statistical analysis of the activation function outputs of the three models, an appropriate scaling factor was determined. Specific experiments were conducted using Equations 2.6, 2.7, and 2.8. In these equations, "a" represents the original output after the activation function. Next, by applying the scaling factor, all values greater than 1 were normalized to 1, thereby adjusting the activation input to fit within a specific range. Table 2.25 presents the original accuracy results before scaling, as well as the accuracy obtained after retraining the models using these equations to scale the activation functions. The study found that applying a scaling factor to the activation functions and retraining the model after weight quantization significantly improves accuracy compared to the unscaled model. Furthermore, Equation 2.6 demonstrated the best performance as the scaling factor for the activation function, and thus was selected as the final scaling factor.

$$h(a) = \text{clip}(a \times 0.015625, 0, 1) \quad (2.6)$$

$$h(a) = \text{clip}(a \times 0.00390625, 0, 1) \quad (2.7)$$

$$h(a) = \text{clip}(a \times 0.00048828125, 0, 1) \quad (2.8)$$

Table 2.25 The accuracy of using different scaling factors.

The scaling factor	Bit-width of weight	Accuracy
1(original)	9	69.57%
0.015625	9	72.65%
0.00390625	9	54.25%
0.00048828125	9	17.18%

However, the original DoReFa-Net mapped input activation values to fixed-point numbers using Equation 1.17, which required additional multipliers in hardware implementation. To avoid this requirement, Equation 2.9 was adopted as the activation quantization function, which can be computed directly on hardware using shifts. Using Equation 2.9 for activation quantization increases the number of activation bits to $k + 1$ bits. Table 2.26 shows the accuracy when applying Equation 2.9 with different activation bit widths. It can be observed that the performance is optimal with an activation bit width of $10 + 1$ bits, as the decrease in accuracy compared to higher bit numbers is not significant; further reducing the number of bits sharply decreases accuracy.

$$f_a^k(r) = \frac{1}{2^k} \text{round}((2^k)r) \quad (2.9)$$

Table 2.26 The accuracy associated with varying activation bit widths.

Bit-width of weight	Bit-width of activation ($k + 1$ bits)	Accuracy
9	Float32	72.65%
9	11 + 1	72.28%
9	10 + 1	72.07%
9	9 + 1	47.95%

Table 2.27 lists the hyperparameters and loss function used during the training process of the proposed hierarchical model.

Table 2.27 Hyperparameters setting and loss function used in hierarchical models.

Hyperparameters	Value
Train epochs	300
Retrain epochs of weight quantization	300
Retrain epochs of activation quantization	300
Batch size	120
Learning rate	0.001
Approaches to handling class imbalance	DRW
Optimization algorithm	Adam
Loss function	Cross entropy

Chapter 3 Hardware Implementation

3.1 Fixed-Point weights and Batch Normalization Layer

After completing the model training in the previous software phase, considering subsequent hardware implementation, the DoReFaNet was used for weight quantization in Section 2.4. To store these quantized weights, they needed to be stored in a lookup table using fixed-point representation. After quantizing weights using DoReFaNet, the integer part needs to be represented with 2 bits because the weights are constrained within the range of -1 to 1. The next step was to determine the number of decimal places to retain when storing these weights in the lookup table. To minimize memory usage, the decision was made to minimize the number of decimal places while minimizing the decrease in precision. According to Table 3.1, overall precision significantly decreases when the number of decimal places is less than 7. Therefore, 7 was ultimately chosen as the number of decimal places for the weight fraction part.

Table 3.1 Comparison of accuracy of weight across different bit levels.

Comparison table of weight across different bit levels			
Integer bits	Decimal bits	Total bits	Accuracy
FP32			72.07%
2	8	10	72.06%
2	7	9	72.04%
2	6	8	70.69%

Furthermore, to further reduce memory usage, the parameters of Batch Normalization (BN) were quantized. Prior to quantization, each BN layer's four

parameters were simplified to two, as derived from Equations 3.1 and 3.2. These equations stem from the original BN formula (see Equations 1.3 to 1.6), where μ_B represents the batch mean and α_B^2 represents the batch variance. Thus, the entire BN operation can be computed for the layer using these two parameters from Equation 3.1, effectively reducing the required parameter storage from four to two. In the proposed model structure, the parameters required for batch normalization (BN) have been reduced from 2,688 to 1,344.

$$\gamma' = \frac{1}{\sqrt{\alpha_B^2}} \gamma \quad (3.1)$$

$$\beta' = -(\mu_B \gamma') + \beta \quad (3.2)$$

$$y_i = \gamma' x_i + \beta' \quad (3.3)$$

After reducing the parameters of BN from 4 to 2, the next step is to convert BN parameters from floating-point to fixed-point. In this thesis, the first step involves processing γ' in fixed-point. All integer bit widths are kept at 9 bits, while the number of decimal bits is determined based on test results, as detailed in Table 3.2. The research found that when γ' has fewer than 6 decimal bits, its accuracy significantly decreases. However, with 6 decimal bits, the accuracy can be maintained at 71.84%. Therefore, in this thesis, γ' is configured with 9 integer bits and 6 decimal bits, totaling 15 bits.

Table 3.2 Comparison of accuracy of γ' across different bit levels.

Comparison table of γ' across different bit levels			
Integer bits	Decimal bits	Total bits	Accuracy
FP32			72.04%
9	7	16	71.94%
9	6	15	71.84%

9	5	14	71.36%
9	4	13	71.26%
9	3	12	71.04%

Batch Normalization (BN) involves two parameters. After converting γ' from floating-point to fixed-point, the next step is to process the second parameter, β' , into fixed-point as well. The integer part of β' retains a width of 6 bits, while the number of bits for the fractional part is determined through multiple tests, as shown in Table 3.3. Research indicates that to ensure an accuracy of 71.6%, the fractional part of β' needs at least 7 bits. Therefore, in this work, β' is set with an integer part of 6 bits and a fractional part of 7 bits, making a total of 13 bits.

Table 3.3 Comparison of accuracy of β' across different bit levels.

Comparison table of β' across different bit levels			
Integer bits	Decimal bits	Total bits	Accuracy
FP32			71.84%
6	10	16	71.83%
6	9	15	71.85%
6	8	14	71.80%
6	7	13	71.76%
6	6	12	71.21%
6	5	11	68.85%

Due to the structure of this work, a substantial number of bits need to be stored after Batch Normalization (BN) calculations, significantly increasing memory usage. Considering hardware implementation, this thesis applies fixed-point representation to the results of BN operations to reduce memory consumption. All integer bit widths are

retained at 8 bits, while the fractional bit width is determined based on test results, as detailed in Table 3.4. It was found that when the fractional bits are fewer than 6, the accuracy drops significantly. However, with 6 fractional bits, the accuracy can be maintained at 71.7%. Therefore, in this thesis, the integer bit width after BN operations is set to 8 bits, and the fractional bit width is set to 6 bits, making a total of 14 bits.

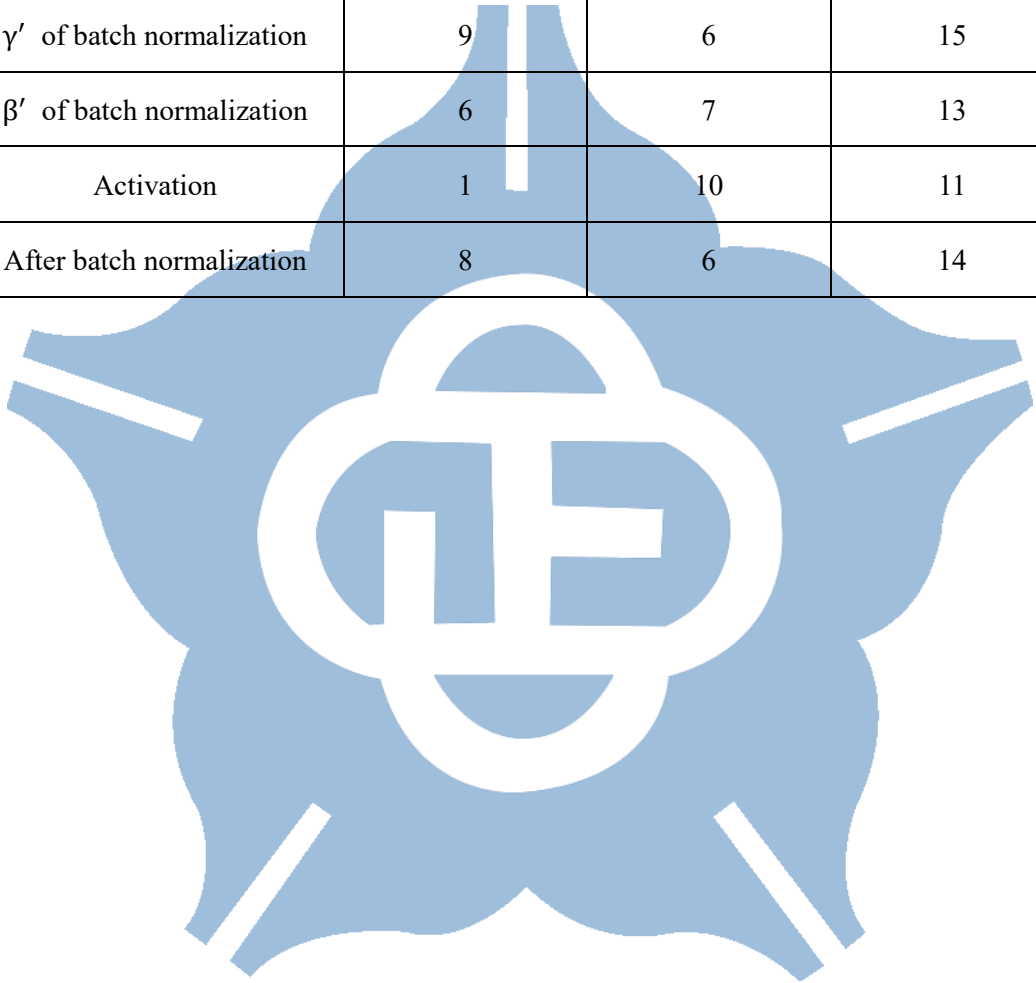
Table 3.4 The accuracy comparison table of fixed-point values after batch normalization.

Comparison table of fixed-point values after batch normalization			
Integer bits	Decimal bits	Total bits	Accuracy
8	19	27	71.76%
8	8	16	71.75%
8	7	15	71.72%
8	6	14	71.70%
8	5	13	69.83%

After the aforementioned procedures, by storing all parameters as fixed-point numbers and utilizing the minimum bit width, hardware resource consumption has been effectively reduced. The overall test results for each operation are shown in Table 3.5. After numerous tests, the proposed Hierarchical Depthwise Separable Convolutional Neural Network (HDSC) has consistently maintained a high level of accuracy.

Table 3.5 The fixed-point decision for hardware implementation.

The fixed-point decision for hardware implementation			
	Integer bits	Decimal bits	Total bits
Input data	8		8
Weight (DoReFa-Net)			9
Weight (lookup table)	2	7	9
γ' of batch normalization	9	6	15
β' of batch normalization	6	7	13
Activation	1	10	11
After batch normalization	8	6	14



3.2 HDSC Hardware Accelerator Architecture

This section provides a detailed exploration of implementing the Hierarchical Depthwise Separable Convolutional Neural Network (HDSC) on hardware. Figure 3.1 illustrates the complete hardware architecture, where the memory units are used for storing weights and feature maps, alongside other modules dedicated to computation. The SRAM files are single-port memories generated by a memory compiler in TSMC 16nm process, used to store input images and feature maps computed by convolutional layers. Psum, synthesized by circuit synthesis tools, consists of a register array that records partial sums generated by each channel in the convolutional layers. For weight storage, such as the weights of convolutional layers, it is stored through ROM. To enhance the performance of fully connected layers and considering their relatively small number of parameters, the weights are stored using a lookup table rather than employing ROM for storage.

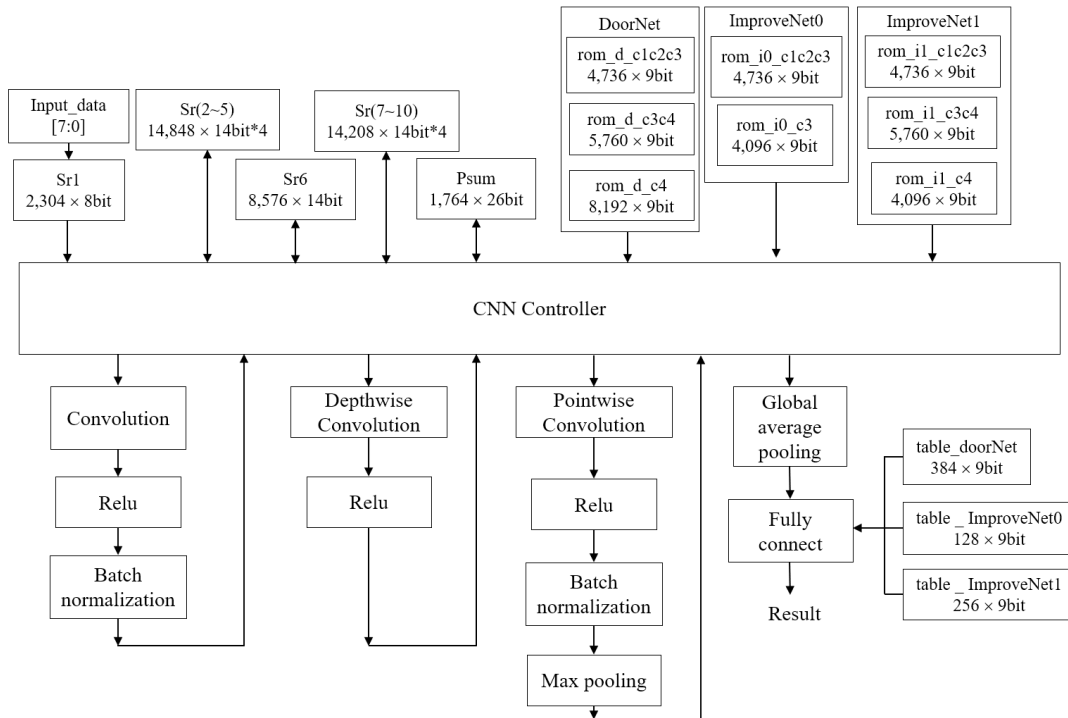


Figure 3.1 The proposed HDSC model hardware architecture.

In the initial phase of execution, predictions are made using doorNet. The computation process includes standard convolution, depthwise convolution, and pointwise convolution, followed by ReLU activation and batch normalization. Max pooling can be optionally performed. The resulting feature maps are written back to SRAM. Besides standard convolution, other modules are reused in each layer's computation. The prediction results of DoorNet are processed through multiple convolutional layers, then compressed via a global average pooling layer, and finally passed through a fully connected layer to generate the final prediction output. Subsequently, a decision is made on whether to proceed with the computations of ImproveNet0 and ImproveNet1. The final prediction result is obtained at the end of the computation.

Table 3.6 illustrates the specific utilization of the various RAM modules and Psum in the DoorNet network architecture. In this architecture, the input image size is 48×48 pixels and is a grayscale image, meaning the image contains only a single channel. Each pixel is 8 bits, requiring one storage block (Sr1) to store the input image, totaling 18 Kb. Sr1 retains its value after reading the image and is not used for subsequent feature map storage. Feature maps are stored in five SRAMs (Sr2-6), totaling 925.7 Kb. The feature maps for the second layer are stored in four SRAMs (Sr7-10), totaling 606.3 Kb. Since the input image for the first layer has a single channel, and the subsequent depthwise operation is performed only on this single channel, only the pointwise part of the following operations aggregates channels. In the third layer, the aggregated channels are stored in one Psum register, with a total size of 44.7 Kb. After the third layer, max-pooling reduces the image size, and further computations are performed on these downscaled feature maps. After the doorNet prediction, ImproveNet0 and ImproveNet1 calculations might be necessary, with memory usage as shown in Tables 3.7 and 3.8, which are similar to doorNet and reuse doorNet's memory. From the

memory usage table for each layer, it can be observed that the maximum memory usage at any given time during operation is 1532 Kb. Additionally, unused SRAM and ROM are shut down to conserve energy, as explained in detail in Section 3.3.

Table 3.6 Memory usage table for each layer of doorNet.

Store data	Data size (height × width × channel × bit-width)	Total size(bits)	Data management
Input data	48×48×1×8	18,432	Sr1
Feature map of layer 1	46×46×32×14	947,968	Sr(2 ~ 6)
Feature map of layer 2	42×42×32×11	620,928	Sr(7 ~ 10)
Psum of layer 3	42×42×1×26	45,864	Psum
Feature map of layer 3	21×21×64×14	395,136	Sr2, Sr3
Feature map of layer 4	17×17×64×11	203,456	Sr4, Sr5
Psum of layer 5	17×17×1×26	7,514	Psum
Feature map of layer 5	8×8×64×14	57,344	Sr2
Feature map of layer 6	4×4×64×11	11,264	Sr6
Psum of layer 7	4×4×1×26	416	Psum
Feature map of layer 7	2×2×128×14	7,168	Sr2

Table 3.7 Memory usage table for each layer of ImproveNet0.

Store data	Data size (height × width × channel × bit-width)	Total size(bits)	Data management
Input data	48×48×1×8	18,432	Sr1
Feature map of layer 1	46×46×32×14	947,968	Sr(2 ~ 6)

Feature map of layer 2	42×42×32×11	620,928	Sr(7 ~ 10)
Psum of layer 3	42×42×1×26	45,864	Psum
Feature map of layer 3	21×21×64×14	395,136	Sr2, Sr3
Feature map of layer 4	17×17×64×11	203,456	Sr4, Sr5
Psum of layer 5	17×17×1×26	7,514	Psum
Feature map of layer 5	8×8×64×14	57,344	Sr2

Table 3.8 Memory usage table for each layer of ImproveNet1.

Store data	Data size (height × width × channel × bit-width)	Total size(bits)	Data management
Input data	48×48×1×8	18,432	Sr1
Feature map of layer 1	46×46×32×14	947,968	Sr(2 ~ 6)
Feature map of layer 2	42×42×32×11	620,928	Sr(7 ~ 10)
Psum of layer 3	42×42×1×26	45,864	Psum
Feature map of layer 3	21×21×64×14	395,136	Sr2, Sr3
Feature map of layer 4	17×17×64×11	203,456	Sr4, Sr5
Psum of layer 5	17×17×1×25	7,514	Psum
Feature map of layer 5	8×8×64×14	57,344	Sr2
Feature map of layer 6	4×4×64×11	11,264	Sr6
Psum of layer 7	4×4×1×26	416	Psum
Feature map of layer 7	2×2×64×14	3,584	Sr2

During the Verilog simulation phase, the architecture's required cycles at each stage are illustrated in Figure 3.2. Once the image is input, the recognition result can be obtained from doorNet after 20,359,111 cycles. Following the results from doorNet, if

the data needs to be passed to the next stage model, ImproveNet0, its recognition result can be achieved after an additional 19,821,434 cycles. If the result from ImproveNet0 requires further processing in ImproveNet1, the final outcome can be obtained after an additional 20,094,401 cycles. The analysis reveals that if the model successfully recognizes the image during the first stage with doorNet, the overall process is nearly three times faster compared to waiting for the final recognition in ImproveNet1, significantly reducing computational resources. This approach is more efficient than conventional network architectures, where results are only available at the final stage, thereby reducing computational overhead.

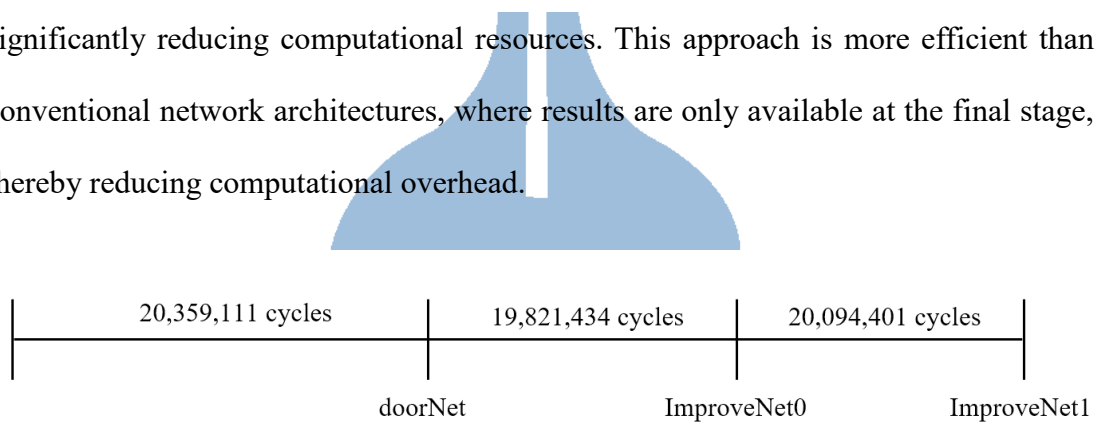


Figure 3.2 Counting cycles at the RTL level across various hierarchical levels.

3.3 Memory Power Gating

In section 3.2, the usage of memory was itemized, and in this section, we design the application of power gating control in the architecture based on the aforementioned plan. By employing power gating to shut down unused RAM and ROM, we reduce power consumption. The memory power is alternately turned on or off according to operational needs, thereby reducing power consumption during idle periods. The proposed power gating control method is illustrated in Figure 3.3. Signals pgen1 to pgen6 are control signals used to turn the static random-access memory (SRAM) on and off. Additionally, signals pgen7 to pgen14 are used for the control of ROM. Detailed information regarding the power gating configuration for doorNet, ImproveNet0, and ImproveNet1 can be found in Tables 3.9, 3.10, and 3.11, respectively.

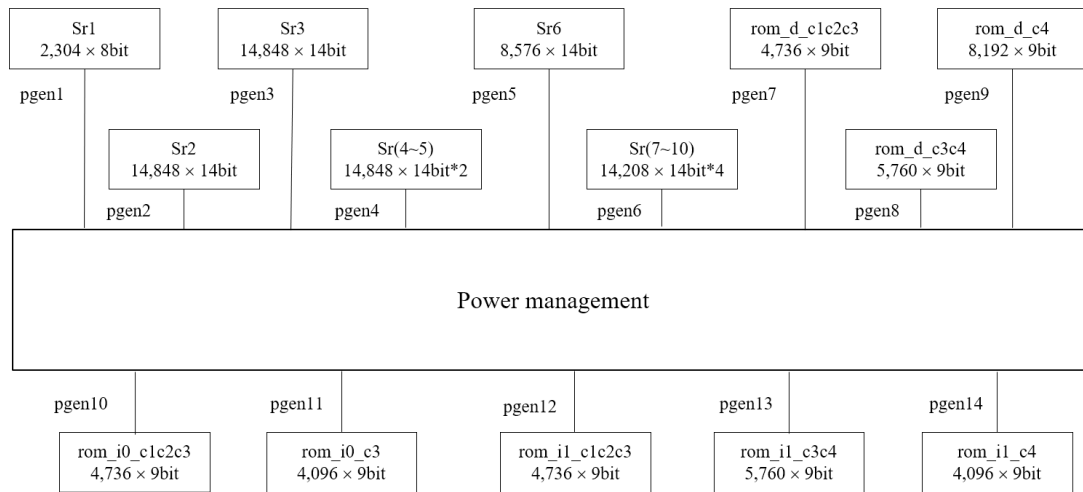


Figure 3.3 The power gating control of HDSC.

Table 3.9 The power state table of doorNet.

Power domain	Layer							
	Conv1	DW1	PW1	DW2	PW2	DW3	PW3	FC
pgen1	On	Off	Off	Off	Off	Off	Off	Off
pgen2	On	On	On	On	On	On	On	On
pgen3	On	On	On	On	Off	Off	Off	Off
pgen4	On	On	Off	On	On	Off	Off	Off
pgen5	On	On	Off	Off	Off	On	On	Off
pgen6	Off	On	On	Off	Off	Off	Off	Off
pgen7	On	On	On	On	Off	Off	Off	Off
pgen8	Off	Off	Off	Off	On	On	Off	Off
pgen9	Off	Off	Off	Off	Off	Off	On	Off
pgen10	Off	Off	Off	Off	Off	Off	Off	Off
pgen11	Off	Off	Off	Off	Off	Off	Off	Off
pgen12	Off	Off	Off	Off	Off	Off	Off	Off
pgen13	Off	Off	Off	Off	Off	Off	Off	Off
pgen14	Off	Off	Off	Off	Off	Off	Off	Off

Table 3.10 The power state table of ImproveNet0.

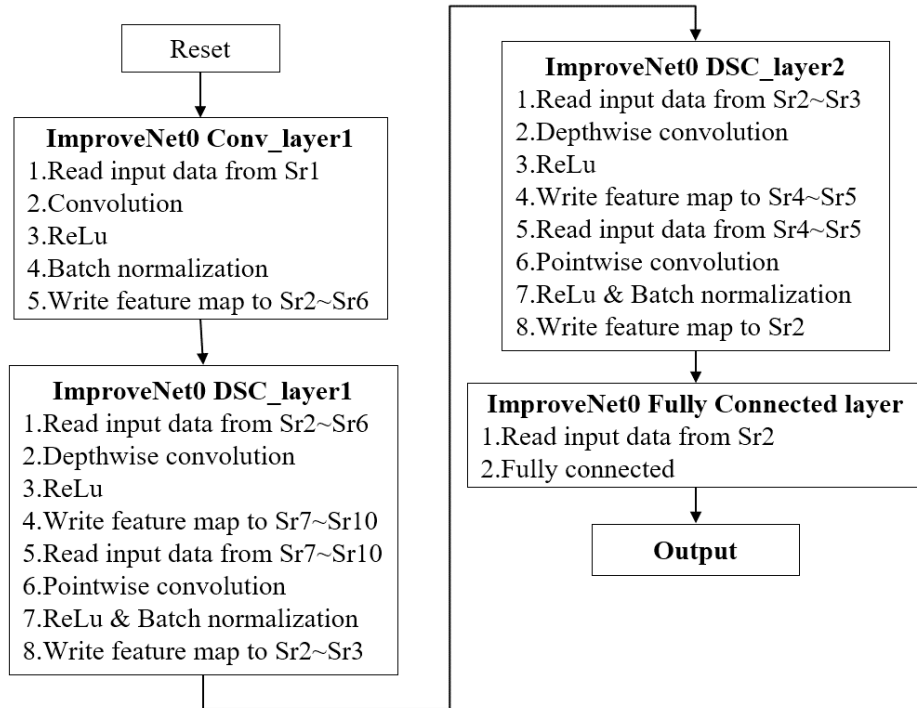
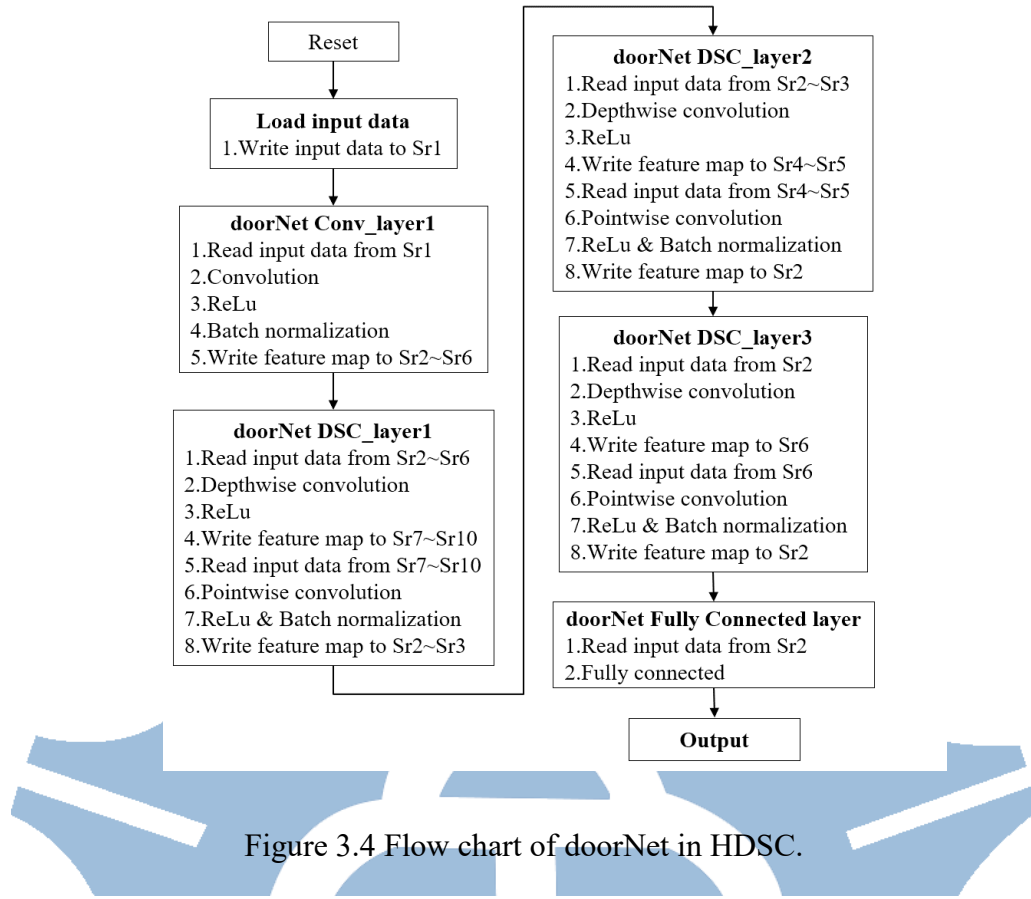
Power domain	Layer					
	Conv1	DW1	PW1	DW2	PW2	FC
pgen1	On	Off	Off	Off	Off	Off
pgen2	On	On	On	On	On	Off
pgen3	On	On	On	On	Off	Off
pgen4	On	On	Off	On	On	Off
pgen5	On	On	Off	Off	Off	Off
pgen6	Off	On	On	Off	Off	Off
pgen7	Off	Off	Off	Off	Off	Off
pgen8	Off	Off	Off	Off	Off	Off
pgen9	Off	Off	Off	Off	Off	Off
pgen10	On	On	On	On	Off	Off
pgen11	Off	Off	Off	Off	On	Off
pgen12	Off	Off	Off	Off	Off	Off
pgen13	Off	Off	Off	Off	Off	Off
pgen14	Off	Off	Off	Off	Off	Off

Table 3.11 The power state table of ImproveNet1.

Power domain	Layer							
	Conv1	DW1	PW1	DW2	PW2	DW3	PW3	FC
pgen1	On	Off	Off	Off	Off	Off	Off	Off
pgen2	On	On	On	On	On	On	On	On
pgen3	On	On	On	On	Off	Off	Off	Off
pgen4	On	On	Off	On	On	Off	Off	Off
pgen5	On	On	Off	Off	Off	On	On	Off
pgen6	Off	On	On	Off	Off	Off	Off	Off
pgen7	Off	Off	Off	Off	Off	Off	Off	Off
pgen8	Off	Off	Off	Off	Off	Off	Off	Off
pgen9	Off	Off	Off	Off	Off	Off	Off	Off
pgen10	Off	Off	Off	Off	Off	Off	Off	Off
pgen11	Off	Off	Off	Off	Off	Off	Off	Off
pgen12	On	On	On	On	Off	Off	Off	Off
pgen13	Off	Off	Off	Off	On	On	Off	Off
pgen14	Off	Off	Off	Off	Off	Off	On	Off

The hardware operation flow of the HDSC architecture is illustrated in Figures 3.4, 3.5, and 3.6. The hardware operation flow of the doorNet component is depicted in Figure 3.4. Initially, the input image is read and written into memory. Subsequently, the image is processed by doorNet, beginning with a standard convolution layer. This is followed by three layers of Depthwise Separable Convolution (DSC), and finally, a fully connected layer, resulting in the classification output of doorNet. Following this, the classification result of doorNet determines whether ImproveNet0 is required. If ImproveNet0 is necessary, its hardware operation flow is shown in Figure 3.5. Initially, the image undergoes a standard convolution operation, followed by two DSC layers. Finally, a fully connected layer produces the classification output of ImproveNet0.

Based on the classification result of ImproveNet0, it is determined whether ImproveNet1 is needed. If so, the hardware operation flow of ImproveNet1 is depicted in Figure 3.6. Initially, the image is processed by a standard convolution layer, followed by three DSC layers, and finally, a fully connected layer, resulting in the classification output of ImproveNet1, which provides the final classification result.



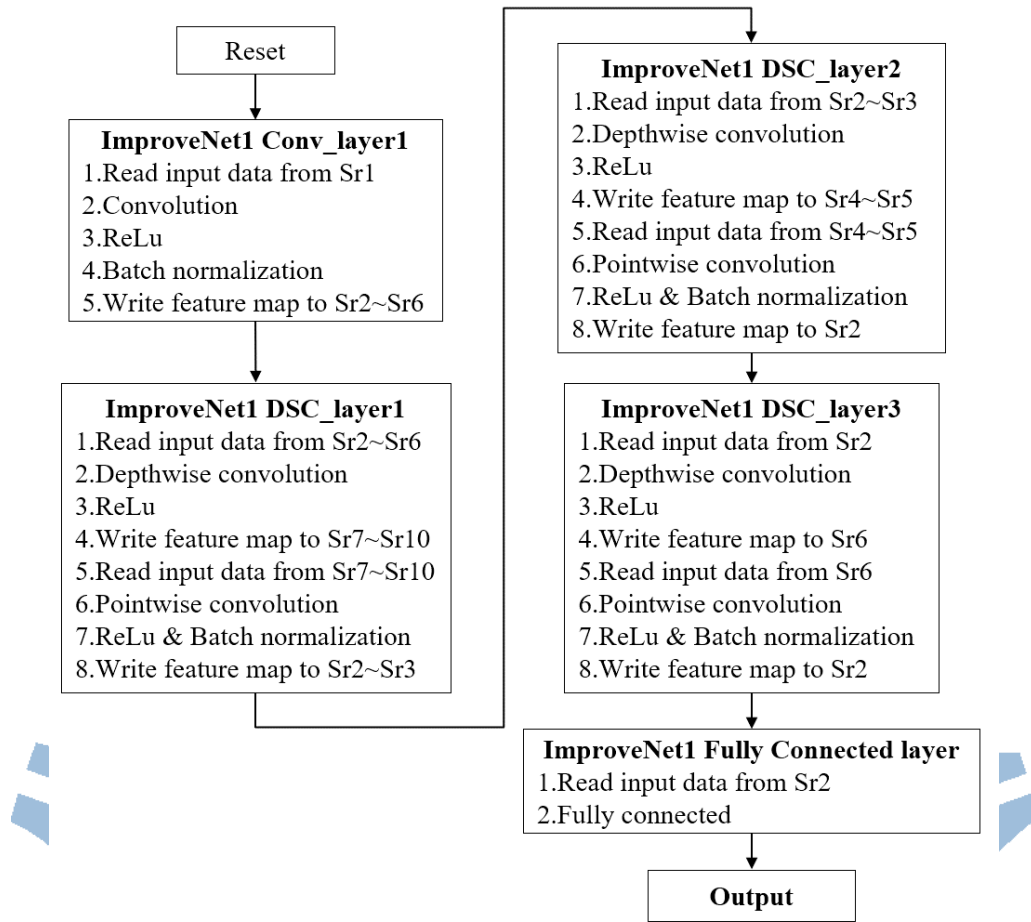


Figure 3.6 Flow chart of ImproveNet1 in HDSC.

3.4 Hardware Implementation Analysis and Comparison

The proposed HDSC hardware design was implemented using TSMC 16nm CMOS process. In the post-layout simulation phase, a working frequency of 555MHz was achieved. Power management measures were applied to this hardware architecture. The overall power analysis is shown in Figures 3.7, 3.8, and 3.9.

In the first phase of the architecture, the doorNet model (as shown in Figure 3.7) exhibited a total power consumption of 37.68 mW when no power management was applied, i.e., when the chip enable pin (cen) was turned off. When power management was applied, meaning the chip enable pin (cen) was turned on, but without power-gating, the power consumption was reduced to 20.06 mW. Further applying power-gating along with the control, the power consumption dropped to 17.37 mW.

In the second phase, following the model's evaluation, power analysis was conducted on ImproveNet0 (as shown in Figure 3.8). Without any power management, the total power consumption was 57.49 mW. With power management but without power-gating, the consumption decreased to 38.46 mW, and with both control and power-gating, it further reduced to 35.5 mW.

In the third phase, after evaluating the model, power analysis was conducted on ImproveNet1 (as shown in Figure 3.9). Without any power management, the total power consumption was 76.69 mW. With power management but without power-gating, the consumption decreased to 56.23 mW, and with both control and power-gating, it further reduced to 53.86 mW.

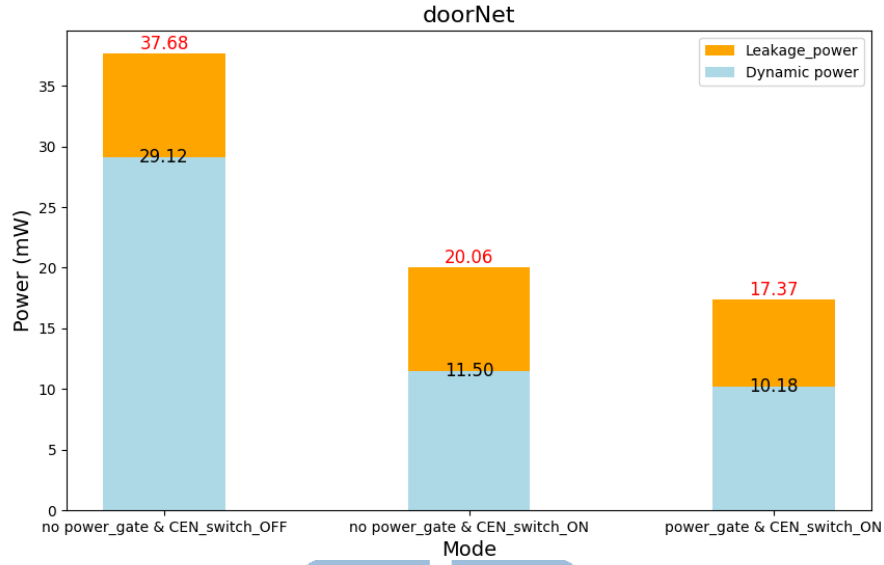


Figure 3.7 Compare the power consumption of the doorNet model in HDSC at 555 MHz with and without power management.

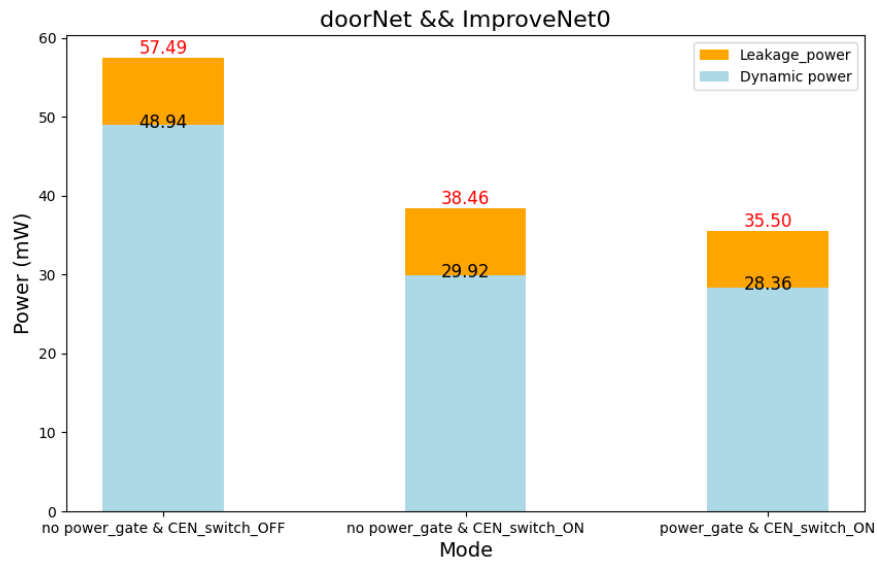


Figure 3.8 Compare the power consumption of the doorNet and ImproveNet0 models in HDSC at 555 MHz with and without power management.

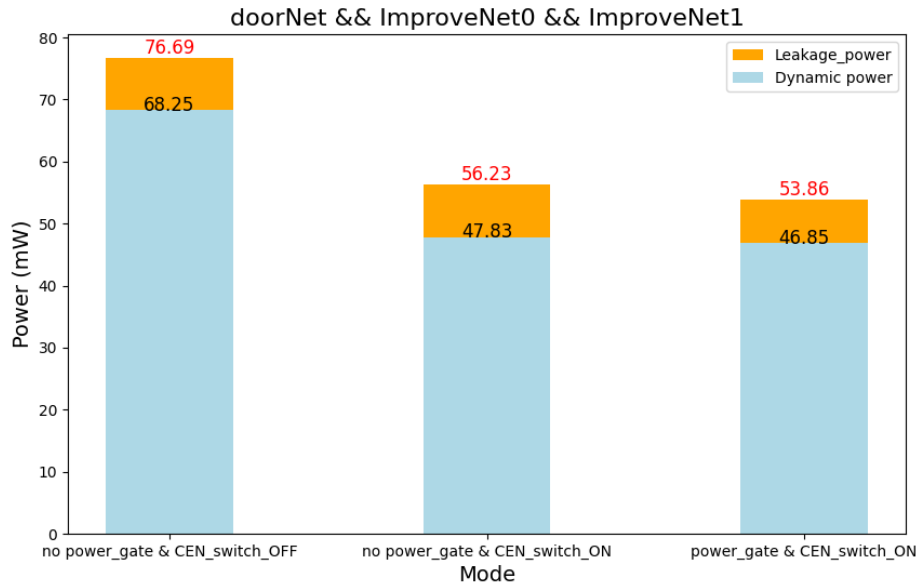


Figure 3.9 Compare the power consumption of the doorNet, ImproveNet0, and ImproveNet1 models in HDSC at 555 MHz with and without power management.

Additionally, Tables 3.12, 3.13, and 3.14 present detailed information on each memory unit under conditions where the entire system needs to run on ImproveNet1, comparing scenarios with and without power control. Table 3.12 shows the case where the chip enable pin (CEN) control is off, with no power-gating applied. Table 3.13 presents the case where the chip enable pin (CEN) control is on, still without power-gating. Finally, Table 3.14 illustrates the scenario where the chip enable pin (CEN) control is on, with power-gating applied.

Table 3.12 When there is no power control, the power consumption of each memory module under the condition of running to ImproveNet1.

Leakage	Internal	Switching	Total	Instance
8.75376e-05	2.23140e-04	4.36120e-11	3.10677e-04	/MNN/rom_d_c1c2c3
9.54192e-05	1.73179e-04	4.85520e-11	2.68598e-04	/MNN/rom_d_c3c4
1.14138e-04	1.65437e-04	1.11079e-10	2.79575e-04	/MNN/rom_d_c4
8.75376e-05	1.49389e-04	0.00000e+00	2.36927e-04	/MNN/rom_i0_c1c2c3
8.26117e-05	4.82000e-06	0.00000e+00	8.74317e-05	/MNN/rom_i0_c3
8.75376e-05	4.82000e-06	0.00000e+00	9.23576e-05	/MNN/rom_i1_c1c2c3
9.54192e-05	4.82000e-06	0.00000e+00	1.00239e-04	/MNN/rom_i1_c3c4
8.26117e-05	4.82000e-06	0.00000e+00	8.74317e-05	/MNN/rom_i1_c4
2.38811e-04	1.10552e-03	4.18724e-09	1.34434e-03	/MNN/sr1
6.42825e-04	2.46548e-03	3.09174e-09	3.10831e-03	/MNN/sr2
6.42825e-04	2.47308e-03	3.36897e-09	3.11591e-03	/MNN/sr3
6.42825e-04	2.48423e-03	1.60012e-08	3.12707e-03	/MNN/sr4
6.42825e-04	2.48211e-03	3.23630e-09	3.12494e-03	/MNN/sr5
4.04615e-04	2.28676e-03	1.60353e-09	2.69137e-03	/MNN/sr6
6.18518e-04	2.45948e-03	4.49432e-09	3.07801e-03	/MNN/sr7
6.18518e-04	2.45659e-03	4.54909e-09	3.07511e-03	/MNN/sr8
6.18518e-04	2.45342e-03	4.81015e-09	3.07195e-03	/MNN/sr9
6.18518e-04	2.45026e-03	4.71469e-09	3.06879e-03	/MNN/sr10
6.42161e-03	2.38474e-02	5.02604e-08	3.02690e-02	Total

Table 3.13 When the CEB switch control is set to "on," the power consumption of each memory unit, under the condition of running to ImproveNet1.

Leakage	Internal	Switching	Total	Instance
8.75376e-05	2.05958e-04	4.62980e-11	2.93496e-04	/MNN/rom_d_c1c2c3
9.54192e-05	2.24973e-05	6.20610e-11	1.17917e-04	/MNN/rom_d_c3c4
1.14138e-04	6.90922e-06	1.38762e-10	1.21047e-04	/MNN/rom_d_c4
8.75376e-05	1.49389e-04	0.00000e+00	2.36927e-04	/MNN/rom_i0_c1c2c3
8.26117e-05	4.82000e-06	0.00000e+00	8.74317e-05	/MNN/rom_i0_c3
8.75376e-05	4.82000e-06	0.00000e+00	9.23576e-05	/MNN/rom_i1_c1c2c3
9.54192e-05	4.82000e-06	0.00000e+00	1.00239e-04	/MNN/rom_i1_c3c4
8.26117e-05	4.82000e-06	0.00000e+00	8.74317e-05	/MNN/rom_i1_c4
2.38811e-04	7.48162e-04	4.18724e-09	9.86977e-04	/MNN/sr1
6.42825e-04	8.11614e-04	2.92020e-09	1.45444e-03	/MNN/sr2
6.42825e-04	3.27117e-04	3.37166e-09	9.69946e-04	/MNN/sr3
6.42825e-04	2.04922e-04	1.59202e-08	8.47764e-04	/MNN/sr4
6.42825e-04	9.58145e-05	3.46618e-09	7.38643e-04	/MNN/sr5
4.04615e-04	6.85947e-05	1.59238e-09	4.73211e-04	/MNN/sr6
6.18518e-04	1.94831e-04	4.49432e-09	8.13353e-04	/MNN/sr7
6.18518e-04	1.93883e-04	4.54909e-09	8.12406e-04	/MNN/sr8
6.18518e-04	1.93148e-04	4.49432e-09	8.11671e-04	/MNN/sr9
6.18518e-04	1.92177e-04	4.53341e-09	8.10699e-04	/MNN/sr10
6.42161e-03	3.43430e-03	4.97761e-08	9.85596e-03	Total

Table 3.14 When the CEB switch control is set to "on" and includes power-gating, the power consumption of each memory module, under the condition of running to ImproveNet1.

Leakage	Internal	Switching	Total	Instance
8.05208e-05	1.87962e-04	4.28640e-11	2.68483e-04	/MNN/rom_d_c1c2c3
9.46538e-05	2.22982e-05	5.25200e-11	1.16952e-04	/MNN/rom_d_c3c4
1.14138e-04	6.90922e-06	1.56928e-10	1.21047e-04	/MNN/rom_d_c4
8.75376e-05	1.49389e-04	0.00000e+00	2.36927e-04	/MNN/rom_i0_c1c2c3
8.26117e-05	4.82000e-06	0.00000e+00	8.74317e-05	/MNN/rom_i0_c3
8.75376e-05	4.82000e-06	0.00000e+00	9.23576e-05	/MNN/rom_i1_c1c2c3
9.54192e-05	4.82000e-06	0.00000e+00	1.00239e-04	/MNN/rom_i1_c3c4
8.26117e-05	4.82000e-06	0.00000e+00	8.74317e-05	/MNN/rom_i1_c4
1.82270e-04	4.98137e-04	4.18724e-09	6.80411e-04	/MNN/sr1
6.42825e-04	8.11614e-04	2.96694e-09	1.45444e-03	/MNN/sr2
6.11250e-04	2.98376e-04	3.47188e-09	9.09630e-04	/MNN/sr3
5.53013e-04	1.53872e-04	1.64582e-08	7.06902e-04	/MNN/sr4
5.53013e-04	7.18541e-05	3.37015e-09	6.24871e-04	/MNN/sr5
3.29075e-04	4.66553e-05	1.54712e-09	3.75732e-04	/MNN/sr6
3.55381e-04	4.76124e-05	4.40115e-09	4.02998e-04	/MNN/sr7
3.55381e-04	4.73754e-05	4.87455e-09	4.02762e-04	/MNN/sr8
3.55381e-04	4.73742e-05	4.67211e-09	4.02760e-04	/MNN/sr9
3.55381e-04	4.71313e-05	4.63619e-09	4.02517e-04	/MNN/sr10
5.01800e-03	2.45584e-03	5.08378e-08	7.47389e-03	Total

Overall, it is evident that implementing power management—specifically by enabling the chip enable pin (cen) and incorporating power-gating—results in a significant reduction in power consumption compared to the architecture without any power management. Under the worst-case scenario in this architecture, where the system needs to operate ImproveNet1, the total power consumption is reduced by approximately 29.76%, as shown in Table 3.15.

Table 3.15 Power consumption at each stage at 555 MHz with and without power management control.

Method	Total (mW)		
	doorNet	ImproveNet0	ImproveNet1
CEN_switch_ON & Power gating	17.37	35.5	53.86
CEN_switch_ON & No Power gating	20.06	38.46	56.23
CEN_switch_OFF & No Power gating	37.68	57.49	76.69
CEN_switch_ON & Power gating vs CEN_switch_OFF & No Power gating Reduction (%)	53.9%	38.25%	29.76%

Additionally, Equations 3.1 to 3.3 provide the computation time required for each of the three stages, demonstrating that the proposed hardware architecture can achieve a frame rate ranging from 9 to 27 FPS (frames per second).

$$20,359,111 \times 1.8 \text{ ns} = 36,646,399.8 \text{ ns} \approx 0.036 \text{ s} \quad (3.1)$$

$$40,180,545 \times 1.8 \text{ ns} = 72,324,981.0 \text{ ns} \approx 0.072 \text{ s} \quad (3.2)$$

$$60,274,946 \times 1.8 \text{ ns} = 108,494,902.8 \text{ ns} \approx 0.108 \text{ s} \quad (3.3)$$

The layout of the HDSC model chip proposed in this paper is shown in Figure 3.10, with the locations of the SRAM and ROM clearly marked. The chip area is confirmed to be $1400 \times 1400 \mu\text{m}^2$.

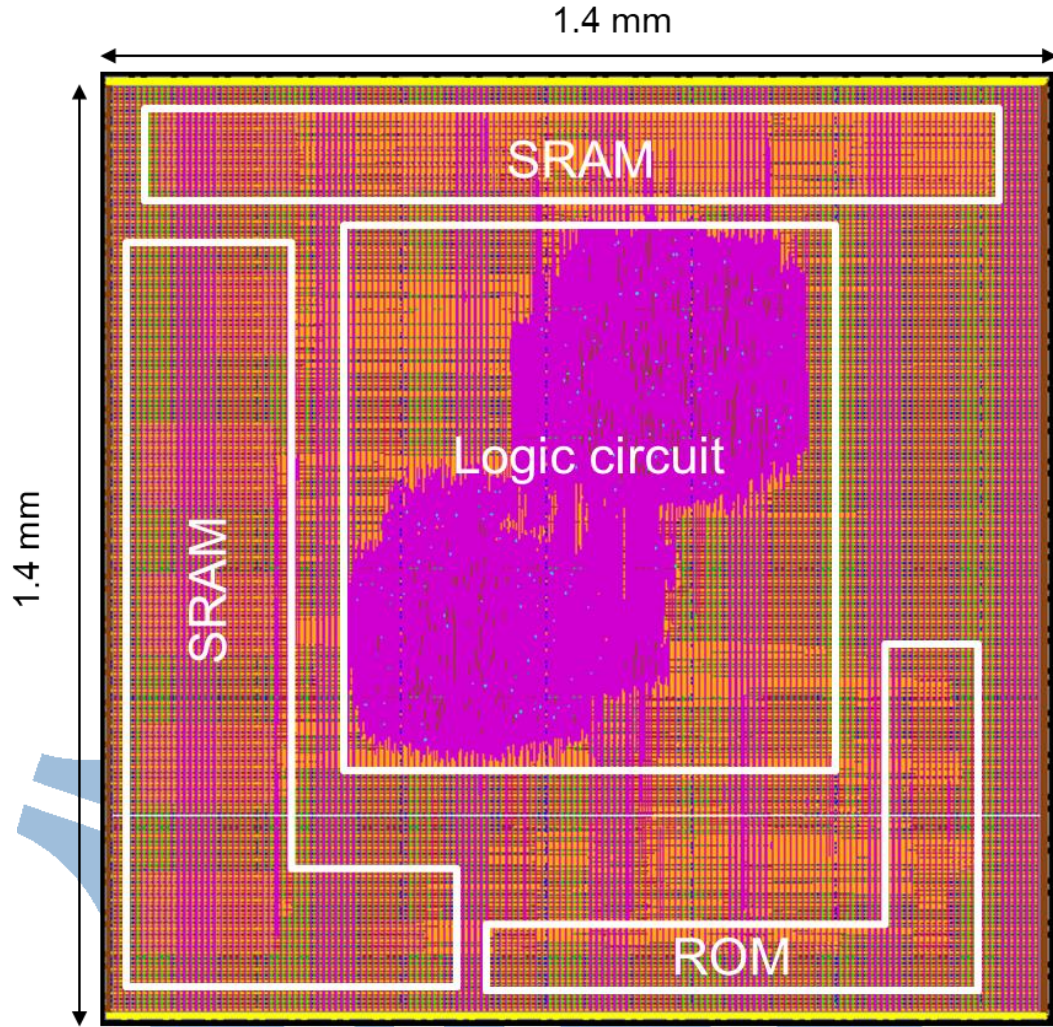


Figure 3.10 The layout of the HDSC model chip.

Before performing convolution operations, it is necessary to specify the access location of the ROM through the circuit. In the next cycle, the corresponding index value is retrieved from the ROM for table lookup. Once the position information is received from the ROM, the operation of obtaining the index value must be completed within half a cycle, but the ROM read operation requires a certain wait time. According to Table 3.16, the read times for each ROM used in this HDSC hardware architecture vary. Among them, rom_d_c4, which has the most bits, has the longest access time, requiring 0.7848 ns per access. This is also the critical path in this hardware architecture.

Table 3.17 presents the implementation results of the proposed HDSC architecture

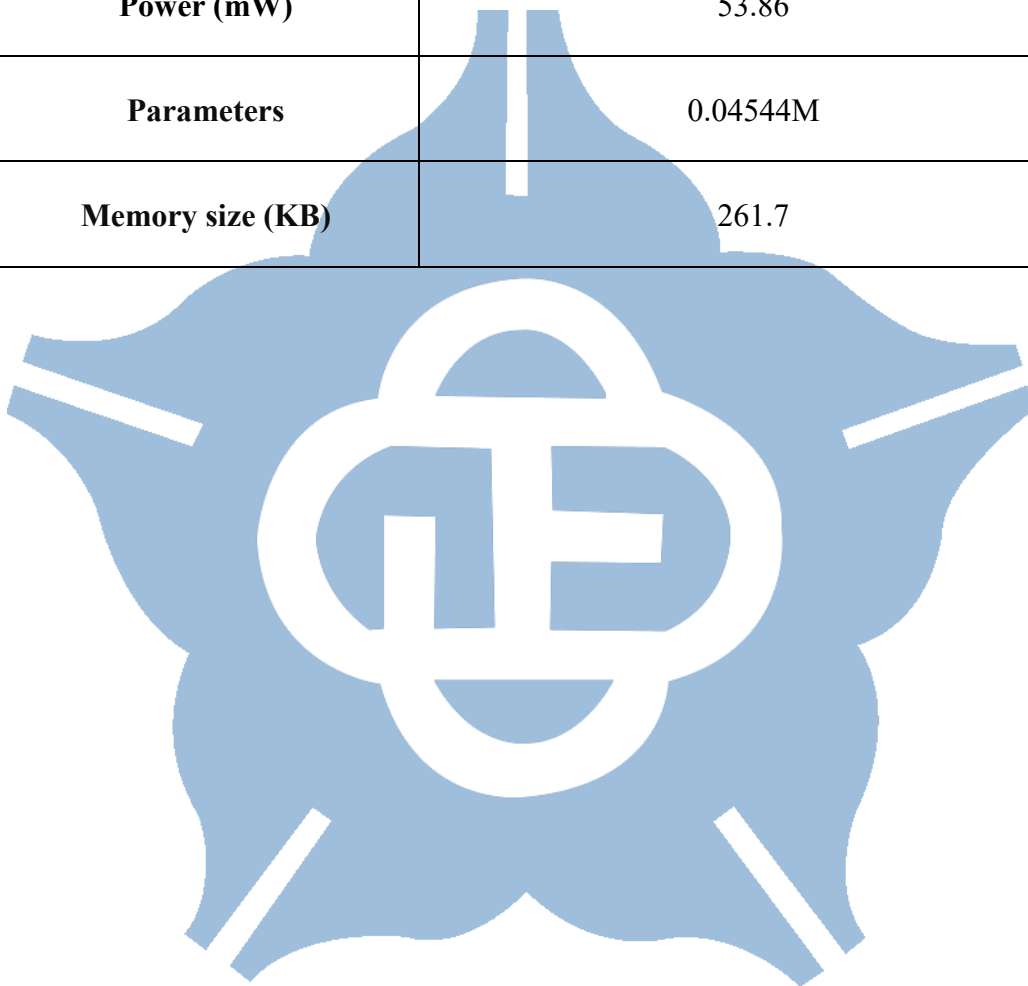
hardware design. At a frequency of 555 MHz, the power consumption is 53.86 mW, the total memory size is 261.7 KB, and the number of parameters is 0.04544M. These results were achieved using TSMC's 16nm CMOS process.

Table 3.16 Time required for accessing ROM.

The name of ROM	The size of ROM (bits)	The access time of ROM (ns)
rom_d_c1c2c3	42,624	0.7235
rom_d_c3c4	51,840	0.7417
rom_d_c4	73,728	0.7848
rom_i0_c1c2c3	42,624	0.7235
rom_i0_c3	36,864	0.7122
rom_i1_c1c2c3	42,624	0.7235
rom_i1_c3c4	51,840	0.7417
rom_i1_c4	36,864	0.7122

Table 3.17 The HDSC hardware implementation results

	The proposed architecture
Technology	TSMC 16nm CMOS
Frequency (MHz)	555
Power (mW)	53.86
Parameters	0.04544M
Memory size (KB)	261.7



3.5 Summary

Table 3.18 Comparison with other methods.

	[37] Vis Comput' 22	[38] IEEE Access' 23	[15] ICASSP' 22	[33] Sensors' 22	[34] IEEE Access' 24	Proposed work
Dataset	FER2013	FER2013plus	FER2013	FER2013	FER2013	FER2013
Number of classes	7	6	7	7	7	7
Architecture	Dual CNNs	Transfer- learning with CNNs	SDNET	Light-FER	CLCM	HDSC
Hardware information	Nvidia Xavier	Nvidia K80/T4	NVIDIA GTX1050	Jetson Nano	Intel i7-6700k +GTX 1080TI GPU	16nm ASIC
Number of parameters	1.08M	1.49M	2.17M	N/A	2.3M	0.04544M
FPS	60	N/A	N/A	2.5	20	27 (doorNet) 13 (ImproveNet0) 9 (ImproveNet1)
FLOPs	N/A	N/A	5.38M	N/A	N/A	0.088M
Accuracy	72.27%	82.1%	67.08%	69.87 %	63%	71.70%

Note: The FER2013plus dataset is an enhanced version of FER2013, where low-quality images have been removed, and incorrect labels have been corrected.

Table 3.18 compares the results of studies using the same dataset and contrasts them with the hierarchical model proposed in this thesis. In reference [37], the

researchers employed a Dual CNNs architecture on the FER2013 dataset, achieving higher accuracy than the hierarchical model proposed here, albeit with a significantly increased number of parameters. Despite the superior FPS performance of this architecture, the platform's power consumption is considerably higher, which limits its applicability in robotic systems.

In reference [38], a standard transfer learning approach combined with a CNNs architecture was used, trained on the FER2013plus dataset. The exclusion of the neutral class in the final testing phase resulted in an accuracy that appears notably higher. However, the parameter count of this method remains significantly higher, leading to challenges in achieving real-time applications. In reference [33], neither the FPS nor the accuracy surpasses the approach presented in this thesis. Additionally, the hardware platform used in their experiments, Jetson Nano, consumes more power compared to the platform used in this study.

The accuracy in reference [15] is lower than that of the method presented in this thesis, with the parameter count being approximately 48 times higher. Although reference [34] demonstrates relatively better FPS performance, the power consumption of its operating platform is significantly higher, and its accuracy is inferior, with the parameter count also exceeding that of the proposed method by several times.

Chapter 4 Conclusion and Future Works

4.1 Conclusion

This thesis proposes a Hierarchical Depthwise Separable Convolutional Neural Network (HDSC) model for emotion robots to recognize facial expressions. The model is composed of three sub-models and addresses the issue of data imbalance during training. Experimental results based on the FER2013 dataset demonstrate that this design achieves an accuracy of 69.61% in the software phase with only 0.045M parameters, significantly reducing computational load. The architecture's FLOPs range from 0.0392M in the best case to 0.088M in the worst case, reducing the computation by approximately 55.45%.

Moreover, to minimize hardware resource consumption, the weights are quantized to 9 bits using DoReFaNet. Further quantization of the activation functions reduces memory usage. Additionally, by simplifying the parameters of batch normalization and applying fixed-point quantization, memory usage is drastically reduced. Finally, by quantizing the values post batch normalization, maximum reduction in hardware resources is achieved, reaching an accuracy of 71.70% at this stage.

In conclusion, regarding the hardware implementation, the HDSC hardware design employs power gating techniques to analyze and shut down idle memory blocks, thereby reducing power consumption. The proposed HDSC hardware design is realized using TSMC's 16nm CMOS technology, achieving an operational frequency of up to 555 MHz with a power consumption of 53.86 mW.

4.2 Future Work

Edge devices frequently encounter resource constraints, which prevent many applications from running smoothly on these devices. This thesis proposes a hierarchical lightweight neural network and employs quantization and fixed-point techniques to significantly reduce computational resource requirements. However, there is still considerable room for improvement in terms of accuracy. Therefore, a critical research direction for the future is how to simplify models to reduce computational resource demands while maintaining accuracy.

In recent years, with the rise of Transformer, more and more researchers have attempted to integrate Transformer models to enhance the accuracy of their models, achieving notable results. However, there is still room for improvement in terms of accuracy with the current architecture. Therefore, combining the architecture with Transformers to increase accuracy is a promising direction for future efforts.

Besides emotional robots, facial recognition has numerous applications, such as in crime detection or monitoring the mental state of drivers. These applications require real-time recognition to prevent tragedies. Consequently, ensuring that facial recognition can be conducted more quickly and accurately is a crucial direction for future development.

By analyzing memory usage patterns, it is possible to design power gating control. However, this power control increases the overall complexity of circuit design. Additionally, considerations need to be made regarding whether data should be retained, discarded, or preserved at a low voltage state when memory is turned off, which will increase the overall cost of circuit design. Therefore, achieving optimal low-power performance through power gating control design is a significant research topic for the future.

Reference

- [1] Charles Robert Darwin, “The expression of the emotions in man and animals,” 1st ed. London: John Murray, 1872.
- [2] Paul Ekman and Wallace V. Friesen, “Constants across cultures in the face and emotion,” *Journal of Personality and Social Psychology*, vol. 17, no. 2, pp. 124-129, 1971.
- [3] Maja Pantic and Leon J. M. Rothkrantz, “Automatic analysis of facial expressions: the state of the art,” *Journal of IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 12, pp. 1424-1445, Dec. 2000.
- [4] Md Tauhid Bin Iqbal, M. Abdullah-Al-Wadud, Byungyong Ryu, Farkhod Makhmudkhujaev, and Oksam Chae, “Facial expression recognition with neighborhood-aware edge directional pattern (NEDP),” *IEEE Transactions on Affective Computing*, vol. 11, no. 1, pp. 125–137, Jan. 2020.
- [5] Farkhod Makhmudkhujaev, M. Abdullah-Al-Wadud, Md Tauhid Bin Iqbal, and Byungyong Ryu, “Facial expression recognition with local prominent directional pattern,” *Signal Processing: Image Communication*, vol. 74, pp. 1-12, May 2019.
- [6] Evangelos Sariyanidi, Hatice Gunes, and Andrea Cavallaro, “Automatic analysis of facial affect: A survey of registration, representation, and recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 37, no. 6, pp. 1113-1133, Jun. 2015.
- [7] Changbo Hou, Jiajun Ai, Yun Lin, Chenyang Guan, Jiawen Li, and Wenyu Zhu, “Evaluation of online teaching quality based on facial expression recognition,” *Journal of Future Internet*, vol. 14, no. 6, Art no. 177, Jun. 2022.
- [8] Zhe Li, Tianfan Zhang, Xiao Jing, and Youning Wang, “Facial expression-based analysis on emotion correlations, hotspots, and potential occurrence of urban

- crimes,” *Journal of Alexandria Engineering*, vol. 60, no. 1, pp. 1411-1420, Feb. 2021.
- [9] Ibtissam Saadi, Douglas W. Cunningham, Abdelmalik Taleb-Ahmed, Abdenour Hadid, and Yassin El Hillali, “Driver’s facial expression recognition: A comprehensive survey,” *Journal of Expert Systems With Applications*, vol. 242, Art no. 122784, May 2024.
- [10] Yousif Khaireddin and Zhuofa Chen, “Facial emotion recognition: state of the art performance on FER2013, arXiv:2105.03588 [cs.CV] ,” *arXiv.org*, May 2021.
- [11] Kuang Liu, Mingmin Zhang, and Zhigeng Pan, “Facial expression recognition with CNN ensemble,” in *Proceedings of International Conference on Cyberworlds (CW)*, Sep. 2016, pp. 163-166.
- [12] Christopher Pramerdorfer and Martin Kampel, “Facial expression recognition using convolutional neural networks: state of the art, arXiv:1612.02903 [cs.CV] ,” *arXiv.org*, Dec. 2016.
- [13] Gede Putra Kusuma, Jonathan Jonathan, and Andreas Pangestu Lim, “Emotion recognition on FER-2013 face images using fine-tuned VGG-16,” *Advances in Science, Technology and Engineering Systems Journal*, vol. 5, no. 6, pp. 315-322, Jun. 2020.
- [14] Octavio Arriaga, Matias Valdenegro-Toro, and Paul Plöger, “Real-time convolutional neural networks for emotion and gender classification,” in *Proceedings of 27th European Symposium on Artificial Neural Networks (ESANN)*, Apr. 2019, pp. 221-226.
- [15] Lifang Zhou, Siqin Li, Yi Wang, and Junlin Liu, “SDNET: lightweight facial expression recognition for sample disequilibrium,” in *Proceedings of IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, May 2022, pp. 2415-2419.

- [16] Ozioma Collins Oguine, Kanyifeechukwu Jane Oguine, Hashim Ibrahim Bisallah, and Daniel Ofuani, “Hybrid facial expression recognition (FER2013) model for real-time emotion classification and prediction, arXiv:2206.09509v2 [cs.CV],” *arXiv.org*, Aug. 2022.
- [17] Arielle AJ Scoglio, Erin D Reilly, Jay A Gorman, and Charles E Drebing, “Use of social robots in mental health and well-being research: systematic review,” *Journal of Medical Internet Research*, vol. 21, no. 7, Art no. e13322, Jul. 2019.
- [18] Yann Lecun, Léon Bottou, Yoshua Bengio, and Patrick Haffner, “Gradient-based learning applied to document recognition,” in *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278-2324, Nov. 1998.
- [19] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton, “ImageNet classification with deep convolutional neural networks,” in *Proceedings of Neural Information Processing Systems (NIPS)*, Dec. 2012.
- [20] Karen Simonyan and Andrew Zisserman, “Very deep convolutional networks for large-scale image recognition, arXiv:1409.1556v6 [cs.CV],” *arXiv.org*, Apr. 2015.
- [21] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jun. 2016, pp. 770-778.
- [22] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam, “MobileNets: efficient convolutional neural networks for mobile vision applications, arXiv:1704.04861v1 [cs.CV],” *arXiv.org*, Apr. 2017.
- [23] Abhinav Goel, Sara Aghajanzadeh, Caleb Tung, and Shuo-Han Chen, “Modular neural networks for low-power image Classification on embedded devices,” *ACM Transactions on Design Automation of Electronic Systems*, vol. 26, no. 1, Art no. 1, pp. 1-35, Jan. 2021.

- [24] Mohammad Reza Nayeri, Babak Nadjar Araabi, and Behzad Moshiri, "Fault detection and isolation of gas turbine: Hierarchical classification and confidence rate computation," *Journal of the Franklin Institute*, vol. 359, no. 17, pp. 10120-10144, Nov. 2022.
- [25] Kaidi Cao, Colin Wei, Adrien Gaidon, Nikos Arechiga, and Tengyu Ma, "Learning imbalanced datasets with label-distribution-aware margin loss, arXiv:1906.07413 [cs.LG]," *arXiv.org*, Oct. 2019.
- [26] Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie, "Class-balanced loss based on effective number of samples," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, Jan. 2019, pp. 9268-9277.
- [27] Song Han, Huizi Mao, and William J. Dally, "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding, arXiv:1510.00149v5 [cs.CV]," *arXiv.org*, Feb. 2016.
- [28] Jimmy Ba and Rich Caruana, "Do deep nets really need to be deep?," in *Proceedings of Neural Information Processing Systems (NIPS)*, Dec. 2014.
- [29] Matthieu Courbariaux, Itay Hubara, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio, "Binarized neural networks: Training deep neural networks with weights and activations constrained to +1 or -1, arXiv:1602.02830v3 [cs.LG]," *arXiv.org*, Mar. 2016.
- [30] Haotong Qin, Ruihao Gong, Xianglong Liu, Mingzhu Shen, Ziran Wei, Fengwei Yu, and Jingkuan Song, "Forward and backward information retention for accurate binary neural networks," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2250-2259, Jun. 2020.
- [31] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan, "PACT: parameterized

- clipping activation for quantized neural networks, arXiv:1805.06085v2 [cs.CV],” *arXiv.org*, Jul. 2018.
- [32] Shuchang Zhou, Yuxin Wu, Zekun Ni, Xinyu Zhou, He Wen, and Yuheng Zou, “DoReFa-Net: Training low bitwidth convolutional neural networks with low bitwidth gradients, arXiv:1606.06160v3 [cs.NE],” *arXiv.org*, Feb. 2018.
- [33] Alexander M. Pascual, Erick C. Valverde, Jeong-in Kim, Jin-Woo Jeong, Yuchul Jung, Sang-Ho Kim, and Wansu Lim, “Light-FER: A Lightweight Facial Emotion Recognition System on Edge Devices,” *Sensors*, vol. 22, no. 23, Art no. 9524, Nov. 2022.
- [34] Mustafa Can Gursesli, Sara Lombardi, Mirko Duradoni, Leonardo Bocchi, Andrea Guazzini, and Antonio Lanata, “Facial Emotion Recognition (FER) Through Custom Lightweight CNN Model: Performance Evaluation in Public Datasets ,” *IEEE Access*, vol. 12, pp. 45543-45559, Mar. 2024.
- [35] Fatma Mazen Ali Mazen, Ahmed Aly Nashat, and Rania Ahmed Abdel Azeem Abul Seoud, “Real Time Face Expression Recognition along with Balanced FER2013 Dataset using CycleGAN,” *International Journal of Advanced Computer Science and Applications(IJACSA)*, vol. 12, no. 6, pp 1-12, 2021.
- [36] Zhang Bingzhen, Qiao Xiaoming, Yang Hemeng, and Zhou Zhubo, “A Random Forest Classification Model for Transmission Line Image Processing,” in *Proceedings of 2020 15th International Conference on Computer Science & Education (ICCSE)*, pp.613–617, Aug. 2020.
- [37] Sumeet Saurav, Prashant Gidde, Ravi Saini, and Sanjay Singh, “Dual integrated convolutional neural network for real-time facial expression recognition in the wild,” *The Visual Computer*, vol. 38, pp. 1083–1096, Mar. 2022.
- [38] Dina Shehada, Ayad Turkey, Wasir Khan, Bilal Khan, and Abir Hussain, “A Lightweight Facial Emotion Recognition System Using Partial Transfer Learning

for Visually Impaired People,” *IEEE Access*, vol. 11, pp. 36961-36969, Apr. 2023.

